HLayout

Thick-Film Hybrid Layout CAD Tool

User Manual

Covers: layout editing · layers · the command bar & Python scripting · mask operations & DRC · the Thick-Film Resistor Designer · the Trim Simulator · GDS library merging · Gerber/DXF export

Reference build: hlayout_app.py · October 2026

Table of Contents

1. Introduction....3
2. Getting Started....5
3. The Main Window....6
4. File Formats....13
5. Drawing and Editing....14
6. Layers....16
7. Technology Setup....17
8. Attached Libraries and Reusable Parts....19
9. The Command Bar and Python Scripting....21
10. Mask Operations and Design Rule Check....28
11. The Thick-Film Resistor Designer....34
12. The Resistor Trim Simulator....37
13. Merging GDS Libraries....39
14. Manufacturing Export....40
15. Example Generator Scripts....44
16. Keyboard Shortcut Reference....46
17. Appendix: Python Scripting API by Category....47
18. Assembly Drawings: RESTABLE, PINTABLE, SHEET and ASSEMBLY....51
19. Python API Cookbook....54
20. Thermal Analysis....85
21. Autorouting with Freerouting....89

1. Introduction

HLayout is a Python/Tkinter CAD application for designing thick-film hybrid microelectronic circuits — the kind of layout work that involves printing resistor inks, conductor traces, dielectric layers, and glaze overprints onto a ceramic substrate, alongside standard surface-mount component footprints. It reads and writes standard GDSII (.gds) files, supports up to 64 independent layers, and can be driven with the mouse, a short AutoCAD-style command language, or full Python scripting — all three operate on the same underlying design, so you can mix and match freely.

This manual walks through every menu, dialog, toolbar control, and built-in tool in the application, organized by what you are trying to do rather than by source-code structure. For the exhaustive, searchable syntax of every command-bar command and every Python API method, use the in-app Help → Command Reference (F1), which opens docs/help.html — this manual is the companion guide that explains what each feature is for and how its options behave.

1.1 Key capabilities at a glance

- GDSII-native design database: 1 DBU = 1 micron, up to 500 × 500 mm design area, 64 layers each with its own color, fill/stipple pattern, and name.
- Mouse-driven drawing and editing (rectangles, polygons, paths, circles, text, cell instances, arrays, 2-layer routing with vias) with full undo/redo.
- A short command-bar language and an equivalent Python scripting API (the Session object), so anything you can click you can also type or script.
- Up to 2 attached reference libraries for reusable parts (vias, pads, packages), plus the ability to merge two whole GDS libraries into one.
- A Thick-Film Resistor Designer that sizes resistors from value/tolerance/power and a chosen ink, and a laser-trim simulator that models plunge and L-cut trimming with a live resistance/sensitivity/heat-map view.
- Thermal Analysis that estimates the temperature of every resistor, die and powered component from the layout, the film stack and a power per part.
- An Assembly Drawing generator that lays out a titled drawing sheet, a scaled top view and the pin and resistor tables in one step, plus banded thermal maps with a legend and mm axes.
- Mask Operations (oversize/undersize/boolean layer combining) and a Design Rule Check (DRC) engine, both built on shapely.
- Autorouting with Freerouting for two-conductor multilayer designs and for one-conductor chip-and-wire hybrids, where every wire bond's substrate end becomes a bond finger for the router.
- Manufacturing export to Gerber (RS-274X) and DXF (AutoCAD R12), plus example scripts that generate ready-to-use SOT package, ceramic chip-capacitor, and LCC package footprint libraries.

1.2 How this manual is organized

Chapters 2–4 get you oriented: installing and launching the app, a guided tour of the main window, and the file formats. Chapters 5–8 cover day-to-day layout work: drawing, editing, layers, technology setup, and attached libraries. Chapter 9 documents the command bar and the Python scripting layer that underlies everything. Chapter 10 covers Mask Operations, DRC, netlist extraction, dielectric fill and the top view. Chapters 11–12 are the Thick-Film Resistor Designer and its Trim Simulator. Chapter 13 covers merging GDS libraries and Chapter 14 manufacturing export. Chapter 15 documents the bundled example generator scripts, Chapter 16 is a keyboard-shortcut quick reference, and Chapter 17 is an appendix mapping out the

Python scripting API by category. Chapter 18 covers assembly drawings, Chapter 19 is a cookbook of tested Python API examples, Chapter 20 covers Thermal Analysis, and Chapter 21 covers autorouting with Freerouting, including chip-and-wire hybrids.

2. Getting Started

2.1 Requirements

HLayout is a Python application. Its GUI is built on Tkinter, which ships with the standard Python installer on Windows and macOS; on Debian/Ubuntu Linux you may need to install it separately:

```
sudo apt install python3-tk
```

Install the Python package dependencies from the repository root:

```
pip install -r requirements.txt
```

The dependency list and what each one is used for:

Package	Used for
gdstk >= 0.9	Reading and writing GDSII (.gds) files — required for all normal use.
shapely >= 2.0	Mask Operations and Design Rule Check (DRC). Only needed if you use those tools.
numpy >= 1.24	The Resistor Trim Simulator's finite-difference solve. Imported lazily — the rest of the app runs fine without it.
scipy >= 1.10	The Resistor Trim Simulator's sparse linear solve. Same lazy-import note as numpy.

If numpy/scipy are missing and you open the Trim Simulator, HLayout shows a friendly dialog telling you to `pip install numpy scipy` rather than crashing.

2.2 Launching the application

From the repository root, run:

```
python hlayout_app.py                # start with a blank library
python hlayout_app.py mydesign.gds   # open an existing GDSII file directly
```

The main window opens at title “HLayout — Thick-Film Hybrid Layout CAD.” A blank start gives you a library named UNTITLED with one cell named TOP.

2.3 The design database, briefly

- 1 database unit (DBU) = 1 micron (0.001 mm) exactly — this matches most external GDSII libraries and means Resistor Designer body dimensions in DBU can be used directly as microns.
- Maximum design size is 500 × 500 mm.
- There are 64 layers (numbered 0–63), each independently configurable: on/off visibility, color, fill on/off, one of 9 stipple patterns, and an editable name.
- A design can have any number of cells; cells can instance other cells (single instances or rectangular arrays), so hierarchical designs are fully supported.
- Coordinates you type into commands and scripts are in DBU unless a command explicitly says millimetres (most dialogs, being aimed at humans, work in mm).

3. The Main Window

The main window is divided into: a menu row across the top, a bottom toolbar just above the canvas, the drawing canvas itself (with a command bar and status bar beneath it), and a fixed-width panel on the right with Layers / Properties / Script Console tabs.

3.1 Menu bar

The menu bar is drawn with themed dropdown buttons rather than the operating system's native menu, so menu text stays a consistent size on every platform. The menus are File, Setup, View, Create, Edit, Tools, and Help, plus a plain About button on the right that shows the app version and a one-line capability summary.

File menu

Item	Shortcut	What it does
New	Ctrl+N	Starts a blank library (UNTITLED, one cell TOP); clears undo history and detaches any attached libraries.
Open GDS...	Ctrl+O	Opens a .gds file. If a matching .hlaux sidecar file exists alongside it, its technology stack and attached libraries are restored automatically.
Save GDS...	Ctrl+S	Saves to the current file path (or prompts for one). Writes the .gds plus a .hlaux sidecar capturing technology and attached libraries.
Save As...	Ctrl+Shift+S	Always prompts for a location (pre-filled with the current file's folder/name), then saves there and makes it the new current file, unlike Save GDS..., which reuses the current file once one is set. Use it to save a copy under a new name or move the design to a different folder; the old file on disk is left untouched.
Save As Library...	—	Prompts for a library name and a path, then writes a bare .gds with no sidecar — intended for reuse as an attachable parts library, not as your working design file.
Archive Design...	—	Bundles the design's .gds, its .hlaux sidecar (if any), and every attached library .gds still reachable at its recorded path into one .zip file, for handoff or backup. Requires the design to already be saved.
Merge GDS Libraries...	—	Combines two .gds files, cell for cell, into a third output file. See Chapter 14.
Gerber Export Sets...	—	Opens the Gerber batch export window (same tool as Tools → Gerber Export Sets...).
DXF Export Sets...	—	Opens the DXF batch export window (same tool as Tools → DXF Export Sets...).
Export to FreeCAD (3D Macro)...	—	Writes a FreeCAD Python macro that reconstructs the design as 3D solids (per-layer extrusions using the Layer Thicknesses set in Technology Properties, wire bonds as raised loops) for a 3D preview in FreeCAD. Placed components are auto-matched against the component STEP library and substituted as real 3D bodies, with a preview of every match before anything is written. See 14.3.
Build FreeCAD Component Library...	—	Writes a second macro that, run in FreeCAD, builds a small library of generic 3D bodies (ceramic chip capacitors, the 20-pin hybrid

Item	Shortcut	What it does
		flatpack, the SOT gull-wing family) and exports each as its own STEP file into a shared folder used by every design. Never overwrites a STEP file that already exists. See 14.3.
Component Library Manager...	—	Visual manager for the cell-to-STEP-file mapping: one row per placed component cell, showing the STEP file and rotation it is tied to and whether that came from a manual pin or an auto-match. Set STEP File..., Clear Override, and Refresh. See 14.3.
Exit	—	Closes the application.

Setup menu

Item	What it does
Attach Library...	Picks a .gds file and attaches it as a reference part library (maximum 2 attached at once).
Attached Libraries...	Info dialog listing each attached library's index, name, cell count, and file path.
Layers...	Switches the right-hand panel to the Layers tab.
Snap to Grid	Checkbutton — mirrors the bottom toolbar's own Snap to grid checkbox.
Grid Spacing...	Dialog with a Grid spacing (mm) spinbox; also updates the snap pitch to match.
Path Style...	Dialog setting the default path Width (mm) and End cap (round / square / flush) used by the Path drawing tool.
Circle Segments...	Dialog choosing how many segments (8 / 16 / 32 / 64) approximate a drawn circle.
Technology Properties... (Route tab)	Route tab of the one tabbed Technology Properties dialog: the two conductor layers the Route (2-Layer) tool alternates between, and the via cell it places on Space. A “New Via Cell…” button opens the Via Wizard here, and the tab saves even with no via cell chosen, so a missing via cell no longer traps the dialog. The other tabs are Resistors, Ref-Des (7.1), Wire Bonds (8.5) and Dielectric (10.4); one OK validates and saves them all.
Technology Properties... (Resistors tab)	Dialog setting the default overlap (A), pad length (B), side space (C), and glaze oversize (D) dimensions, plus default ink/conductor/glaze/documentation layers, used whenever a resistor is generated without overriding them.
FreeCAD Component Library Folder...	Views or changes the shared folder Build FreeCAD Component Library... writes into and Export to FreeCAD matches against. Asked for once, the first time either feature needs it; remembered from then on.
Remap Layers...	Dialog for bulk-moving all shapes from one layer to another, across the whole library or one named cell.
Load Technology File...	Opens a .json technology file and applies its layer stack and tech parameters to the current design in place.
Save Technology File...	Saves the current layer stack and tech parameters to a .json file, independent of the design itself.
Orthogonal/45° Routing	Checkbutton (also in the bottom toolbar) — on by default. While on, every new Path or Route segment snaps to horizontal, vertical, or 45° from the last point placed, so traces come out on the conventional routing angles instead of an arbitrary drag angle.

View menu

Item	Shortcut	What it does
Fit	F	Fits the entire current cell in the viewport.
Zoom In	+	Zooms in around the view center.
Zoom Out	–	Zooms out around the view center.
Toggle Grid	G	Shows/hides the background dot grid.
Zoom Window	W	One-shot tool: drag a box on the canvas to zoom/pan to fill it, then automatically reverts to Select mode.
Pan	P	Drag to move the view (hand cursor); stays on until Esc or another tool. Pan Step → Left/Right/Up/Down (Ctrl+arrows) moves a quarter screen; Pan to Point... centres on typed X/Y (mm).
New View Window	—	Opens an independent window showing the same library and cell, sharing the same undo history as the main window — handy for viewing two parts of a design at once.

Create menu

A set of radio-button tool selectors — choosing one switches the canvas into that drawing mode until you pick another tool or press Escape/Select: Rectangle, Polygon, Path, Circle, Text, Cell Instance, Array Instance, then Route (2-Layer), Wire Bond, Create Cell from Selection... (wraps the current selection into a new cell, replacing it with one instance), and finally Find & Place Cell..., which opens a search dialog for locating and placing a cell from the current design or an attached library by name.

Edit menu

Undo and Redo; the Select and Measure modes; Edit Selected... and Delete Selected; Insert Vertex on Edge, Continue Path..., Number Pins..., Ref-Des to Pin 1 and Auto-Route Two Pins; Move Wire Bond and Delete Wire Bond; **Delete Via** (click a via: its pads, its dielectric window and any placed via cell go in one undo step); **Highlight Net** (click a pad or track: every pad and track on that net is highlighted and its net name is shown in the status bar — layers only connect through via windows in the dielectric) and Clear Net Highlight; then Rotate 90°, Copy..., Smash Selected to SRefs, Merge Selected... and Properties.

Item	Shortcut	What it does
Undo	Ctrl+Z	Steps back one checkpoint. Greyed out when there is nothing to undo.
Redo	Ctrl+Y (also Ctrl+Shift+Z)	Steps forward one checkpoint. Greyed out when there is nothing to redo.
Select	—	Switches the canvas to Select mode (radio button).
Measure	—	Click a start point, then a second, to read the distance between them (radio button); reverts to Select automatically once the second point is placed.
Edit Selected...	—	Opens the property-edit dialog for whatever is currently selected — equivalent to double-clicking it.
Delete Selected	Delete	Removes the current selection.
Insert Vertex on Edge	—	Adds a new point on the currently selected edge, at the spot you clicked to select it — works on both Path and Polygon edges.

Item	Shortcut	What it does
Continue Path...	—	Re-enters Path drawing pre-seeded with the selected path's own points, so you can click more points onto its end instead of starting a new path from scratch.
Number Pins...	—	Enters the click-to-increment pin-numbering tool: click each pad in physical pinout order and each click stamps the current pin number, then advances automatically. Prompts for a starting number (default 1); Esc stops and returns to Select. See §8.4.
Ref-Des to Pin 1	—	Moves the selected part's ref-des beside its pin 1, or every connector/header's (J, P, X, CN, CON, HDR) when nothing is selected. Automatic when a connector is placed or labelled.
Auto-Route Two Pins	—	Two-click auto-router: click a first pin, then a second, and a clear orthogonal trace is routed between them on the active Route conductor layer, around every other shape on it. Stays armed for the next pair; Esc exits.
Move Wire Bond	—	Click near the end of the bond wire you want to move (the closer end is picked) — a live preview then follows the cursor with the bond's O------O rubber-band, showing the new span before you click its destination; the other end stays put. The new span must still fall within min/max and the bond may not cross another. Stays armed; Esc exits. See §8.5.
Delete Wire Bond	—	Click on (or near) a bond wire to remove it — connector and both end discs. Stays armed; Esc exits.
Rotate 90°	R	Rotates the selection 90° counter-clockwise about its own centroid/origin; click again (or press R) to keep turning — four presses is a full turn. For an arbitrary angle, use the ROTATE <degrees> command bar command instead.
Copy...	—	Prompts for a dx,dy offset (in design units) and creates a copy of the selection at that offset.
Smash Selected to SRefs	—	Explodes a selected array instance (AREF) into individual, independently editable cell instances (SREFs).
Merge Selected...	—	Unions a window-selected group of same-layer Polygon/Rectangle shapes that touch or overlap into a single polygon — needs a dragged group selection, not a single click. Same as the command bar's UNION command.
Properties	—	Switches the right-hand panel to the Properties tab.

Tools menu

Script Console, Run Script File..., a separator, Command Bar (moves keyboard focus to the command-bar entry), another separator, then the built-in design tools: Thick-Film Resistor Designer..., Via Wizard..., Pad Wizard..., Chip Wizard..., Mask Operations..., Mask Operation Batch..., Top View..., Assembly Drawing... (§18.5), DRC..., Extract Netlist..., Autoroute (Freerouting)... (Chapter 21), Thermal Analysis... (Chapter 20), Generate Dielectric Fill..., Remove Dielectric Fill, and finally the same Gerber Export Sets... / DXF Export

Sets... entries found under File. Directly below Command Bar is MCP Server (Claude), a checkbox that starts and stops the MCP bridge (§9.3).

Help menu

Command Reference (F1) opens `docs/help.html` in your system's default web browser — a generated, searchable single-page reference covering every command-bar command and every Python API method with syntax, description, and a runnable example. If the file can't be found next to the application, an in-app scrollable text dialog with the same reference content is shown instead.

User Manual opens this manual as a PDF (docs/HLayout_User_Manual.pdf) in your system's PDF viewer, with a clickable outline of every chapter and section. If the PDF isn't there it opens the Word original (HLayout_User_Manual.docx) instead, and if neither can be found it says so. The installers include the PDF, so the menu works the same in an installed copy.

3.2 Bottom toolbar

A single row directly above the command bar, left to right:

- Fit, Select, Zoom Win, Pan, Grid (toggle) buttons — Select, Zoom Win and Pan set the draw mode (same as Edit → Select / View → Zoom Window / View → Pan); Fit and Grid are shortcuts to the equivalent View menu actions. (Zoom In/Zoom Out stayed on the View menu with +/− accelerators; they were dropped from this toolbar in favor of Zoom Win, which covers “zoom to an area” in one drag instead of repeated clicks.)
- New View — opens a second view window (same as View → New View Window).
- **Cell:** — a combobox that switches the current cell and fits the view to it on change.
- **Layer:** — a combobox choosing the active draw layer, shown as “N: name”; you can type ahead by either the layer number or its name.
- **Snap to grid — a checkbox, plus a live “Grid: N mm” readout and a Grid... button opening the Grid Spacing dialog.**
- **Ortho/45° routing — a checkbox (on by default) constraining new Path/Route segments to horizontal, vertical, or 45° from the last point placed; also in Setup → Orthogonal/45° Routing.**
- Delete Selected — same action as Edit → Delete Selected.
- A status-hint label (in blue) that shows usage help for whichever tool is currently active, such as routing status while the Route tool is in use.

3.3 The canvas

The canvas is where you draw and edit geometry directly with the mouse. It has a dark background with a minor/major dot grid (a major line every 10 minor lines, at true grid intersections only — not every dot in a major column or row); every grid point in view gets a dot at any zoom level, rendered as a single image rather than one item per dot so it stays fast.

Navigation

- Pan: middle-mouse drag or right-mouse drag.
- Zoom: mouse wheel zooms centered on the cursor; View → Zoom In/Out zoom by fixed steps (25% in, 20% out) centered on the view.
- Fit (F): fits the whole current cell with about 8% padding.

- Zoom Window (W): drag a box; the view zooms and pans to fill it with about 4% padding, then reverts to Select mode automatically.
- Pan (P): drag with the left button to move the view; the cursor becomes a hand and the tool stays on until Esc or another tool is picked. Middle- or right-drag still pans in every tool.
- Pan Step → Left / Right / Up / Down (Ctrl+arrow keys on the canvas): move the view a quarter of the visible width or height. Pan to Point...: type X and Y in mm (pre-filled with the current view centre) to centre the view there without changing the zoom.

Selecting and editing

- Click a shape or cell instance to select it (it highlights white); click-drag to move it; double-click to open its Edit dialog.
- Drag starting over empty space opens a dashed marquee (window-select) box for selecting multiple objects at once; clicking any member of an existing multi-selection and dragging moves the whole group together.
- Polygon, Path, and Rectangle shapes support direct reshaping: grabbing an individual vertex moves just that point, grabbing an edge moves the whole segment (vertex grabs take priority over edge grabs when both are close). Dragging a Rectangle's vertex or edge promotes it to a Polygon on that first drag — carrying over its layer and pin — so it reshapes exactly like a Polygon from then on. While Ortho/45° routing is on (the default), dragging an edge keeps it parallel to its original orientation (it moves only along its own normal, so an axis-aligned edge's neighbors stay orthogonal), and dragging a Path's loose end — extending it — snaps the new tip to horizontal/vertical/45° from its one neighbor, the same rule a freshly drawn segment gets.
- Arrow keys (Up/Down/Left/Right) nudge the current selection — a whole shape/cell instance, a window-selected group, or a grabbed vertex/edge — by one grid unit in that direction, regardless of whether Snap to grid is on (that toggle only affects mouse clicks/drags).
- When Snap to grid is on (the default), a cyan crosshair-and-circle marker previews exactly where your next click will land.

Drawing

Every draw tool (Rectangle, Polygon, Path, Circle, Text, Cell Instance, Array Instance, Route) shows a dashed cyan rubber-band preview as you place points; Polygon and Path additionally preview a “ghost” of the actual stroke width and layer color while you draw, so you can see the real result before committing.

- Escape cancels the shape currently being drawn.
- Enter or double-click finishes a polygon, path, or route.
- Backspace removes the last placed point while drawing a polygon or path.
- Space (only while routing) places an instance of the configured via cell at the current point and switches the active conductor layer to the other configured layer, so you can continue the trace on the new layer without stopping.

The Route (2-Layer) tool routes on one of two configured conductor layers (set, along with the via cell to use, in Setup → Technology Properties, Route tab). Pressing Space mid-route places an instance of that via cell at the current point and continues the route on the other conductor layer from there. When Ortho/45° routing is on (the default), every new segment is constrained to horizontal, vertical, or 45° from the last point placed.

Shapes are always drawn in ascending layer order (layer 0 first, layer 63 always on top) regardless of the order you actually drew them in, including geometry nested inside cell instances. Every mouse-driven edit —

move, rotate, copy, delete, vertex/edge reshaping — is a single undoable step, exactly like command-bar and scripted edits.

3.4 Command bar

A single **Command:** text entry with a Run button, docked at the bottom of the main window. It accepts the short command-bar language described in Chapter 10, or raw Python (unrecognized input falls through to the Python interpreter automatically). Up/Down arrow keys recall command history, shared with the Script Console tab.

3.5 Status bar

The bottom-most strip of the window. While drawing, it shows the live mouse position in millimeters; after any command or action it shows a one-line result or status message.

3.6 Right-hand panel

Layers tab

A scrollable table of all 64 layers with columns: On (a visibility checkbox), # (layer number), Name (click to rename), Color (click the swatch to open a color picker), Fill (checkbox), and Stipple (a combobox of 9 named fill patterns — solid, horizontal, vertical, diagonal-/, diagonal-\, cross, diagonal-cross, sparse dots, dense dots — disabled while Fill is off). All On / All Off buttons toggle every layer's visibility at once.

Properties tab

A read-only text pane describing whatever is currently selected: shape type, layer, coordinates, width, or — for a cell instance — the cell name and rotation. When a multi-object window-selection is active it lists up to 25 objects, one per line, followed by “...and N more” if there are more. Shows “Nothing selected.” when nothing is selected.

Script Console tab

A dark, read-only output pane above an entry field and Run button. It accepts the same input as the command bar — either the short command language or raw Python — and shares command history with it. The hint text reads: “Python or a command (RECT, CELL, ...) — same as Command Bar. F1 for full list.”

4. File Formats

HLayout works with several file types, each with a distinct purpose.

Format	Extension	Purpose
GDSII	.gds	The native, default format for your working design, for Save As Library output, and for attached libraries. Round-trips exactly.
HLayout sidecar	.hlaux (JSON)	Written automatically next to a .gds by File → Save GDS.../SAVE <path> (without a library name). Captures the full 64-layer technology stack, technology parameters (including Resistor Designer rules), and an embedded full copy of every attached library — so File → Open GDS... restores everything, not just geometry. Save As Library deliberately skips this file.
Technology file	.json	Setup → Load/Save Technology File...: just the layer stack and technology parameters, independent of any one design — useful for sharing a process setup across multiple projects.
Gerber	.gbr (RS-274X)	Manufacturing export only, one flattened file per visible non-empty layer, named <prefix>_L<NN>_<layer name>.gbr. See Chapter 15.
DXF	.dxf (AutoCAD R12 / AC1009)	Manufacturing/interchange export only, a single file with one DXF layer per HLayout layer. See Chapter 15.
CSV	.csv	Resistor Designer Import/Export only — a flat spreadsheet of resistor definitions and (on export) their computed solutions.

A note on precision: attaching or merging a library is lossless as long as its source file's DBU is 1 micron or coarser (the standard case) — HLayout's own DBU is fixed at exactly 1 micron and is never rescaled.

5. Drawing and Editing

This chapter expands on the Create and Edit menu tools introduced in Chapter 3, describing each drawing primitive and its options.

5.1 Drawing primitives (Create menu)

- **Rectangle** — click two opposite corners. Rectangles are always defined by their two corner points and cannot be reshaped by dragging a vertex or edge — use Edit → Edit Selected... to change corner coordinates directly.
- **Polygon** — click any number of points (minimum 3), then Enter or double-click to close and finish the shape. Backspace removes the last point while you're still drawing.
- **Path —** click a sequence of points describing a centerline; the path is drawn with a configurable width and end-cap style (round, square, or flush), set in Setup → Path Style... or overridden per-command. Each new segment snaps to horizontal/vertical/45° from the last point while Ortho/45° routing is on (Setup menu / bottom toolbar).
- **Circle** — click a center, then a point defining the radius. Circles are stored as regular polygons with 8, 16, 32, or 64 segments (Setup → Circle Segments..., default 32).
- **Text** — click an insertion point, then type the string in the prompt that appears. Text has a configurable height and font.
- **Cell Instance** — click a placement point for a single instance (an SRef) of a chosen cell — from the current design or an attached library — with optional rotation, magnification, and mirroring.
- **Array Instance** — click a placement point for a rectangular array (an ARef) of a chosen cell: you specify column/row counts and column/row spacing, plus the same rotation/mag/mirror options as a single instance.
- **Route (2-Layer) —** click a start point and subsequent route points on the currently active conductor layer (each segment ortho/45°-constrained the same as Path, unless that toggle is off); press Space at any point to place an instance of the configured via cell and continue the route on the other configured conductor layer; Enter or double-click finishes the route, Escape cancels the segment in progress.
- **Find & Place Cell...** — opens a search dialog — type part of a cell name to filter the list of cells available in the current design and any attached libraries, then place the match with a click.

5.2 Editing

Selected objects (Select mode, or the radio button under Edit) can be moved by dragging, or edited precisely through Edit → Edit Selected... (or double-click), which opens a dialog of the object's exact properties — layer, coordinates, width, cap style, rotation, and so on depending on the shape type.

- Edit → Rotate... prompts for a rotation angle in degrees, counter-clockwise, applied about the selection's own origin.
- Edit → Copy... prompts for a dx,dy offset in design units and creates a duplicate at that offset, leaving the original untouched.
- Edit → Smash Selected to SRefs breaks a selected array instance apart into individual, independently movable/editable cell instances — useful when you need to hand-edit or delete just one member of what was previously a uniform array.
- Edit → Delete Selected (or the Delete key, or the toolbar's Delete Selected button) removes the current selection.

Every edit — mouse-driven or via the command bar/scripting — is checkpointed for undo/redo (Ctrl+Z / Ctrl+Y or Ctrl+Shift+Z).

6. Layers

HLayout supports 64 layers, numbered 0–63. Every shape, path, and text object belongs to exactly one layer. Layers are purely a presentation/organization concept in the underlying GDSII data (a plain layer number) — the name, color, fill, and stipple pattern you assign are cosmetic and are saved either in the .hlaux sidecar (for your working design) or a .json technology file (for reuse across designs).

6.1 The Layers panel

Open it via Setup → Layers... or the Layers tab of the right-hand panel. Each row shows: On (visibility), the layer number, an editable Name, a clickable Color swatch, a Fill checkbox, and a Stipple pattern combobox (9 named patterns, disabled unless Fill is on). All On / All Off buttons toggle every layer at once. Shapes are always drawn in ascending layer order — layer 63 is always drawn on top of layer 0 — regardless of the order they were created in, including geometry nested inside placed cell instances.

6.2 Choosing the active draw layer

The Layer: combobox in the bottom toolbar sets which layer new shapes are drawn onto. It is labeled “N: name” and accepts type-ahead by either the number or the name.

6.3 Remapping layers

Setup → Remap Layers... moves every shape from one layer to another in bulk. The dialog lets you build a list of From → To layer pairs (Add Row / Remove Row) and choose a scope — the whole library, or a single named cell — before clicking Apply. This is also available as the `LAYER MOVE` command bar command or the `remap_layers` Python method.

7. Technology Setup

The Setup menu's remaining items configure defaults used throughout the app — grid, path style, circle resolution, routing, resistor geometry rules, and the technology file itself.

Dialog	Fields / options
Grid Spacing...	Grid spacing (mm) — also updates the snap pitch to match unless it has been set independently.
Path Style...	Width (mm); End cap: round, square, or flush — the defaults used by the Path drawing tool.
Circle Segments...	8, 16, 32, or 64 — how many straight segments approximate a drawn circle.
Technology Properties... (Route tab)	Layer A, Layer B (the two conductor layers the Route (2-Layer) tool alternates between), and Via cell name — the via cell placed as an instance when you press Space to drop a via. Build via cells with Tools → Via Wizard... first.
Technology Properties... (Resistors tab)	A (overlap, mm), B (pad length, mm), C (side space, mm), D (glaze oversize, mm), plus default layer pickers for the ink layer (fallback when a specific ink layer isn't configured), conductor pad layer, and glaze overprint layer (the reference-designator layer is on the Ref-Des tab, not here). These are the defaults RESGEN and the Resistor Designer's Place in Layout use unless overridden.
FreeCAD Component Library Folder...	Views or changes the shared folder Build FreeCAD Component Library... writes into and Export to FreeCAD matches against. Asked for once, the first time either feature needs it; remembered from then on.
Remap Layers...	See §6.3.
Load/Save Technology File...	Loads or saves the entire layer stack plus all of the above technology parameters as a standalone .json file, independent of any specific design — so you can carry one process setup across multiple projects.

Snap to Grid (checkbox, also in the bottom toolbar) toggles whether mouse clicks lock to the configured grid pitch; a cyan marker on the canvas previews where the next click will land whenever snapping is on.

7.1 Technology Properties

Setup → Technology Properties... collects every technology-file-backed setting into one tabbed window — Route, Resistors, Ref-Des, Wire Bonds, Dielectric, and a More… tab linking to the DRC, Mask Operation Batch and export-set editors. One OK validates and saves every tab; a tab that fails validation is selected and its message shown. It replaces the former separate Route Layers, Resistor Geometry Rules, Ref-Des Annotation Layer and Wire Bond Rules dialogs.

The Ref-Des tab holds the design’s one reference-designator layer: it is where a generated resistor bakes its ref-des Text, and the layer whose visibility gates the annotation the canvas floats at each placed part’s origin, so every ref-des shows and hides together (“(none)” leaves that annotation always shown). The Resistors tab no longer has its own documentation-layer field. The Dielectric tab stores the defaults for Tools → Generate Dielectric Fill... (see 10.4) — fill layer, margin, clearance, extra keep-out layers — and a blank fill layer here just means “not configured yet”. The Resistors tab’s E field (dielectric keep-out oversize) is covered in 11.7, the Wire Bonds tab in 8.5.

Technology files carry the process, not one design: File → Save Technology File... leaves out the Resistor Designer's resistor list, and loading a technology file never brings one in or replaces a design's own. A design keeps its resistors in its own saved file.

8. Attached Libraries and Reusable Parts

An attached library is a second (or third — up to 2 at once) .gds file you keep open alongside your working design purely as a source of reusable cells: vias, pads, packages, or anything else. Attaching a library does not merge it into your design file; it just makes its cells available to place.

8.1 Attaching and browsing

- Setup → Attach Library... picks a .gds file and attaches it (maximum 2 at a time).
- Setup → Attached Libraries... lists each attached library's index, name, cell count, and path.
- Create → Find & Place Cell... searches by name across the current design and every attached library, then places the chosen cell with a click.

A design's .hlaux sidecar (written automatically by File → Save GDS...) embeds a full copy of every attached library, so reopening the design restores them automatically — you don't need to keep re-attaching the same source files by hand.

8.2 Via Wizard

Tools → Via Wizard... builds a standard via cell — a plated hole connecting two conductor layers, sized as a square or round pad — with an editable Cell name (defaults to VIA_Layer<n1>_to_Layer<n2>, but stops auto-updating as soon as you type your own), an optional “Add dielectric via window” checkbox that draws a matching-shape opening of a chosen size on a chosen dielectric layer between the two conductors, a Save to library picker for which attached library to save into, and an optional overwrite of an existing via cell of the same name. The equivalent command-bar/scripting form is VIA <layer1> <layer2> [shape=][size=][name=][dielectric_layer=][dielectric_window=][index=][overwrite=].

8.3 Pad Wizard

Tools → Pad Wizard... builds a rectangular conductor pad (with an optional matching solder-paste opening) as a cell — optionally as a rectangular array of pads sharing a pitch — and saves it into an attached library file the same way the Via Wizard does. The equivalent command form is PAD <name> <conductor_layer> <paste_layer> <length_mm> <width_mm> <margin_mm> [rows=][cols=][pitch_x=][pitch_y=][index=][overwrite=].

8.4 Pin Numbering

A pin is a short number or name ("1", "2", "K", ...) attached to one pad -- a Rectangle or Polygon shape -- inside a footprint or device cell, identifying that pad's electrical role. Combined with a placed instance's ref designator (Edit Selected...'s Label field, or auto-set by RESGEN/the Resistor Designer), a pin gives every physical pad in the design a unique identity such as "R1.2" -- exactly what Netlist Extraction (§10.3) reads to determine which pins belong to which net.

A genuine 2-pad part is unambiguous, so it's numbered for you automatically: the Pad Wizard's 2-pad footprints and every resistor RESGEN/the Resistor Designer places get pin "1"/"2" (the lower-X pad is "1") with no extra step. Anything else -- a 3+ pin footprint, a package imported from KiCad, a custom part -- needs numbering by hand, since which physical pad is pin 1 depends on the part's actual datasheet pinout, not just its geometry.

Edit → Number Pins... enters a click-to-increment tool for exactly that: type a starting number (default 1), then click each pad in the order its pins are actually numbered on the part -- each click stamps the current number and moves on to the next one automatically. Re-clicking an already-numbered pad asks before

overwriting it. Esc stops and returns to Select. The same field is also visible/editable one pad at a time via Edit Selected...'s Pin field, and scriptable via PIN LIST/PIN SET (§9.1) or Session.list_pins()/set_pin() (§17.14) for numbering many pads at once or auditing what's already assigned.

Edit → Ref-Des to Pin 1 moves a placed part's ref designator to just beside its pin 1, on the outside of the pin row (0.8 mm beyond pin 1's pad, towards the nearest edge of the part). With an instance (or its ref-des) selected it moves that one; with nothing selected it moves every connector/header in the cell — ref designators J, P, X, CN, CON or HDR followed by a number. This happens automatically when a connector is placed with its ref-des (sref/place_cell label=) or first given one in Edit Selected..., so header ref-des sit next to pin 1 by default. The command-bar equivalent is REFDES PIN1 [<refdes>|ALL] [gap=<dbu>], and the Python one Session.refdes_to_pin1(target=None, gap=None).

The tool can also stamp a pin onto a placed cell instance itself (SRef.pin) rather than one of its own pads — the way a round lead built from a shared Via Wizard template gets its own per-placement number, since the template's shapes are common to every instance and cannot carry one individually; clicking the instance (not one of its shapes) stamps this instead. Every click now gives status-bar feedback — which pad or instance was just stamped and a running count, or, for an invalid click (empty canvas, a shape that is not a pad), what to click instead — and Esc reports how many were stamped before the tool was cancelled.

8.5 Wire Bonds

A wire bond is a chip-and-wire assembly jumper — a ball-to-ball "O------O" figure — connecting a bare-die pad to a conductor pad, a conductor pad to a package pin, or two conductor pads directly. Draw one with the Create menu's Wire Bond tool (click one pad, then the other) or the BOND command bar/scripting equivalent (§9.1); Extract Netlist (§10.3) connects a bond at its two ends only — the wire itself flies over whatever lies between — so a bond carries connectivity straight through to the die pin without shorting to traces underneath it.

Edit → Move Wire Bond picks the bond nearest your first click and moves whichever end is closer to it — the other end stays fixed. As you move the cursor toward the new position, a live preview follows it: the bond's O------O rubber-band (with its span length) redraws from the fixed end to the cursor, over a faint line showing where the bond sits now, so you can see the new span against the min/max length rules before committing it with a second click. Edit → Delete Wire Bond removes a whole bond — the connector path and both end discs — with a single click near it.

Setup → Technology Properties..., Wire Bonds tab sets the bond layer, Wire width (mm), End ball dia. (mm), and Min/Max length (mm) — the defaults BOND/wire_bond() use unless overridden per call. The built-in default wire width is 0.05 mm (50 microns, ~2 mil). The same tab holds Isolated pad layers: a comma-separated list (e.g. 58, 49) of layers whose pads sit above the substrate rather than on it — bare-die bond pads, header/package pins. Every shape on those layers is kept apart from the substrate copper underneath and joined to a net only where a wire bond lands on it. It is the same setting as the Isolated column in Tools → Extract Netlist... (§10.3), so ticking a layer there updates this tab and vice versa.

9. The Command Bar and Python Scripting

Every edit you can make with the mouse can also be made by typing a short command into the command bar or Script Console, or by calling an equivalent method on a Python Session object. All three routes edit the exact same underlying design, so they can be freely mixed — draw something by hand, then select it and type MOVE 5,0, then finish the design with a Python loop.

9.1 The command language

Commands are case-insensitive. A point argument accepts either `x,y` or `x y`. Anywhere a `<layer>` argument appears, you may give either a layer number (0–63) or an assigned layer name. Anything HLayout doesn't recognize as a command is passed straight through to the Python interpreter, so the command bar doubles as a Python prompt. Typing `HELP` prints the full reference (the same text that backs Help → Command Reference's fallback dialog).

Cells & shapes

Command	Purpose
CELL <name>	Create or switch to the named current cell.
RECT <layer> <x1,y1> <x2,y2>	Draw a rectangle.
POLY <layer> <x1,y1> <x2,y2> ...	Draw a polygon (3+ points).
PATH <layer> <width> <cap> <x1,y1> ...	Draw a path; cap is round, square, or flush.
CIRCLE <layer> <cx,cy> <radius> [segments]	Draw a circle (8/16/32/64 segments, default 32).
TEXT <layer> <x,y> "<string>" [height] [font]	Place a text label.
SREF <cellname> <x,y> [rotation] [mag] [mirror]	Place a single cell instance.
AREF <cellname> <x,y> <cols> <rows> <col_sp> <row_sp> [rotation] [mag] [mirror]	Place a rectangular array of a cell.
DELETE LAST	Remove the most recently added shape/instance.

Layers, view, selection, undo/redo

Command	Purpose
LAYER <n|name> [color=][stipple=0-8][fill=0|1][visible=0|1][name=]	Configure a single layer's display properties.
LAYER ALL visible=0|1	Show or hide every layer at once.
LAYER MOVE <from> <to> [cell=]	Bulk-move shapes from one layer to another.
ZOOM IN|OUT|FIT|<factor>	Zoom controls.
ZOOM WINDOW <x1,y1> <x2,y2>	Zoom to a specific box.
PAN <dx,dy> | PAN TO <x,y> | PAN LEFT|RIGHT|UP|DOWN [f]	Move the view by dx,dy DBU, centre it on a point, or step it by a fraction of the screen (default 0.25).
FIT	Fit the current cell in view.
GRID / GRID ON|OFF / GRID <spacing_mm>	Query or set grid visibility/spacing.
MOVE <dx,dy> / ROTATE <deg> / COPY <dx,dy>	Act on the current selection (a shape must be selected first).
PROPS	List the current selection's properties.

Command	Purpose
SMASH	Explode a selected array instance (AREF) into individual SRefs.
UNDO / REDO	Step the undo history backward/forward.

Files, technology, attached libraries

Command	Purpose
SAVE <path.gds> [library_name]	Without a name: saves the working design plus its .hlaux sidecar. With a name: writes a bare, publishable .gds.
OPEN <path.gds>	Opens a design, restoring its .hlaux sidecar if present.
GERBER <output_prefix> [cell=]	One-shot Gerber export.
MERGE <lib1.gds> <lib2.gds> <lib3.gds> [library_name]	Merge two GDS libraries into a third file (see Chapter 14).
TECH LOAD/SAVE <path.json>	Load or save the technology file.
TECH SET <key> <value> / TECH GET <key> / TECH PARAMS	Read or write individual technology parameters.
ATTACH <path.gds>	Attach a reference library (max 2).
LIBS / LIBLIST [current|0|1] / FIND <substring>	List attached libraries, list their cells, or search cell names.
PLACE <cellname> <x,y> [source=current|0|1][rotation][mag][mirror]	Place a cell from the current design or an attached library.
VIA <layer1> <layer2> [shape=][size=][name=][dielectric_layer=][dielectric_window=][index=][overwrite=]	Build and save a via cell into an attached library; name defaults to VIA_Layer<n1>_to_Layer<n2>; dielectric_layer/dielectric_window (given together) also cut a matching window in the dielectric between the two conductor layers.
PAD <name> <clayer> <player> <len_mm> <width_mm> <margin_mm> [rows=][cols=][pitch_x=][pitch_y=][index=][overwrite=]	Build and save a pad cell into an attached library.

Thick-film resistors, mask operations, and DRC

Command	Purpose
RESGEN <x,y> <body_width> <body_length> [rotation=][ink=][ref=][a=][b=][c=][d=][rlayer=][clayer=][glayer=][dlayer=	Builds a resistor cell (ink body, glaze overprint, two conductor pads, optional ref-des label) and places one instance. The two pads are automatically numbered pin "1"/"2" (lower-X pad is "1"), and when ref= is given it's also set as the placed instance's label, so the resistor is

Command	Purpose
	immediately ready for Netlist Extraction (§10.3) with no extra numbering step. ink= looks up an ink layer by sheet resistance; geometry letters default to Setup → Resistor Geometry Rules.
MASKOP OVERSIZE|UNDERSIZE <layer> result=<layer> amount=<dbu> [cell=][flatten=0|1][simplify=<dbu>]	Grows or shrinks all geometry on a layer by a fixed amount, writing the result to another layer.
MASKOP AND|OR|NOT|XOR <layer_a> <layer_b> result=<layer> [cell=][flatten=][simplify=]	Boolean-combines two layers. flatten defaults to on (descends into placed cell instances); simplify applies Douglas-Peucker vertex reduction to the result.
DRCRULE ADD <kind> <layer_a> [layer_b] value=<n> [name=]	Adds a design rule. kind is MIN_WIDTH, MIN_AREA, or MIN_SPACE (one layer), or MIN_SPACE_BETWEEN or MIN_ENCLOSURE (two layers).
DRCRULE LIST / REMOVE <index> / CLEAR	Manage the saved rule set.
DRC RUN [cell=][error=<layer>]	Runs every saved rule (descending into cell instances) and optionally draws violation markers on a chosen layer.
PY <python source> / HELP	Run raw Python, or print the full command reference.

Both mask operations and DRC require the shapely package (see §2.1) — they are unavailable, with a clear error message, if it isn't installed.

Pin numbering and netlist extraction

Command	Purpose
PIN LIST [cell=<name>]	Lists every pad in the given (or current) cell with its shape index, layer, pin, and bounding box -- useful before/after numbering, or to find an index for PIN SET.
PIN SET <index> <pin>	Sets the pin of the pad at that shape index (from PIN LIST) in the current cell -- scripted equivalent of one click of the Number Pins... tool.
REFDES PIN1 [<refdes>|ALL] [gap=<dbu>]	Puts a placed part's ref-des beside its pin 1; with no ref-des, every connector/header (J, P, X, CN, CON, HDR).
NETLIST RUN [layer1 layer2 ...] [cell=<name>] [export=<path>] [format=csv|report|kicad|tango] [bonds=0|1]	Extracts a netlist from the given conductor layer(s) in the current (or named) cell -- or, with no layers, from the design's saved Conductor selection in Tools → Extract Netlist... -- and prints each net's RefDes.Pin members, its bond count and any open bond ends; export= writes the same result to a file — .csv, .txt (report), .net (KiCad) or

	.tng (Tango/Protel), or the format= given, bonds=0 leaves wire bonds out (§10.3).
BOND <x1,y1> <x2,y2> [layer=][width=][pad=][min=][max=][cell=]	Draws a chip-and-wire assembly wire bond (an O-----O figure) between two pads on the bond layer. Span must be within min/max and must not cross an existing bond. Defaults from Setup - Technology Properties, Wire Bonds tab; NETLIST RUN includes bonds automatically, joining what each bond's two ends land on (8.5).
AUTOROUTE <x1,y1> <x2,y2> [layer=][grid=][width=][clearance=][margin=][cell=]	Routes a clear orthogonal trace between the two points on the routing layer (default: Route layer_a), A*-searching a grid so it dodges every other shape on that layer. grid/width default to the canvas's current values. Also the two-click Edit - Auto-Route Two Pins tool (click the first pin, then the second).

Nets, autorouting and chip-and-wire

Command	Purpose
VIA DELETE <x,y> [tol=] [layers=] [cell=]	Removes the via at x,y: its pads on both conductors, its dielectric window and any placed via cell, as one undo step. Tracks are left. Also Edit → Delete Via.
HILITE <x,y> | <RefDes.Pin> | <net> | OFF	Highlights every pad and track on a net and reports its name. Layers connect only through via windows in the dielectric. OFF clears it. Also Edit → Highlight Net.
NETNAMES IMPORT <file.csv> [merge=1] | CLEAR	Loads the design's net names from a CSV with a net column and a pad (U1.3) or ref + pin columns; NETNAMES alone counts them. Highlight Net and the autorouters use these names.
FREEROUTE RUN [passes=] [keep=] [fresh=] [layers=] [finger=]	Exports a Specctra DSN, runs Freerouting and imports the routing (Chapter 21). EXPORT <file.dsn> and IMPORT <file.ses> do the two halves by hand.
HYBRID IMPORT <file.routed.json> [cell=] [fill=]	Draws a chip-and-wire router result in its own cell and keeps its bond diagram (§21.9). HYBRID BONDS [<file.csv>] lists or writes the bond diagram.
BONDTABLE <cell> <layer>	Draws the bond diagram as a table cell; the assembly drawing adds it automatically.
BONDINFO <x,y>	The wire bond nearest x,y: id, net, ends, length, wire, loop height, departure angles.
MOVEFINGER <pad|x,y> <x,y>	Moves a bond finger or jumper land; its wires follow and are checked against the bond rules (§21.10).
RESIZE <refdes> [length=] [width=] [preview=1]	Re-sizes an imported printed resistor; its value follows R = Rs × L / W (§21.10). RESIZE <refdes> alone reports its size and value.

Drawing sheets, table generators, and printing

Command	Purpose
CLEAR <layer> [cell=<name>]	Deletes every shape on layer from the named (or current) cell's own shapes -- handy before a TOPVIEW rerun, since TOPVIEW always appends rather than replacing.
OUTLINE <layer> width=<dbu> [cell=<name>]	Replaces every Rectangle/Polygon on layer with an equivalent closed-ring, stroked outline Path of that width -- e.g. so a substrate or package body reads as an outline instead of a filled shape.
TOPVIEW result=<layer> [layers=][exclude=][hatch=][width=][cell=]	Draws a top-down, hidden-line-removal pictorial assembly view onto result -- the visible outline edges of the design as seen from directly above, with occluded edges genuinely removed (§10.5). TOPVIEW with no arguments rebuilds the view from the settings saved in Tools → Top View.... refdes=1 adds every part's ref-des as text, written last (refdes_height=<dbu>).
RESTABLE <cell_name> <layer> [origin=][title=][overwrite=]	Builds a resistor BOM table cell (REF DES / VALUE / TOL% / AIM VALUE / INK) from the saved Resistor Designer rows -- generic and re-runnable (Chapter 18).
PINTABLE <cell_name> <layer> <pin_count> [pergroup=][groupgap=][title=]	Builds a pinout reference table cell -- PIN # / PIN FUNCTION column pairs in groups of pergroup pins, blank by default for hand fill-in (Chapter 18).
SHEET <cell_name> <layer> [size=][orientation=][title=][drawnby=]...	Builds a standard A4/A3 drawing sheet cell -- a ruled border and title block (TITLE, DWG NO/ISSUE, DRAWN BY/CHECKED BY/APPROVED BY with dates) (Chapter 18).
PRINT [cell=][out=][scale=][pinlayers=]	Renders the design as a printable HTML page and hands it to the system print flow (§14.4).
ASSEMBLY [size=][orientation=][scale=][layout=][title=][dwgno=]...	Builds the complete assembly drawing in one step -- drawing sheet, a scaled top view and the pin and resistor tables, each its own cell, placed into cell ASSEMBLY (§18.5). Settings are saved with the design, so a bare ASSEMBLY rebuilds the last drawing. Backs Tools → Assembly Drawing...

9.2 Python scripting

The `Session` class (in `hlayout.scripting.api`) is the single object exposed to both a standalone script and the GUI's built-in console. It wraps a design library with current-cell/current-layer convenience state, undo/redo, and a change callback so the GUI repaints whenever a script edits the data.

```
from hlayout.scripting.api import Session

s = Session.new("MYLIB")
s.new_cell("TOP")
s.rect(layer=0, x1=0, y1=0, x2=100, y2=50)
```

```
s.polygon(layer=1, points=[(0, 0), (10, 0), (5, 10)])
s.path(layer=2, points=[(0, 0), (50, 50)], width=5, cap="round")
s.text(layer=3, origin=(0, 0), text="HELLO")
s.save("out.gds")
```

In the GUI's Script Console, a `Session` is already bound as both `s` and `hl`, along with `library` and the core shape classes (`Cell, Library, Path, PathCap, Polygon, Rectangle, SRef, Text`) for convenience — you can start scripting immediately without any imports.

Both the command bar and the Python API are two front ends onto the same session, so the same design can be built either way:

```
# Command bar
RECT 0 0,0 100,50
SREF VIA_Layer1_to_Layer3 200,0

# Python — same result
from hlayout.scripting.api import Session
s = Session.new("MYLIB")
s.new_cell("TOP")
s.rect(layer=0, x1=0, y1=0, x2=100, y2=50)
s.save("out.gds")
```

A few more examples worth trying in the console or command bar:

```
MERGE lib1.gds lib2.gds combined.gds

ATTACH parts.gds
PLACE VIA_Layer1_to_Layer3 200,0

s.attach_library("sot_packages.gds")
s.place_cell("SOT-23", (0, 0))

TECH LOAD process.json
```

A bad script line is caught and its traceback printed to the console rather than crashing the application, so experimenting is safe. Command history (Up/Down arrow recall, your last 10 commands) is shared between the command bar and the Script Console.

For the full method-by-method Python API reference, see the Appendix (Chapter 17) and Help → Command Reference (F1). For worked examples, with the output each one prints, see the Python API Cookbook (Chapter 19).

9.3 Connecting Claude (MCP server)

HLayout includes an MCP (Model Context Protocol) server, so an MCP client such as Claude can drive the layout as a set of tools: open and save designs, run command-bar lines, run Python against the Session, and inspect cells. The client starts the server itself, as a separate process that talks over stdio and needs only the Python standard library:

```
python -m hlayout.mcpserver [--headless] [--design path.gds]
```

The server reaches a design in one of two ways:

- Live: an HLayout window is running with its MCP bridge on. Every tool call is forwarded to that window, so edits appear on the canvas and go into its normal undo history (Edit → Undo).

- Headless: no window is listening, or the server was started with --headless. The server keeps its own in-memory session and works on files directly; nothing appears on screen. --design path.gds opens a file at start.

Every reply says which of the two was used whenever that changes, so it is always clear whether an edit went to the visible window or to a private session.

Starting and stopping from the GUI

Tools → MCP Server (Claude) is a checkbox. Tick it to start the bridge in the running window; untick it to stop. The status bar shows “MCP bridge on (127.0.0.1:port)” or “MCP bridge off”. To have it on from launch instead, use the --mcp switch or the HLAYOUT_MCP environment variable; the menu item then starts ticked:

```
python hlayout_app.py --mcp [mydesign.gds]
set HLAYOUT_MCP=1
```

The bridge is off until you ask for it, because it lets a local process run Python inside the application. It listens on 127.0.0.1 only, on a random port, and every request must carry a random per-run token. Port and token are written to ~/.hlayout/mcp_bridge.json and removed when the bridge stops or HLayout exits. Each start uses a new port and token, and the server re-reads them on every tool call, so Claude does not need restarting after you toggle the menu item. While the bridge is off, tool calls fall back to a headless session.

Registering the server with Claude

Claude Code picks the server up from the .mcp.json file in the HLayout folder. For the Claude desktop app, add the following to claude_desktop_config.json, with PYTHONPATH set to your own HLayout folder:

```
{"mcpServers": {"hlayout": {
  "command": "python",
  "args": ["-m", "hlayout.mcpserver"],
  "env": {"PYTHONPATH": "C:\\dev\\HLayout"}
}}}
```

The tools

- hlayout_status: live/headless mode, open file, library, current cell, cell list, named layers, undo state.
- hlayout_open: open a GDSII file.
- hlayout_save: save the design.
- hlayout_command: run command-bar lines (§9.1).
- hlayout_python: run Python against the design. The Session is bound as s (alias hl), as in the Script Console (§9.2).
- hlayout_api: list the Session methods, or show the full documentation for one.
- hlayout_describe_cell: bounding box, shape counts per layer, instances and pins of a cell.

10. Mask Operations and Design Rule Check

Both tools operate on layer geometry as polygons via the shapely library, and both are available as dialogs (Tools menu), command-bar commands, and Python methods.

10.1 Mask Operations

Tools → Mask Operations... performs a single sizing or boolean operation on layer geometry and writes the result to a (usually different) result layer, leaving the source layer(s) untouched:

- Oversize / Undersize — grows or shrinks every shape on a layer by a fixed amount (in DBU), commonly used to generate solder-mask openings, glaze overprints, or clearance rings from a pad or trace layer. Corners stay sharp (mitred) by default; from a script, “Session.mask_op()”’s join_style argument (“mitre” default / “round” / “bevel”) can round the offset instead — needed when the source layer already includes a many-facet curve (e.g. a pad unioned with a castellation scallop), where a mitre join would otherwise spike a small spurious notch at the facet-to-straight-edge seam. Not yet exposed in the Mask Operations dialog or MASKOP command.
- AND / OR / NOT / XOR — standard boolean combination of two layers' geometry.

Both kinds accept an optional cell (default: the current cell), a flatten option (on by default — descends into any placed cell instances so nested geometry is included), and a simplify tolerance (Douglas-Peucker vertex reduction on the result, useful for keeping generated geometry lightweight).

Tools → Mask Operation Batch... runs a saved sequence of mask operations in one pass — handy for a repeatable set of derived layers (solder mask, paste, glaze) that always follow the same rules relative to your drawn geometry.

10.2 Design Rule Check (DRC)

Tools → DRC... manages a set of design rules and runs them against the current design:

Rule kind	Layers	Checks
MIN_WIDTH	one	Minimum feature width on a layer.
MIN_AREA	one	Minimum shape area on a layer.
MIN_SPACE	one	Minimum spacing between separate shapes on the same layer.
MIN_SPACE_BETWEEN	two	Minimum spacing between shapes on two different layers.
MIN_ENCLOSURE	two	Minimum enclosure of one layer's geometry by another's (e.g. a pad enclosed by its solder mask opening).

Rules are added, listed, removed individually, or cleared entirely, and are saved with the design's technology settings so they persist across sessions. Each rule has its own colour (assigned automatically, or set by clicking its swatch in Add/Edit Rule…), which tints its rows in both the rule table and the Violations list so error types are easy to tell apart at a glance. Running DRC descends into placed cell instances, checks every saved rule, and can optionally draw violation markers directly onto a chosen layer for visual review on the canvas — spacing-type violations are marked with a thin 0.1 mm wide line or dot at the gap. Every checked cell's existing markers on that layer are cleared before the new ones are drawn, so re-running DRC after a fix never leaves stale markers mixed in with — or standing in for — the current results.

MIN_WIDTH allows a 1 µm tolerance, so a track drawn exactly at the limit, or with round ends, isn't flagged; a track that really narrows below the limit still is.

10.3 Netlist Extraction

Tools → Extract Netlist... turns copper connectivity plus pin/ref-designator identity into a list of electrical nets. The window lists the design's layers in the same style as the Layers panel, each with two checkboxes: tick Conductor for substrate copper (routing layers and printed pads — touching or overlapping shapes on any ticked conductor layer are one net, including through vias), and tick Isolated for pads that sit above the substrate and are reached only by wire bonds (bare-die pads, header pins). A layer can be one or the other, not both; leave every other layer unticked. The wire-bond layer is shown as "wire bonds (auto)" and is included whenever Include wire bonds is on. Show only layers used in the design hides empty layers; Clear all unticks everything. Every tick is saved with the design immediately (the Isolated column is the same setting as the Wire Bonds tab's Isolated pad layers), so the selection is there next time. Choose a cell (default: current) and press Run (or F5). Each pinned pad found inside a net contributes a RefDes.Pin member (e.g. "R1.2"), using the label of its immediate placing instance; a pinned pad whose instance has no label yet shows as "?.2" and is counted as unresolved. Results appear in two lists. Nets shows each net's members, bond count, unresolved count and open bond ends; clicking a net zooms the canvas to it and rings every member pin (blue = good, yellow = unresolved, red = open bond end). Issues lists every problem found, one per row with its location in mm: Open bond end (a bond end that lands on nothing), Unassigned pin (a pin on a layer that isn't ticked), Unresolved (a part with no RefDes, or a bonded die pad with no pin number) and No pins (copper that no pin touches, e.g. a die-attach pad). Clicking an issue switches the canvas to the extracted cell, zooms to the problem and drops a labelled crosshair marker on it, with the run's other issues ringed around it. Markers stay through panning and zooming until Clear markers, the next Run, or closing the window.

Export... writes the last Run's netlist to a file, in the format chosen by the file type in the save dialog (or the extension typed): CSV (.csv — one row per net, ref_des, pin), Text report (.txt — the design, cell and conductor layers, every net with its members and bond count, then every issue with its location in mm), KiCad netlist (.net — an (export (version D) ...) s-expression that KiCad's Pcbnew and Eeschema import) or Tango / Protel netlist (.tng — [component] and (net) blocks read by most PCB tools). In the KiCad and Tango files every placed part with a ref-des is a component, its cell name used as value and footprint; members with no ref-des, bonded pads with no pin number and pins on unticked layers are left out of those two (they cannot be resolved by another tool) but are listed in the report. Files go to a netlist folder next to the design by default, and a dialog then shows the full path with Open Folder and Copy Path. The command-bar equivalent is NETLIST RUN [layer1 layer2 ...] [cell=<name>] [export=<path>] [format=csv|report|kicad|tango] [bonds=0|1] (§9.1) — with no layers it uses the design's saved Conductor selection, and the format follows the export file's extension unless format= is given. The Python equivalents are Session.extract_netlist(conductor_layers=None, cell_name=None) — None again meaning the saved selection — and Session.export_netlist(nets, path, fmt=None, cell_name=None), which returns the format written (export_netlist_csv(nets, path) still writes the plain CSV) (§17.15).

Netlist extraction only knows about the layers you tick, so a missing conductor layer produces an incomplete result rather than an error -- pins on unticked layers are reported as Unassigned pin issues, and a bond end that lands on nothing as an Open bond end, so check the Issues list when a net looks wrong. An array instance (AREF) has no per-copy ref designator or pin number, so a pinned part placed as an array reports the array's own label for every copy; place parts that need individual pin numbers (header pins, for example) as separate instances.

Chip-and-wire designs. Tick Conductor only for the substrate copper — the routing conductor(s) and any pad layers that are real printed copper. Tick Isolated for the die-pad layer and the header-pin layer. Never tick a die-pad or header-pin layer as Conductor: conductor layers merge wherever they overlap, so a die pad would short to the die-attach pad beneath the die, and a header pin to any trace it sits over.

How each kind of layer is handled: conductor layers — touching or overlapping copper is one net. Bond layer — each bond-layer path joins whatever its two endpoints land on; the end-ball circles are pictorial. Isolated pad layers — each shape is its own node; a bond end joins the smallest pad shape under it, and a pad-layer shape that encloses other pads (a die outline drawn on the pad layer) is never a landing. A bond end that lands on nothing, or only on a die outline, is listed as an open bond end on its net, so a bond that misses its pad is visible. A bonded die pad with no pin number appears as RefDes.@x,y (its centre in DBU) and counts as unresolved.

Example (notch filter): Conductor ticked on 3 and 5; Isolated ticked on 58 (die pads) and 49 (header pins); bond layer 59 (Wire Bonds tab) shows as wire bonds (auto). Put pin numbers on the conductor-layer pads of chip parts (not on their epoxy dots), number each header pin as its own placed instance (an array cannot carry per-copy pin numbers), and give the die and header placements ref designators. A path such as header pin → bond → trace → part, or header pin → bond → die pad, then comes out as one net.

10.4 Generate Dielectric Fill

Tools → Generate Dielectric Fill... lays a solid dielectric sheet with holes, built entirely from the dielectric layer's own geometry. Draw the outline you want filled and the openings you want left clear on that layer at datatype 0: the largest shape that encloses all the others is taken as the fill boundary (or, if nothing encloses the rest, the design's bounding box grown by the margin), and every other datatype-0 shape inside it becomes a hole. A via window stamped by drop_via, a resistor's between-terminals keep-out, and a hand-drawn exclusion rectangle are all treated the same way, each grown by the clearance before it is cut. Extra keep-out layers can be named too, for exclusion zones on other layers.

The filled result is written back on the same layer at datatype 1, automatically fractured into hole-free polygons so none exceeds the 200-point GDSII limit. Re-running replaces just that fill and never touches the datatype-0 inputs, so the sheet can be regenerated after the layout changes. Tools → Remove Dielectric Fill (or DIELECTRIC CLEAR) deletes the datatype-1 fill and leaves the inputs in place. From the command bar the operation is DIELECTRIC [layer=] [clearance=] [margin=] [keepout=]; from a script, Session.generate_dielectric() / clear_dielectric(). Both require shapely.

10.5 Top-Down Assembly View (TOPVIEW)

Tools → Top View... (or the TOPVIEW command / Session.mask_op_top_view()) draws a top-down, hidden-line-removal pictorial assembly view onto a result layer — the classic CAD/drafting sense of the term, as in AutoCAD's HIDE. Rather than a filled silhouette, it draws the visible OUTLINE edges of the design as seen from directly above, with the part of a lower entry's edge buried under a higher entry's filled footprint genuinely removed instead of drawn underneath it — the picture a technician actually needs to identify and locate every part during assembly.

With no explicit layers= list, stacking follows the real fabrication process order by ROLE, regardless of layer numbers: placed components and their epoxy/keep-out footprints sit topmost, then the resistor ink layer(s), then every conductor layer (including the Route Layers), then any other visible layer with real drawn geometry, then the substrate. A placed cell is recognised as "a component" by NAME (a known EIA size code), by FOOTPRINT (a known gull-wing pin pattern), by an explicit component_models/Component Library entry, or STRUCTURALLY as a "bare" part — a placed cell whose own shapes sit entirely on non-role layers and that has at least one child instance, e.g. a naked die with no size code and no catalogued pin-count table entry. Glaze AND the Wire Bond layer are both drawn as non-occluding overlays, always shown in full — glaze is a thin overprint rather than an opaque film in this convention, and a wire bond terminates ON a pad rather than running underneath it, so occluding it would clip it short of the very pad it bonds to. None of this looks at a layer's fired-film thickness — only its stacking role and whether it has real geometry.

exclude= drops specific layers out of the role-based default, and also gates the structural “bare part” recognition above: a bare part is walked into instead of treated as an occluding component when ALL of its own shapes sit on an excluded layer — the escape hatch for a sealed package's own case/lid, whose outline can legitimately span the entire board and would otherwise occlude every real component and all conductor/resistor content inside it. hatch= fills every conductor-tier layer's own visible region with a cross-hatch at that pitch, the standard technical-drawing convention for a solid material. TOPVIEW always APPENDS new paths to its result layer rather than replacing what's there, so repeated runs while iterating accumulate leftover paths — run CLEAR on the result layer first for a clean rerun.

```
TOPVIEW result=63 exclude=61 hatch=100
s.mask_op_top_view(result_layer=63, exclude_layers=[61], hatch_conductor_pitch=100)
```

Tools → Top View... opens a window laid out like Extract Netlist: a layer table with a Show checkbox for each layer and its role in the view (resistor ink, conductor, other layer, substrate at the bottom; glaze and wire bonds as overlays), listed in stacking order — placed parts and dies are drawn on top. A package the substrate sits in (a header or bathtub can: a component model referenced to the substrate underside, or a part whose outline spans most of the design) is drawn underneath the substrate instead, in detail — case edge, well wall, every pin and its glass seal — so whatever the substrate doesn't cover shows and the package never hides the circuit. The substrate is included even when its layer is hidden on screen. Untick a layer to leave it out; Show only layers used in the design hides empty layers, and Show all / Hide all tick or untick every row. Below the table are the Result layer (on first use a layer named Top View is picked for you), the Output cell (TOPVIEW unless renamed), the line width, Hatch conductor with its pitch and angles, Include placed cells, Simplify, and Include ref-des with its text height (on by default, 0.6 mm): every placed part's reference designator is written as text on the result layer after all the outlines — the last thing in the view, never hidden — where the canvas shows it (beside pin 1 for a header, or at a part's own baked ref-des text). Build (F5) draws the view of the main design cell into the output cell and keeps the window open, so you can change a setting and build again; Open Output Cell switches the canvas to the result. The result layer is never read back in as input. Every choice is saved with the design the moment it changes, so the window opens with the same settings next time, and TOPVIEW typed with no arguments (or Session.top_view_from_settings()) rebuilds the view from them. The full option set (explicit layer order, per-run overrides) is still available through the TOPVIEW command. Both require the shapely package.

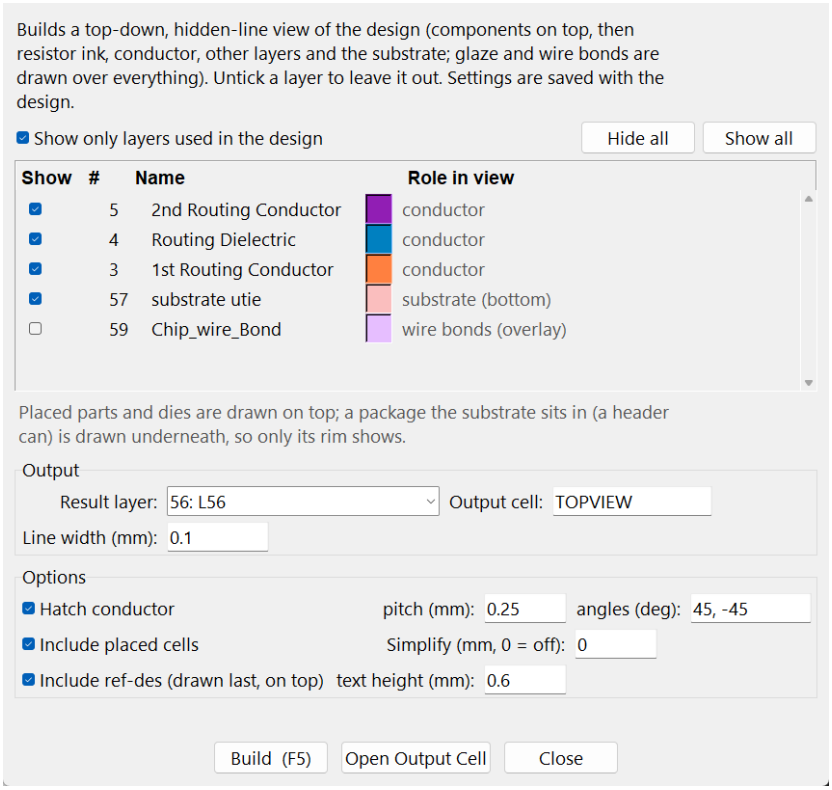


Figure 10.1 – Tools → Top View...: the layer table, the result layer and the hatching options.

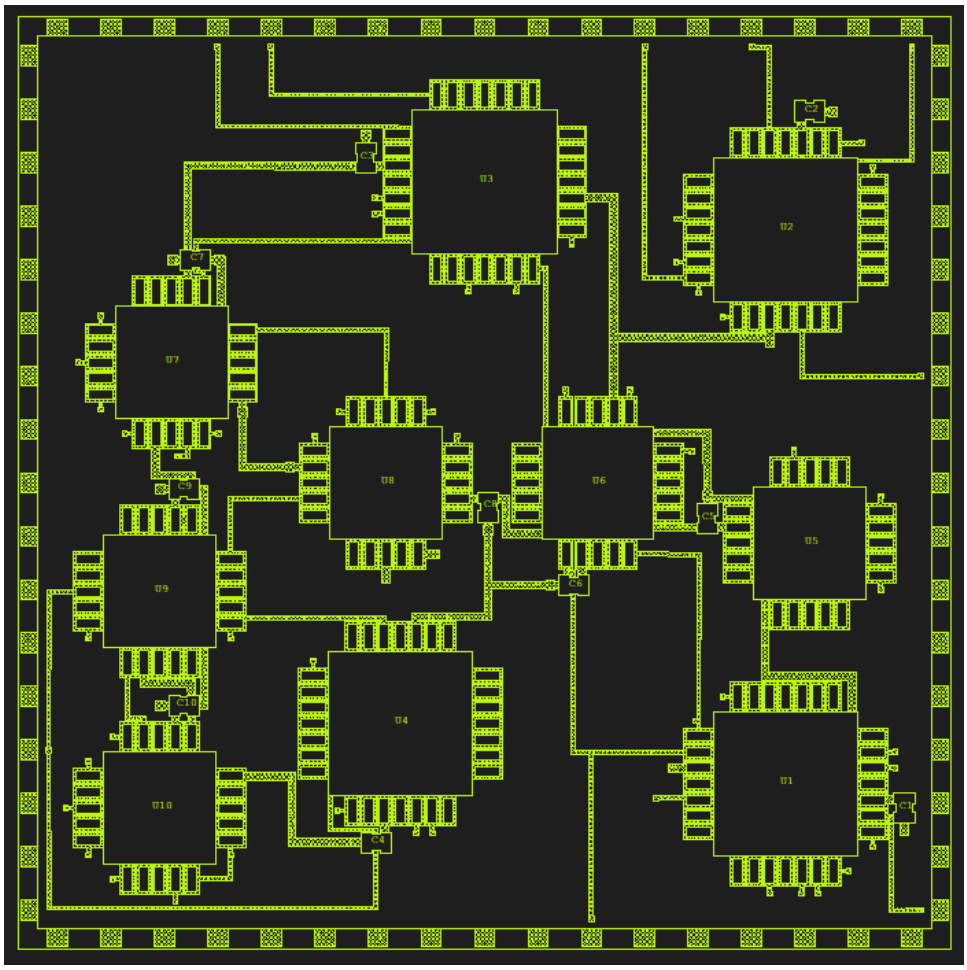


Figure 10.2 – The ALU16 module's top view: hidden-line outlines with the visible conductor cross-hatched at 0.25 mm, component bodies on top and reference designators drawn last.

Reference designators in the top view are centred on the middle of each placed part's body (its geometry, ignoring any text inside the part). Where the canvas floats a ref-des — a dragged label, a ref-des moved to pin 1, or a baked-in ref-des Text off to one side — is deliberately ignored here, so every label sits on its part in the assembly view.

Build runs in the background behind a small progress window with an elapsed-time counter, and the status bar shows the same, so HLayout keeps responding on a large, hatched design. Only the conductor layers are hatched, never the printed dielectric. Parts are recognised by name, including LCC and PLCC chip carriers, and only a part's body hides what is under it: its pads are drawn and hatched as conductor where they show.

11. The Thick-Film Resistor Designer

Tools → Thick-Film Resistor Designer... opens a dedicated window ("Thick-Film Resistor Designer") for sizing and placing printed thick-film resistors. Given a target value, tolerance, and power, it chooses an ink sheet resistance and a body length/width so the resulting number of squares lands within an allowed aspect-ratio window, then can draw the sized part directly into your layout.

11.1 Window layout

Toolbar: Add... · Edit... · Delete · Auto-Select All · Recalculate · Design Rules... · Place in Layout... · Place All in Layout... · Trim Simulator... · Import CSV... · Export CSV... · Print / Preview Report... · About.

The main panel is a resistor list table with columns: Ref Des, Value, Aim Value, Tol, Power (mW), Width, Ink (Ω/□), Squares, L (mm), Ratio, Sols (solution count), P OK, and Status. Rows with no valid solution are tinted red-ish; rows with a power warning are tinted orange-ish. A Solutions tab beside it shows the detail for whichever resistor is selected, and a Summary/BOM tab gives a whole-design rollup. The list starts empty — add resistors with Add... or Import CSV....

11.2 Adding and editing a resistor

The Add Resistor / Edit Resistor dialog has these fields:

Field	Description
Ref Des	The reference designator text, e.g. R1.
Value (Ω)	The nominal resistance value.
Tolerance	One of 0.1%, 0.5%, 1%, 2%, 5%, 10%, 20%.
Power (mW)	The power the resistor must dissipate.
Body Width	One of the standard widths 1.0–3.8 mm; each option is labeled with its trim-derated maximum power. An "Auto from power" checkbox will pick the smallest adequate width for you automatically.

A live OK/WARNING indicator compares the entered power against the trim-derated rating of the chosen body width, so you can catch an undersized part before it's placed.

11.3 How sizing works

The **aim value** is the nominal value pushed toward the high end of tolerance to leave trim headroom: aim = nominal × (aim% + tolerance%). For example, a 1 kΩ resistor at 5% tolerance with a 75% aim percentage targets 800 Ω. For each enabled ink sheet resistance, the tool computes squares = aim ÷ Rsh, keeping only inks whose resulting aspect ratio falls within the configured min/max window (Design Rules..., §11.4); body length is then squares × body width.

Candidate solutions are scored by how close the aspect ratio is to 1:1 (a single square), and the closest is flagged as the recommended solution. The power rating table assumes a 50% width loss during laser trimming (effective power = full rating × 0.5); the standard body widths and their full-power ratings are:

Body width (mm)	Full-power rating (mW)
1.0	62.5
1.4	125
1.8	175

Body width (mm)	Full-power rating (mW)
2.0	250
2.4	350
2.8	500
3.0	600
3.4	800
3.8	1000

11.4 Design Rules...

This dialog controls how sizing is computed for every resistor in the design:

- Aim Value Settings — the aim percentage (0–200%) used in the aim-value formula above.
- Aspect Ratio Limits — the minimum and maximum allowed squares-per-body aspect ratio for a candidate solution to be kept.
- Available Inks — checkboxes for the 11 default sheet resistances (10 Ω/□ through 1 MΩ/□), plus Select All / Select None.
- Ink Layers — a combobox per ink mapping its sheet resistance to a specific drawing layer, or "(default)" to fall back to Setup → Resistor Geometry Rules' fallback ink layer.

Restore Defaults resets the whole dialog; changes are saved immediately into the design's technology file on OK.

11.5 Solutions and the resistor list

Selecting a resistor row shows its candidates in the Solutions tab: a status line (ref des, value, tolerance, aim value, power, width, trim-rated power, solution count) and a table with columns Ink (Ω/□), Squares, L (mm), Width, Trim P (mW), Ratio, and ★ (marks the recommended candidate), alongside a color-coded aspect-ratio diagram (green near 1:1, amber further off, red at the extremes). Click Use This Solution to select a candidate for that resistor — Recalculate and Auto-Select All (which picks the recommended candidate for every resistor at once) are on the main toolbar.

The Summary/BOM tab is a plain-text report: totals (resistor count, how many have/lack a valid solution, how many have a power warning, how many distinct inks are required), an ink-by-ink breakdown, a list of resistors with no valid solution, and a list of power warnings.

11.6 Placing resistors into the layout

Place in Layout... (a chosen solution is required) opens a dialog with X (mm) and Y (mm) fields — pre-filled from a running placement cursor that auto-advances after each placement, so consecutive resistors lay out in a row instead of stacking on top of each other — a Rotation combobox (0/90/180/270), and a Conductor pad layer combobox (defaulting to the technology's configured conductor layer, overridable per placement). Placing draws the ink body, glaze overprint, two conductor pads, and a reference-designator label as a new device cell plus one cell instance.

Place All in Layout... places every resistor in the list that has a chosen solution, one after another in a row from the current placement cursor, spaced clear of one another by footprint plus a margin; any resistor with no solution chosen is skipped and reported. Each placement is recorded as its own undo step.

11.7 Dielectric keep-out and reference designator

When Setup → Technology Properties..., Resistors tab has a dielectric keep-out layer set (field E gives its oversize; a layer of -1 disables it), every generated resistor also carries a keep-out rectangle sized to the resistor ink footprint between its two conductor terminals — the terminal gap (body length) by the resistor's own width, not the wider conductor pad width — grown by E on all four sides. It marks where the inter-layer dielectric must be cleared so the printed ink fires on a bare surface — feed that layer to Tools → Generate Dielectric Fill... (10.4), or draw the keep-out straight on the dielectric layer and it is picked up automatically.

The resistor’s reference designator is drawn once, as a Text shape on the design’s reference-designator layer (Setup → Technology Properties, Ref-Des tab) only. The full designator is also stored on the placed instance’s Label (which NETLIST RUN reads), but the canvas no longer floats that label over the part when the cell already carries the ref-des as real geometry — so it never appears on any layer except the one it is assigned to.

11.8 Import/Export CSV, printing

Import CSV... reads columns `ref_des, value, tolerance, power_mw, body_width` (body_width is optional — it's auto-computed from power if omitted), appending rows and recomputing solutions. Export CSV... writes `ref_des, value, tolerance, power_mw, body_width, ink_rsh, squares, body_length, aspect_ratio, n_solutions, power_ok`.

Print / Preview Report... builds a full HTML report — every resistor and every candidate solution, with the selected/recommended one flagged — and opens it in your system browser, where Ctrl+P prints it (Tkinter has no native print dialog of its own).

12. The Resistor Trim Simulator

The Trim Simulator models what happens electrically as a printed thick-film resistor is laser-trimmed — cutting a kerf into the resistive ink to raise its resistance toward a target value. It solves a finite-difference resistor network over the body on every trim step, reporting resistance, trim sensitivity (dR/d progress), current, and a power-density heat map as the cut advances.

Open it from the Resistor Designer toolbar's **Trim Simulator...** button, which requires a resistor row with a chosen solution selected first. It requires numpy and scipy (see §2.1); if they aren't installed, a dialog explains how to install them instead of crashing. The window is titled “Trim Simulator — <ref des>” and shows the resistor's value, ink sheet resistance, squares, and body dimensions at the top.

12.1 Plunge Cut

A plunge cut starts at one edge of the resistor body and cuts straight in toward the opposite edge, the simplest and most common trim shape.

Control	Meaning
Entry edge	Which edge the cut starts from — bottom or top.
Kerf (µm)	The width of the laser cut.
Depth (% of width)	How far the cut penetrates into the body, as a percentage of the body's width.
Steps	How many simulated increments the cut is broken into — higher gives a smoother resistance/sensitivity curve.
Operating V	The voltage applied across the resistor while simulating, used to compute current and power at each step.

Click Run Plunge to build the simulator for the current resistor and run the cut.

12.2 L-Cut

Once a plunge cut is open (i.e. it has not yet fully severed the resistor), you can extend it with an L-cut: a 90-degree turn from the plunge tip that runs parallel to the entry edge. This is the standard technique for continuing to raise resistance after a straight plunge has gotten close to the far edge, where a plain plunge's sensitivity would otherwise blow up uncontrollably.

Control	Meaning
Turn length (µm)	How far the L-cut runs after the turn.
Direction	Which way the turn goes: +X or −X.
Steps	Simulated increments for the L-cut segment.

Click Add L-Cut to extend the current plunge with this segment.

12.3 Reading the results

Reset clears the simulation and both plots back to their placeholder state. The Scrub step slider (enabled once a run has been made) lets you step back through the recorded history of the cut, updating both visualizations and the status line live as you drag.

The status line shows, for the current step: the step index, resistance (or “OPEN (severed)” if the cut has fully cut the body in two), sensitivity in Ω/µm, current in mA, cumulative cut progress in µm, and which segment kind (plunge or L-cut) the step belongs to.

Heat map panel

Labeled “CURRENT-DENSITY / POWER HEAT MAP,” this shows the power-density distribution across the resistor body at the current step as a black → red → yellow → white gradient, with the cut path itself overlaid as a cyan line — so you can see current crowding around the tip of the cut as it narrows the conducting path.

Resistance / sensitivity history plot

Labeled “RESISTANCE / SENSITIVITY vs CUT PROGRESS,” a solid yellow line traces resistance (Ω) against cumulative cut progress (µm), and a dashed cyan line traces sensitivity (dR/dx, Ω/µm) on the same axis. A white dashed vertical marker tracks the current scrub position, and the resistance trace shows a visible break if the body becomes open-circuit (fully severed) partway through the recorded steps.

12.4 Interpreting sensitivity

Sensitivity is the rate of change of resistance with respect to cut progress — how much resistance each additional micron of cut adds. It stays modest for most of a plunge cut but rises sharply as the cut tip nears the far edge of the body, because the remaining conducting cross-section shrinks toward zero. This is exactly the behavior an L-cut is meant to tame: turning the cut to run parallel to the entry edge keeps removing conducting material — and raising resistance — without approaching a knife-edge of remaining width, keeping sensitivity, and therefore the risk of overshooting the target value, under control.

13. Merging GDS Libraries

File → Merge GDS Libraries... combines two complete .gds files into a third output file — every cell from both source libraries, not just the cells reachable from one top cell. This is a pure file-to-file operation: it never touches whatever design you currently have open.

13.1 Using the dialog

The dialog walks through three file pickers in sequence: “Merge GDS Libraries: first file”, “...second file”, and “...save merged file as”. Pick the two source libraries and the destination path; HLayout reports how many cells were merged and how many needed to be renamed due to name collisions.

13.2 How merging works

- The merged library's layer table and technology parameters are taken from the first ("A") library — both source libraries share HLayout's fixed 1-micron DBU grid and layer numbering, so no rescaling or remapping is ever needed.
- Name collisions: if a cell name exists in both libraries, the first library's copy keeps its original name; the second library's copy is renamed with a numeric suffix (_2, _3, ...) until the name is unique. Every cell instance inside the second library that referenced the renamed cell is automatically updated to point at its new name, so hierarchy and placement stay intact.
- The merged library's top cell prefers the first library's top cell name, falling back to the second library's (renamed if applicable), and finally to the first cell found if neither library declares a top cell.
- Before returning, the merge checks for reference cycles between cells to guarantee the result is a valid, non-circular hierarchy.

13.3 From the command bar or a script

```
MERGE lib1.gds lib2.gds combined.gds
MERGE lib1.gds lib2.gds combined.gds MYLIB   # optional library name

s.merge_gds("lib1.gds", "lib2.gds", "combined.gds", library_name=None)
```

Typical use: combining two independently-built parts libraries — say, one of vias/pads and one of package footprints — into a single attachable library, or folding a colleague's library additions into your master parts file without hand-copying cells.

14. Manufacturing Export

HLayout exports two manufacturing-oriented formats: Gerber for photoplotting/etching and DXF for machines or software that expect AutoCAD-style interchange geometry. Both are reached from the File menu, the Tools menu, and the command bar/scripting layer.

14.1 Gerber export

File/Tools → Gerber Export Sets... opens the Gerber batch export window, where you configure and run one or more named export sets. Each export writes one flattened Gerber file (RS-274X) per visible, non-empty layer, named <prefix>_L<NN>_<layer name>.gbr. Polygons and rectangles are written as filled regions; paths are written as apertured strokes (the cap style — round/square/flush — is not distinguished in the Gerber output, since Gerber apertures don't directly represent it). Text objects are not exported to Gerber; the export reports how many text objects were skipped.

Command-bar equivalent for a one-shot export:

```
GERBER <output_prefix> [cell=<name>]
```

14.2 DXF export

Tools → DXF Export Sets... opens the equivalent batch window for DXF. Unlike Gerber, a DXF export writes a single file containing one DXF layer per HLayout layer; every cell instance is flattened into the output (no block/insert hierarchy is preserved), and coordinates are written in millimeters. Before being written, each layer's polygons, rectangles, and paths are merged through the same shapely pipeline Mask Operations use: a path's width is baked into real, closed polygon geometry (not a DXF “constant width” attribute some readers ignore), and touching or overlapping shapes on one layer become a single clean region — matching the as-fired reality of a screen-printed film rather than a pile of separate outlines. Text becomes native DXF TEXT entities, including every reference designator: one already baked into its own cell as a real Text shape is exported as-is, and a floating SRef label (the more common case, drawn only as a canvas overlay) is synthesized into a real TEXT entity at the same position the canvas computes, on the Ref-Des annotation layer set in Setup → Technology Properties. Naming a layer explicitly in an export job's layer list exports it even if it is currently hidden in Setup → Layers — only the default “every visible layer” behavior is affected by that toggle, since picking a layer by number is a deliberate request. Colors are approximated to the nearest of the 9 standard AutoCAD Color Index colors, since the target format is AutoCAD R12 (AC1009), which predates true color support. Both DXF and Gerber Export Sets default their file-browse dialogs to a gerber/dxf subfolder next to the open design's own file, created automatically the first time it is needed.

14.3 FreeCAD 3D export and the component library

File → Export to FreeCAD (3D Macro)... writes a .FCMacro — a plain Python script for FreeCAD (freecad.org) — that, run inside FreeCAD (Macro → Macros... → Execute, or headlessly via freecadcmd), rebuilds the design as a 3D stack of coloured solids: one solid per printed film, stacked from z = 0 at the elevation its fired-film thickness puts it, each layer merged through the same shapely pipeline DXF export and Mask Operations use, with wire bonds rendered as raised loops. Placed components render as either a fixed-height extruded box or, per the component_models tech-param, a real 3D body substituted from an external STEP file. Requires the shapely package. Output goes into a freecad subfolder next to the open design's file, asking for a folder only if the design has never been saved. When the macro has been written a dialog shows its full path, with Open Folder (opens Explorer with the file selected), Copy Path, and — when FreeCAD is installed — Open in FreeCAD, which starts FreeCAD and runs the macro. Build FreeCAD Component Library... ends with the same dialog.

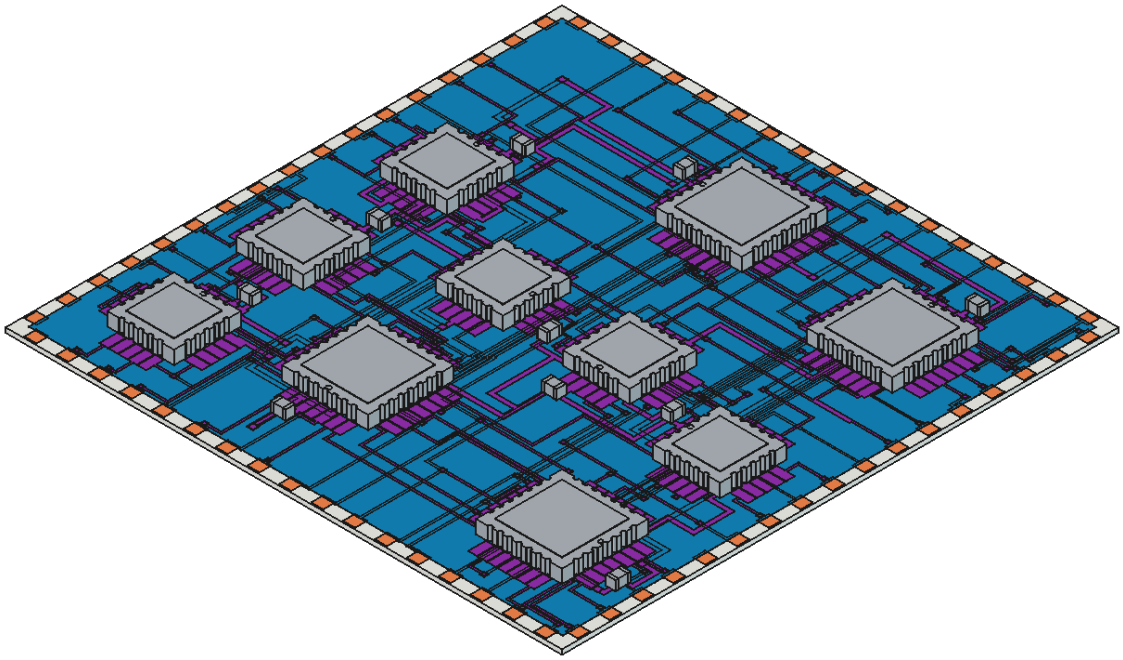


Figure 14.1 – The ALU16 module rebuilt in FreeCAD from the exported macro: substrate, the two conductors and dielectric, I/O pads, castellated LCC20/LCC28 carriers and 0805 capacitors.

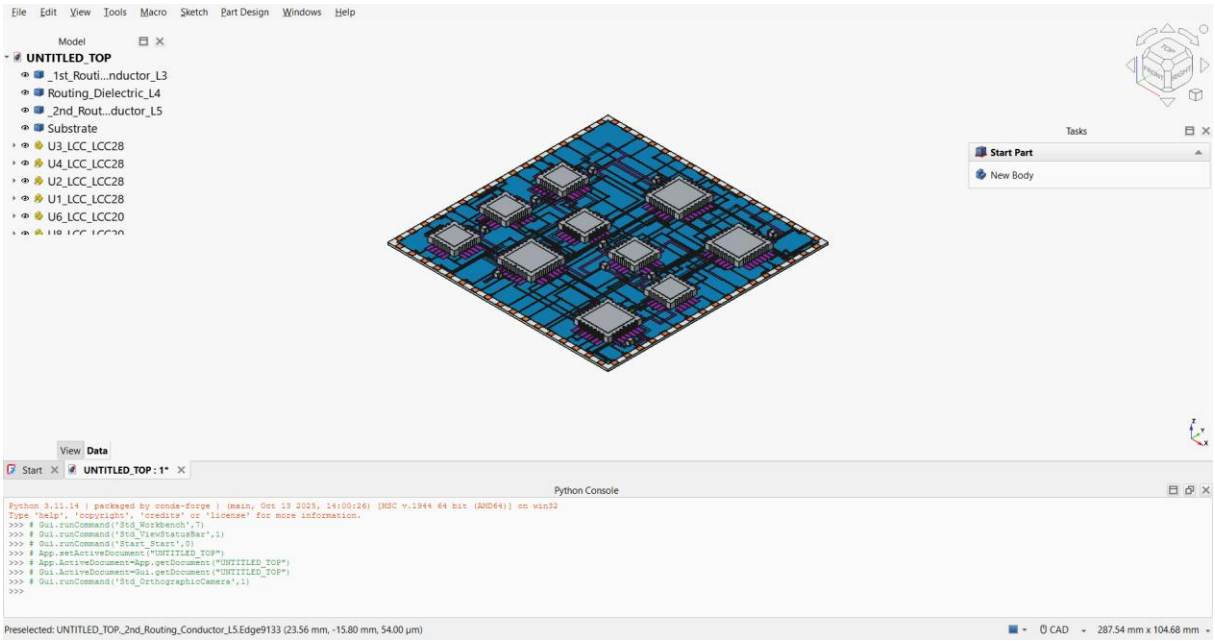


Figure 14.2 – The same model in the FreeCAD window: one object per film layer in the Model tree, plus one per placed component (U1_LCC_LCC28 …).

What the 3D model contains: the substrate (a layer named substrate, alumina or ceramic, or the Thermal setup's substrate layer) is a white slab from minus its thickness up to z = 0 — the substrate layer's own film thickness if one is set, else the Thermal setup's substrate thickness (0.635 mm by default); a cell that draws only the substrate is never shown as a component box. Keep-out markers are never exported: the resistor keep-out layer, the Dielectric Fill rules' extra keep-out layers and any layer named keep-out, and on the dielectric layer itself only the generated fill (datatype 1) is printed dielectric — the shapes you draw there are the fill's boundary and keep-out inputs. Glaze (the resistor glaze layer, or any layer named glaze) is

shown green and see-through so the resistor ink under it stays visible. Isolated pad layers (die pads, header pins — Wire Bonds tab) belong to their part's 3D model and are not stacked as films.

Each component rests on whatever printed film lies under its footprint — found in FreeCAD from the solids already built, so a chip capacitor or a die sits on its conductor rather than floating at the top of the whole film stack. A component model can instead be referenced to the substrate underside (z_ref = substrate_bottom in the component_models entry): a header or bathtub package whose model has its well floor at z = 0 then sits with the substrate inside its well and its pins rising through it. z_offset_um still shifts either reference. Wire bonds are built last and each end lands on the actual surface under it — a die bond pad, a header pin's bond head or a trace — with the loop rising above the higher end. When the macro is run inside the FreeCAD window, STEP models are imported with their own colours (gold pins, green glass, silicon die) and grouped under each part's ref designator; run headless they are read as plain shapes.

Before writing, any placed component cell with no manual override is auto-matched against a built component STEP library. A passive (chip cap, resistor, etc.) is matched by an EIA size code found in its cell's own name — every size the SMD footprint library covers, 0201 through 2512; a chip semiconductor or multi-lead package (the SOT-23/323/89/223/14/16/18 family, and a 20-pin hybrid flatpack) is matched instead by its own pad footprint — pin count plus lead spacing, from either a numbered Rectangle/Polygon pad or a placed cell instance's own pin number (SRef.pin), the way a round lead built from a shared Via Wizard template gets its own number since the template's shared shapes cannot carry one. Matching works regardless of which local axis a footprint happens to lay its lead pitch on, adding an automatic rotation correction when needed. A manual assignment always wins over an auto-match, and a matched package whose STEP file is not built yet is simply left as a flat box rather than a broken reference. A preview of every match (and every miss, and why) is shown before anything is written, but it is purely informational — dismissing it always proceeds with the export.

File → Build FreeCAD Component Library... writes a second macro that, run inside FreeCAD, builds a small library of generic 3D bodies — the standard EIA ceramic chip-capacitor case sizes, a 20-pin sealed metal/ceramic hybrid flatpack, and the SOT gull-wing package family — and exports each as its own STEP file. Ceramic-cap body height defaults to width but is capped at 2.5 mm, since real chip-capacitor thickness plateaus well below that even at the largest case sizes. Unlike the design macro above, this library lives in one folder shared by every design rather than next to whichever one happens to be open, since a parts library only needs building once: the first run asks where to put it and remembers the answer from then on, and Setup → FreeCAD Component Library Folder... views or changes that folder later. Re-running the macro never overwrites a STEP file that already exists, so a hand-replaced vendor model survives.

Chip-and-wire STEP models: examples/step_models/chip_wire/ holds coloured STEP models for the chip_wire library's ceramic chip capacitors (CAP_0201 … CAP_2512, on conductive epoxy dots), the LM741 (µA741) bare die (1.1 × 1.1 × 0.38 mm on a die-attach layer, seven bond pads at the cell's pad positions) and the metal header packages 20_PIN_pkg, 30_PIN_pkg and 40_PIN_pkg. Every model's origin is its 2D cell's origin. The headers are nickel-plated Kovar cans with a 6 mm deep well and a 1 mm floor, 0.5 mm fillets on every edge (inside and outside corners, rim, well floor and bottom surface), 0.63 mm gold pins through 1.61 mm green glass seals, standing 0.75 mm above the floor and running 6.35 mm below, and a gold pin-1 dot on the rim; z = 0 is the well floor, so reference them with z_ref = substrate_bottom. build_step_models.FCMacro in the same folder rebuilds them all; it writes colours into the STEP files, so run it inside FreeCAD or with the GUI layer loaded headless (set QT_QPA_PLATFORM=offscreen, then freecadcmd build_step_models.FCMacro).

Metal header packages in the chip_wire library: 20_PIN_pkg, 30_PIN_pkg and 40_PIN_pkg share a 2.54 mm pin pitch, 20.86 mm row spacing and 24.6 mm × 28.0 mm well and outer width; only the length grows (outer 29.4, 42.1 and 54.8 mm). Pin 1 is at (0.1, 0.2) mm, bottom left, numbered DIP-style: the bottom row 1 to N/2 left to right, the top row on from N/2 + 1 right to left. Each pin is a numbered placement of the

Header_Pin_Round lead, whose pad is on layer 49 (Header_Pads) — list that layer as an isolated pad layer so pins join nets only through wire bonds — and a J-numbered placement gets its ref-des beside pin 1 automatically (§8, Ref-Des to Pin 1).

File → Component Library Manager... is the visual manager for the cell-to-STEP-file mapping: one row per placed component cell, showing the STEP file it is tied to (a Source column says whether that is a manual pin or an auto-match) and its rotation correction. Set STEP File... browses to a .step file and sets its rotation, pinning that cell as a manual override; Clear Override reverts it to auto-matching; Refresh re-runs matching after building more STEP files or placing a new component. This is the first way, in the GUI, to set a manual component-to-model assignment at all.

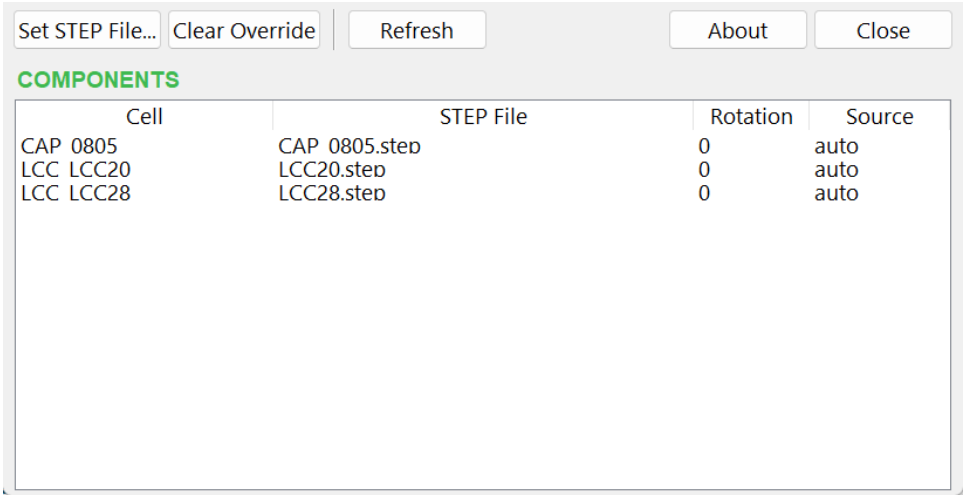


Figure 14.3 – File → Component Library Manager...: each placed component cell, the STEP file it is drawn with, its rotation and whether that came from the auto-match or a manual override.

14.4 Printing the Assembly Drawing

File → Print... (or the PRINT command / Session.print_cell()) renders the current cell as a printable HTML page — a colour-swatch legend plus an SVG drawing of its visible, flattened geometry, reference designators included — and hands it to the system's own print flow (or just opens it in the browser if that fails, so Ctrl+P still works there). scale defaults to auto: a design under about 150 mm across is scaled up to roughly that size instead of printing as a barely-visible speck at “100%”; pass scale=1 to force true physical size.

pinlayers=<n,n,...> draws a numbered label at every pin on those layers — e.g. a bare die's own pad layer, so an assembly operator can see which numbered pad to land a wire bond on. This is deliberately opt-in and layer-scoped: an ordinary part's own 2-terminal pins are also pin-tagged, and labelling those too would clutter every part on the page. A colour that reads fine on the app's own dark canvas (e.g. a chip outline drawn white for contrast) is automatically swapped to black so it stays visible against PRINT's white page.

```
PRINT pinlayers=58
s.print_cell(pin_label_layers=[58])
```

Printed text is drawn at its own height — a 2.5 mm table entry prints 2.5 mm tall — in a narrow font fitted to exactly the width the tables and title block were laid out for, so it never runs over a cell border. Rotation and the magnification of placed cells are honoured, so the ref-des in a 5:1 top view prints at its paper size.

For a complete assembly drawing — pictorial view, resistor and pinout tables, and a titled drawing sheet, all on one printed page — see Chapter 18.

15. Example Generator Scripts

The examples/ folder includes standalone Python scripts that generate ready-to-use parts libraries, including the three documented in this chapter. Each is run directly with Python and writes a self-contained .gds file you can attach and place from just like any other library.

```
python examples/generate_sot_packages.py
python examples/generate_ceramic_capacitors.py
python examples/generate_lcc_packages.py
```

Both scripts print a disclaimer that their dimensions are generic, commonly-used land-pattern sizes derived from a stated formula — not copied from one manufacturer's datasheet — and recommend verifying against a real datasheet or your process before fabrication.

15.1 SOT package generator

Writes `examples/sot_packages.gds`, library name **SOT-PACKAGES**. Generates 7 footprints: SOT-23 (3-lead), SOT-23-5, SOT-23-6, SOT-89 (3-lead with a heat-sink tab), SOT-143 (4-lead), SOT-223 (4-lead power package with a wide tab), and SOT-363/SC-70-6 (6-lead, fine pitch). Each package is one cell containing:

- Copper conductor pads on layer 2, “Pads”.
- A solder-mask opening per pad on layer 3, “Solder Mask” — undersized 0.1 mm per side relative to the pad it covers.
- A body/courtyard outline on layer 63, “Outline”.
- A reference-designator text label on layer 4, “Reference”.

An INDEX cell places every package in a row at 10 mm pitch, for an at-a-glance preview of the whole set.

15.2 Ceramic capacitor generator

Writes `examples/ceramic_capacitors.gds`, library name **CERAMIC-CAPS**. Generates 13 standard EIA-size ceramic (MLCC) chip capacitor footprints: 01005, 0201, 0402, 0603, 0805, 1206, 1210, 1808, 1812, 1825, 2220, 2225, and 2512, using the standard published EIA body length × width table. Each cell contains two rectangular termination pads on layer 1 (“L1”), an outline on the Outline layer, and a reference label on the Reference layer.

Pad geometry is derived from body size by a stated formula rather than one vendor's numbers: each pad covers the outer 40% of the body length plus a toe overhang capped at 0.3 mm, leaving a 20% gap in the center between the two pads; pad width matches body width. The same INDEX-cell convention (10 mm row spacing) previews the full set.

15.3 LCC package generator

Writes examples/lcc_packages.gds, library name LCC-PACKAGES. Generates 7 square Leadless Chip Carrier (LCC) footprints — LCC-8, LCC-12, LCC-16, LCC-20, LCC-24, LCC-28, LCC-32 (pin count = 4 × terminals per side) — dimensioned against a real JEDEC 20-terminal LCC datasheet (0.050” pitch, cross-checked against TI’s own LCCC catalog for the 20- and 28-pin body sizes).

Each package cell contains:

- Copper land pads, pin-numbered 1..N counter-clockwise from the bottom-left corner, straddling the body edge the way a real castellated terminal wraps it, plus the plated half-via castellation scallop at each terminal, unioned into the same copper.

- A chamfered body outline whose corners are cut at 45° — pin 1’s own corner gets a distinctly smaller chamfer as the part’s orientation mark — with the same castellation scallop cut into the outline at every terminal, so the body shows the real physical notch a ceramic LCCC is moulded/machined with, not just a plain chamfered square. On a part with enough pins that this notched outline exceeds the 200-point-per-Path limit, it's written as 2–3 Path pieces that share endpoints, still tracing one continuous outline.

A smaller inset top-surface outline showing the stepped-down face a real LCC's lid sits into, a pin-1 orientation dot, and a reference-designator placeholder text “U”.

If a custom --pitch/--pad-width/--pad-depth/--corner-margin combination would put two adjacent corner pads' copper closer than the configured minimum clearance (0.3 mm by default), the generator automatically chamfers the facing corner of both pads just enough to restore it — with the default parameters this never triggers and pads stay plain rectangles.

The same INDEX-cell convention (16 mm row spacing) previews the full set. --start/--stop/--step select which pin counts to generate (default 8..32 in steps of 4); --pitch/--pad-width/--pad-depth/--corner-margin override the default terminal geometry.

15.4 Using a generated library

```
# Command bar
ATTACH examples/sot_packages.gds
PLACE SOT-23 0,0

# Python
s.attach_library("examples/ceramic_capacitors.gds")
s.place_cell("0805", (0, 0))
```

16. Keyboard Shortcut Reference

HLayout has no separate shortcuts document — every shortcut below is also shown as an accelerator label directly on its menu item. This chapter simply collects them in one place.

Shortcut	Action	Where
Ctrl+N	New (blank library)	File menu
Ctrl+O	Open GDS...	File menu
Ctrl+S	Save GDS...	File menu
Ctrl+Shift+S	Save As...	File menu
Ctrl+Z	Undo	Edit menu
Ctrl+Y / Ctrl+Shift+Z	Redo	Edit menu
F	Fit view to current cell	View menu / toolbar
+	Zoom in	View menu
−	Zoom out	View menu
G	Toggle grid	View menu / toolbar
W	Zoom Window tool (one-shot)	View menu
P	Pan tool (drag to move the view)	View menu
Ctrl+Arrows	Pan the view a quarter screen	View → Pan Step
F1	Command Reference (opens docs/help.html)	Help menu
Delete	Delete Selected	Edit menu / toolbar / canvas
Up / Down / Left / Right	Nudge selection by one grid unit	Canvas (Select mode)
R	Rotate selection 90°	Canvas (Select mode)
Escape	Cancel the shape currently being drawn	Canvas
Enter	Finish a polygon, path, or route	Canvas
Backspace	Remove the last placed point while drawing	Canvas
Space	Drop a via mid-route and switch conductor layer	Canvas (Route tool)
Middle-drag or right-drag	Pan the canvas	Canvas
Mouse wheel	Zoom at cursor	Canvas

17. Appendix: Python Scripting API by Category

This appendix maps out the `Session` class's methods by category, matching the organization used in Help → Command Reference (F1), so you can find the right area quickly before looking up an individual method's exact signature there. All methods are called on a Session instance, e.g. `s.rect(...)`.

17.1 Construction & I/O

- `save, save_design` — write the design (and, for save_design, its .hlaux sidecar) to a .gds file.
- `export_gerber, run_gerber_export_batch` — one-shot or batch Gerber export.
- `export_dxf, run_dxf_export_batch` — one-shot or batch DXF export.
- `print_cell` — render the current cell as a printable HTML page and return its path (§14.4); backs PRINT and File → Print...
- `export_freecad` — write the FreeCAD 3D macro (§14.3); auto-matches placed components against the STEP library and returns a MatchReport alongside writing the .FCMacro file.
- `match_freecad_components` — run just the auto-matching step (used internally by export_freecad and the Component Library Manager) and get back its MatchReport without writing a macro.
- `generate_component_library` — write the macro that builds the shared component STEP library (§14.3).

17.2 Technology

- `load_technology, save_technology` — load/save the layer stack and technology parameters as a .json file.

17.3 Attached libraries

- `attach_library, detach_library, list_attached_libraries` — manage the up-to-2 attached reference libraries.
- `merge_gds` — merge two .gds files into a third (Chapter 13).
- `create_via_cell, create_pad_cell` – the Python equivalents of the Via Wizard and Pad Wizard (§8.1–8.2); create_pad_cell auto-numbers pins "1"/"2" when the grid is exactly two pads (§8.4).

17.4 Library & cell queries

- `list_library, find_cells` — enumerate cells in the current design, or search across it and attached libraries by name substring.

17.5 Change notification

- on_change — register a callback invoked whenever the session's data changes (this is how the GUI repaints after a script edits the design).

17.6 Undo/redo

- `checkpoint, batch` — mark or group edits into a single undo step.
- `undo, redo, can_undo, can_redo` — step the undo history and query whether a step is available.

17.7 Cell & layer context

- `new_cell, use_cell` — create or switch the current cell.
- `set_layer_style` — set a layer's color/fill/stipple/name/visibility.

17.8 Shapes

- `rect, polygon, circle, path, text` — the five drawing primitives, matching the Create menu tools.

17.9 Cell instances (SRef/ARef)

- `sref, place_cell` — place a single cell instance; both accept an optional `label=` to set the instance's ref designator at placement time (read by Netlist Extraction, §10.3, and shown in Edit Selected...'s Label field). Both also accept an optional `pin=` to stamp SRef.pin directly on the instance — for a round lead built from a shared Via Wizard template whose own shapes cannot carry a per-placement pin number (§8.4). A connector/header label (J, P, X, CN, CON, HDR + number) places the ref-des beside pin 1; `refdes_to_pin1` does the same for parts already placed.
- `aref, place_cell_array` — place a rectangular array of a cell.
- `smash_aref` — explode an array instance into individual instances.
- `remove` — delete a shape or instance.

17.10 Vias & resistors

- `drop_via` — place a via at a point on the currently routed conductor layers.
- `generate_resistor` — the Python equivalent of the RESGEN command (Chapter 11's underlying primitive); auto-numbers its two pads "1"/"2" and, when `ref_des` is given, sets the placed instance's label to it (§8.4).
- `generate_resistor_table` — build a resistor BOM table cell (REF DES / VALUE / TOL% / AIM VALUE / INK) from the saved Resistor Designer rows; generic and re-runnable (Chapter 18).

17.11 Resistor Trim Simulator

The Trim Simulator's engine is exposed directly as the `TrimSimulator` class (in `hlayout.core.trim_sim`) for scripted analysis outside the GUI — construct one from a body length/width and sheet resistance, then call `add_plunge(...)` and/or `add_l_cut(...)` to run cuts and inspect the resulting resistance/sensitivity history and power-density grids programmatically. See Help → Command Reference for the full constructor and method signatures.

17.12 Mask Operations & DRC

- `mask_op, run_mask_op_batch` — one-shot or batch mask operations (Chapter 10.1).
- `run_drc` — run the saved DRC rule set (Chapter 10.2).
- `mask_op_top_view` — draw the top-down, hidden-line-removal pictorial assembly view (§10.5); backs TOPVIEW and Tools → Top View... `top_view_from_settings` — rebuild it from the settings saved in Tools → Top View... (TOPVIEW with no arguments); `top_view_layer_roles` — the {layer: role} the role-based stacking uses.
- `clear_layer` — delete every shape on a layer from a cell's own shapes; backs CLEAR.

- `convert_rects_to_outline` — replace a Rectangle/Polygon with an equivalent closed-ring, stroked outline Path; backs OUTLINE.

17.13 Layer remap & geometry queries

- `remap_layers` — bulk move shapes from one layer to another.
- `shapes_on_layer` — enumerate shapes on a given layer.
- `bbox` — compute a bounding box for a cell or selection.

17.14 Pin Numbering

- `list_pins, set_pin – enumerate a cell's pads with their pin/layer/bbox, or set one pad's pin by shape index (§9.1, §8.4). Neither covers a placed instance's own SRef.pin – set that via sref/place_cell's pin= argument instead (§17.9).`
- `generate_pin_function_table` — build a pinout reference table cell -- PIN # / PIN FUNCTION column pairs in groups of pins_per_group pins, blank by default for hand fill-in; generic and re-runnable (Chapter 18).

17.15 Netlist Extraction

- `extract_netlist, export_netlist, export_netlist_csv – extract a netlist from copper connectivity plus pin/ref-designator identity, and write it to a file: CSV, text report, KiCad (.net) or Tango/Protel (.tng) (§10.3, §19.16). Called with no layer list, extract_netlist uses the design's saved Conductor selection from Tools → Extract Netlist... (techfile.load_netlist_settings / save_netlist_settings). Wire bonds join what their two ends land on, and the Wire Bonds tab's isolated pad layers (techfile bond rules die_pad_layers) are kept apart from the substrate copper. Each Net carries .bonds, .open_bond_ends and .location, and each pin an .xy position, so problems can be located. wire_bond, auto_route – draw a wire bond between two pads (8.5), and automatically route a clear orthogonal trace between two points on one layer around every other shape on it. Pin identity for extract_netlist comes from a pad's own Rectangle/Polygon.pin as well as a placed instance's SRef.pin, attributed to the enclosing part's ref designator either way.`

17.16 Drawing Sheets

- `generate_drawing_sheet` — build a standard A4/A3 drawing sheet cell: a ruled border frame plus a title block (TITLE, DWG NO/ISSUE, DRAWN BY/CHECKED BY/APPROVED BY each with its own DATE), every field blank by default; distances are given in real millimetres, not DBU (Chapter 18.3); backs SHEET.
- `generate_assembly_drawing` — build (or rebuild) the complete assembly drawing: drawing sheet, scaled top view, pin function table and resistor table, each in its own cell, all placed into one assembly cell; settings saved with the design (Chapter 18.5); backs ASSEMBLY and Tools → Assembly Drawing...
- `generate_resistor_table, generate_pin_function_table` — the BOM and pinout tables (Chapter 18.1, 18.2); text is centred in every row and resistor values use the letter-and-digit code (1K20, 680K, 10M4).

17.17 Thermal Analysis

- `thermal_parts`, `set_thermal_power` — list the resistors and components that can dissipate power, and set a part's power by reference designator.
- `run_thermal` — steady-state thermal analysis of a cell (Chapter 20); returns a result with each part's peak and average temperature, its limit and a pass or fail.
- `export_thermal_report`, `export_thermal_csv`, `draw_thermal_contours` — write the result as an HTML report or a CSV, or draw its isotherms onto a layer; backs the THERMAL commands.

For exact call signatures, keyword arguments, and a runnable example for every method above, open Help → Command Reference (F1) inside the app and search or scroll to the matching category — it mirrors this appendix's structure exactly. Chapter 19 shows each method in a complete, tested script.

18. Assembly Drawings: RESTABLE, PINTABLE, SHEET and ASSEMBLY

RESTABLE, PINTABLE, and SHEET are three generic, re-runnable cell generators for the documents that accompany a printed assembly drawing — a resistor BOM table, a pinout reference table, and a titled drawing sheet border. All three follow the same convention: they build (or completely rebuild) a named cell from scratch every time they run, so editing the underlying data — the saved resistor list, a pin's function, a title-block field — and re-running just refreshes the cell in place. Place the result wherever it's needed with SREF, alongside the top-down pictorial view from TOPVIEW (§10.5), and print the whole thing with PRINT (§14.4). To build a whole drawing from these pieces in one step, see §18.5.

18.1 Resistor BOM table (RESTABLE)

RESTABLE builds a REF DES / VALUE (OHMS) / TOL% / AIM VALUE / INK (OHMS/SQ) table, reading the current rows straight from the Thick-Film Resistor Designer's own saved list (Chapter 11) — the same data its own table shows. “Aim value” uses the Designer's own formula (value × (aim_pct% + tolerance%)), so the printed table always matches whatever it used to pick each ink. Every cell, header and data alike, is drawn as a Text shape centred within its own column, with ruled grid lines giving every cell a real bordered box. An optional title row (default “RESISTOR TABLE”) spans the full table width, centred by default.

Resistances — the VALUE, AIM VALUE and INK columns — are written in the letter-and-digit (RKM) code used on drawings, with the multiplier letter standing in for the decimal point and three significant figures: 10, 100, 1K20, 680K, 10M4, 4R70. Whole values under 1 kΩ are written plainly; a value that needs a fourth figure to be exact keeps it (1K234). Text in every row of both tables is centred vertically as well as horizontally, with equal space above and below.

```
RESTABLE RESTABLE_1 62 title="RESISTOR TABLE"
s.generate_resistor_table("RESTABLE_1", 62, title="RESISTOR TABLE")
```

overwrite defaults to on (unlike almost every other “cell already exists” guard in the application) precisely because this table is meant to be rebuilt: after adding, editing, or resizing resistors, run RESTABLE again with the same cell name and the table refreshes to match.

18.2 Pinout reference table (PINTABLE)

PINTABLE builds a pinout reference table for a 20/30/40/50-pin (or any multiple-of-10, or a custom group size via pergroup=) package: one PIN # / PIN FUNCTION column pair per group of pins, groups placed side by side left to right. Pin numbers run 1..pin_count; the FUNCTION column starts blank on every pin — a plain fill-in-by-hand template for an assembly operator or engineer to complete, or pre-fill programmatically with a functions={pin_number: "name"} mapping in a script. Groups sit edge to edge by default (groupgap=0); pass groupgap=<dbu> to separate them into visually distinct boxes.

```
PINTABLE PINOUT_1 62 20
s.generate_pin_function_table("PINOUT_1", 62, 20, functions={1: "VCC", 11: "GND"})
```

18.3 Drawing sheet and title block (SHEET)

SHEET builds a standard drawing sheet cell: a ruled border frame sized to a real ISO paper size (A3 portrait by default; A4 also fits — A2 and larger don't, since hlayout's own design database is capped at 500 × 500 mm), with a title block ruled across the full width of the bottom of the frame. The title block has five rows: TITLE (spanning the full width); DWG NO alongside ISSUE; then DRAWN BY, CHECKED BY, and APPROVED BY, each paired with its own DATE column. Unlike RESTABLE/PINTABLE, every SHEET distance (margin, title-

block height, text height, line width) is given in real millimetres, matching how a physical drawing sheet is actually specified, rather than the arbitrary DBU grid the rest of the layout uses.

Every title-block field reads “LABEL:” when blank, or “LABEL: value” once filled in — pass title=/dwgno=/issue=/drawnby=/drawndate=/checkedby=/checkeddate=/approvedby=/approveddate= to pre-fill any of them. The area above the title block, inside the margin, is left completely empty: that's where the actual pictorial view, tables, and any other drawing content get placed.

```
SHEET DWGSHEET 62 title="NOTCH FILTER ASSEMBLY" dwgno=NF-001 drawnby="A. BROWN"
s.generate_drawing_sheet("DWGSHEET", 62,
    title="NOTCH FILTER ASSEMBLY", dwg_no="NF-001", drawn_by="A. BROWN")
```

18.4 Putting it together: a complete assembly drawing

A typical assembly-drawing cell combines all of the above: a SHEET for the border and title block, a TOPVIEW result placed inside it for the pictorial view, and RESTABLE/PINTABLE tables placed alongside it for the BOM and pinout — all as separate cells, each SREF'd into one top-level drawing cell, so any one of them can be regenerated independently without disturbing the others.

```
SHEET DWGSHEET 62 title="NOTCH FILTER ASSEMBLY" drawnby="A. BROWN"
CLEAR 63
TOPVIEW result=63 exclude=61 hatch=100
RESTABLE RESTABLE_1 62
PINTABLE PINOUT_1 62 20
CELL DRAWING_1
SREF DWGSHEET 0,0
SREF RESTABLE_1 310000,10000
SREF PINOUT_1 310000,120000
PRINT cell=DRAWING_1 pinlayers=58
```

Re-running any one line — after adding a resistor, wire-bonding a die, or correcting a title-block field — rebuilds just that cell; the SREFs in DRAWING_1 pick up the change automatically the next time it's viewed or printed.

18.5 Tools → Assembly Drawing: the whole drawing in one step

Tools → Assembly Drawing... (or the ASSEMBLY command / Session.generate_assembly_drawing()) does everything in §18.4 at once. It builds the drawing sheet, a top view of the design, the pin function table and the resistor table, each as its own cell (ASSY_SHEET, ASSY_TOPVIEW, ASSY_PINTABLE, ASSY_RESTABLE), and places them all into one drawing cell, ASSEMBLY by default. Build switches the view to the finished drawing; Print Preview opens it as a printable page.

The window has four groups of settings:

- **Sheet.** Paper size (A4 or A3), orientation, the drawing layer (defaults to the Top View result layer), the design cell to draw and the name of the drawing cell.
- **Title block.** Title, drawing number, issue, and drawn / checked / approved by, each with a date.
- **Top view.** Scale — Auto picks the largest standard scale that fits (100:1 down to 1:10), or choose one. Line width, hatch pitch and ref-des height are paper millimetres: the view is redrawn so they come out that size at the chosen scale. A “TOP VIEW SCALE n:1” label goes under the view unless you turn it off.
- **Tables.** Placement, table text height, and which tables to include. Leave Pins blank to take the pin count from the package placed in the design (a cell named like 20_PIN_pkg, LCC28 or CFP-16); Per group sets how many pins go in each column pair. Pin functions are typed one per line, e.g. 1 = VCC.

Placement decides where the tables go. On a landscape sheet they stack down the right-hand side of the drawing area; on a portrait sheet they sit in a row under the top view, wrapping onto further rows if they don't fit across, with the title block across the bottom. Choosing portrait switches Placement to bottom automatically, and landscape back to side; Auto keeps whichever gives the bigger view.

Every field is saved as you type and when the window closes. The design keeps all of them, so reopening the window or running a bare ASSEMBLY rebuilds exactly the last drawing — after moving parts or editing resistors, just press Build again. Your house style is also remembered for new designs: sheet size, orientation, placement, scale, line and text sizes, which tables to include, and the drawn / checked / approved names. The title, drawing number, issue, dates, pin count, pin functions, layer and design cell stay with each design.

```
ASSEMBLY size=A3 orientation=portrait layout=bottom title="NOTCH FILTER" dwgno=NF-
001
ASSEMBLY

r = s.generate_assembly_drawing(size="A3", orientation="portrait",
                                title="NOTCH FILTER", dwg_no="NF-001",
                                pin_functions={1: "VCC", 20: "GND"})
print(r["scale"], r["layout"], r["warnings"])
```

generate_assembly_drawing() returns the drawing cell, the scale used, the table placement, the part cells and any warnings — for example when there is no package to take the pin count from, or no resistors saved in the Resistor Designer, in which case that table is left out rather than the build failing. The whole build is one undo step.

19. Python API Cookbook

This chapter documents the Python scripting API by example. Chapter 9 introduces the `Session` object and Chapter 17 lists its methods by category; this chapter shows each of them in use, with the output it produces. Every example was run against the HLayout build this manual describes, and the text under an **Output** heading is what that run printed.

19.1 Conventions used in the examples

Each example is a complete script: save it as a `.py` file and run it with `python my_script.py` from the HLayout folder (or from anywhere, with the HLayout folder on `PYTHONPATH`). Three other ways to run the same code:

- Script Console tab (or the command bar): a session is already bound to `s` (and `hl`), so leave out the `import` and `Session.new(...)` lines and the script edits the design that is open on screen.
- Tools → Run Script File...: runs a `.py` file in that same console namespace, so the same rule applies.
- `PY <python source>` in the command bar: one line of Python.

Things that hold for every method:

- Coordinates and sizes are in database units (DBU). 1 DBU = 1 micron = 0.001 mm, so 2 mm is `2000`. `hlayout.core.units` has `mm_to_dbu()` and `dbu_to_mm()` if you would rather think in millimetres. The exceptions take real millimetres and say so in the argument name (`margin_mm`, `text_height_mm`).
- A layer argument accepts a layer number (0-63) or the name of a layer that has already been named.
- A point is an `(x, y)` tuple. A bounding box is `(x1, y1, x2, y2)`.
- Drawing methods add to the current cell, the one most recently passed to `new_cell()` or `use_cell()`.
- Every editing method is one undo step, and returns the object it created or a small summary of what it did.
- Errors are raised as `hlayout.core.errors.LayoutError` (or a subclass: `LayerError`, `GeometryError`, `CellError`, `LibraryError`, `BoundsError`). In the console a traceback is printed and the application carries on.
- Mask operations, DRC, netlist extraction, auto-routing, dielectric fill, DXF and FreeCAD export need the `shapely` package.

```
from hlayout.scripting.api import Session
from hlayout.core.units import mm_to_dbu, dbu_to_mm
from hlayout.core.errors import LayoutError

s = Session.new("DEMO")
s.new_cell("TOP")

print(mm_to_dbu(2.5), dbu_to_mm(1270))

try:
    s.rect("Conductor", 0, 0, 100, 100)     # no layer has that name yet
except LayoutError as exc:
    print("refused:", exc)
```

```
s.set_layer_style(1, name="Conductor")
print(s.rect("Conductor", 0, 0, 100, 100))
```

Output

```
2500 1.27
refused: no layer named 'Conductor'
Rectangle(layer=1, points=[(0, 0), (100, 0), (100, 100), (0, 100)], datatype=0, pin='')
```

19.2 Sessions and files

```
Session.new(name="HLIB")                -> Session
Session.open(path)                      -> Session
Session.open_design(path)               -> Session
s.save(path, library_name=None)
s.save_design(path)                     -> sidecar path
```

`Session.new()` starts an empty library. A new library has no cells, so create one before drawing.

```
from hlayout.scripting.api import Session

s = Session.new("DEMO")              # an empty library called DEMO
s.new_cell("TOP")                    # create a cell and make it current
s.rect(1, 0, 0, 2000, 1000)          # layer 1, 2.0 x 1.0 mm
s.circle(2, (1000, 500), 200)        # layer 2, 0.2 mm radius
s.text(3, (0, 1200), "DEMO", height=200)
s.save("demo.gds")

print(s.library.name, s.current_cell_name, s.bbox())
```

Output

```
DEMO TOP (0, 0, 2000, 1400)
```

`Session.open()` reads any GDSII file and makes the library's top cell current. `library.top_cells()` returns the cells that no other cell places; call `use_cell()` to work in a different one.

```
from hlayout.scripting.api import Session

s = Session.open("demo.gds")
print(s.current_cell_name, [c.name for c in s.library.top_cells()])

s.use_cell("TOP")
for shape in s.cell.shapes:
    print(type(shape).__name__, shape.layer, shape.bbox())
```

Output

```
TOP ['TOP']
Rectangle 1 (0, 0, 2000, 1000)
Polygon 2 (800, 300, 1200, 700)
Text 3 (0, 1200, 480, 1400)
```

`save()` writes a bare `.gds`. `save_design()` also writes a sidecar (`<file>.gds.hlaux`) holding the layer stack, the technology parameters and a copy of every attached library, and `open_design()` restores them. Use the pair for work in progress and plain `save()` to publish a parts library. This is the difference between File → Save GDS... and File → Save As Library...

```
import os
from hlayout.scripting.api import Session

s = Session.new("DEMO")
s.new_cell("TOP")
```

```
s.set_layer_style(1, name="Conductor", color="#d4a017")
s.rect("Conductor", 0, 0, 2000, 1000)

sidecar = s.save_design("design.gds")
print(os.path.basename(sidecar))

again = Session.open_design("design.gds")     # layer names restored
print(again.had_sidecar, again.library.layers[1].name, again.attach_errors)

bare = Session.open("design.gds")             # geometry only
print(repr(bare.library.layers[1].name))
```

Output

```
design.gds.hlaux
True Conductor []
'L1'
```

Passing `library_name` to `save()` renames the library as it is written, which is how a design becomes a named parts library.

```
from hlayout.scripting.api import Session

s = Session.new("SCRATCH")
s.new_cell("PAD_1MM")
s.rect(1, -500, -500, 500, 500, pin="1")
s.save("my_parts.gds", library_name="MY-PARTS")

print(Session.open("my_parts.gds").library.name)
```

Output

```
MY-PARTS
```

19.3 Cells

```
s.new_cell(name)        -> Cell     create a cell and make it current
s.use_cell(name)        -> Cell     switch to an existing cell
s.cell                              the current Cell object
s.current_cell_name                 its name
s.list_library(source="current")    -> [{'name','shapes','srefs','arefs','bbox'}, ..]
s.find_cells(query)                 -> [{'source','name'}, ...]
s.bbox(cell_name=None)              -> (x1, y1, x2, y2)
```

```
from hlayout.scripting.api import Session

s = Session.new("DEMO")
s.new_cell("PAD")
s.rect(1, -250, -250, 250, 250)

s.new_cell("TOP")                       # TOP is now current
s.sref("PAD", (0, 0))
s.sref("PAD", (2000, 0))

for info in s.list_library():
    print(info)

print(s.find_cells("pa"))               # case-insensitive substring
print(s.bbox(), s.bbox("PAD"))          # s.bbox() includes placed instances

s.use_cell("PAD")                       # go back and edit PAD
```

```
s.circle(2, (0, 0), 100)
print(s.current_cell_name, len(s.cell.shapes))
```

Output

```
{'name': 'PAD', 'shapes': 1, 'srefs': 0, 'arefs': 0, 'bbox': (-250, -250, 250, 250)}
{'name': 'TOP', 'shapes': 0, 'srefs': 2, 'arefs': 0, 'bbox': None}
[{'source': 'current', 'name': 'PAD'}]
(-250.0, -250.0, 2250.0, 250.0) (-250, -250, 250, 250)
PAD 2
```

The bbox that `list_library()` reports covers a cell's own shapes only, which is why TOP shows `None` above; `s.bbox()` includes everything the cell places.

A `Cell` holds three lists (`shapes`, `srefs` and `arefs`), which a script can read directly.

```
from hlayout.scripting.api import Session

s = Session.open("demo.gds")
for cell in s.library.cells.values():
    print(cell.name, len(cell.shapes), len(cell.srefs), len(cell.arefs),
          cell.bbox())
```

Output

```
TOP 3 0 0 (0, 0, 2000, 1400)
```

19.4 Layers and technology

```
s.set_layer_style(layer, name=None, color=None, stipple=None,
                  fill=None, visible=None, selectable=None)   -> Layer
s.load_technology(path)       -> dict
s.save_technology(path)
s.show_layers_of_cell(cell_name)
s.library.tech_params         dict of global technology parameters
```

`color` is `#rrggbb`, `stipple` is a pattern number 0-8, and `fill`, `visible` and `selectable` are booleans. Once a layer has a name, every method accepts that name in place of the number.

```
from hlayout.scripting.api import Session

s = Session.new("DEMO")
s.new_cell("TOP")

stack = [
    (1, "Conductor",  "#d4a017", 0),
    (2, "Dielectric", "#3a7bd5", 3),
    (3, "Resistor",   "#202020", 1),
    (4, "Glaze",      "#3cb371", 5),
]
for number, name, color, stipple in stack:
    s.set_layer_style(number, name=name, color=color, stipple=stipple)

s.rect("Conductor", 0, 0, 1000, 500)
s.set_layer_style("Glaze", visible=False)

layer = s.library.layers[4]
print(layer.name, layer.color, layer.stipple, layer.visible)
print(s.library.layers.resolve("Resistor"))
```

Output

```
Glaze #3cb371 5 False
3
```

`show_layers_of_cell()` switches on every layer a cell draws on, which is useful after placing a part whose layers were hidden.

```
from hlayout.scripting.api import Session

s = Session.new("DEMO")
s.new_cell("PART")
s.rect(7, 0, 0, 100, 100); s.rect(8, 0, 0, 50, 50)
for layer in (7, 8):
    s.set_layer_style(layer, visible=False)

s.show_layers_of_cell("PART")
print(s.library.layers[7].visible, s.library.layers[8].visible)
```

Output

```
True True
```

A technology file is the layer stack plus the technology parameters as JSON. Save it once and load it at the start of each new design.

```
from hlayout.scripting.api import Session

a = Session.new("FIRST")
a.set_layer_style(1, name="Conductor", color="#d4a017")
a.set_layer_style(3, name="Resistor", color="#202020")
a.library.tech_params["substrate"] = "96% alumina"
a.save_technology("process.json")

b = Session.new("SECOND")
data = b.load_technology("process.json")
print(sorted(data))
print(b.library.layers[3].name, b.library.tech_params["substrate"])
```

Output

```
['format', 'layers', 'params', 'version']
Resistor 96% alumina
```

The saved rule sets that the Setup menu edits (resistor geometry, wire bond, route layers, dielectric fill, DRC rules, export sets) and the Extract Netlist layer selection live in tech_params too. Read and write them through the load_... and save_... functions in hlayout.io.techfile so the keys stay valid. The wire-bond rules include die_pad_layers, the isolated pad layers that netlist extraction keeps apart from the substrate copper.

```
from hlayout.scripting.api import Session
from hlayout.io import techfile

s = Session.new("DEMO")
print(techfile.load_bond_rules(s.library))

rules = techfile.load_bond_rules(s.library)
rules.update(layer=20, width=25, pad=75, min_length=300, max_length=4000)
techfile.save_bond_rules(s.library, rules)

print(techfile.load_bond_rules(s.library))
print(techfile.load_resistor_geometry_rules(s.library))
```

Output

```
{'layer': -1, 'width': 50, 'pad': 100, 'min_length': 300, 'max_length': 5000,
'die_pad_layers': []}
{'layer': 20, 'width': 25, 'pad': 75, 'min_length': 300, 'max_length': 4000, 'die_pad_layers':
```

```
[]}
{'overlap_a': 200, 'pad_b': 500, 'space_c': 200, 'glaze_oversize': 50, 'keepout_oversize': 0,
'resistor_layer': 30, 'conductor_layer': 31, 'glaze_layer': 32, 'doc_layer': 33,
'keepout_layer': -1, 'ink_layers': {}}
```

19.5 Drawing shapes

```
s.rect(layer, x1, y1, x2, y2, datatype=0, pin="")          -> Rectangle
s.polygon(layer, points, datatype=0, pin="")               -> Polygon
s.circle(layer, center, radius, segments=32, datatype=0)   -> Polygon
s.path(layer, points, width, cap="flush", datatype=0)      -> Path
s.text(layer, origin, text, height=10, font=0, rotation=0.0, datatype=0) -> Text
```

A polygon or path takes at most 200 points. `segments` for a circle must be 8, 16, 32 or 64. `cap` is `"flush"`, `"square"` or `"round"`. Each call returns the shape object, which you can keep to inspect or to pass to `remove()`.

```
from hlayout.scripting.api import Session

s = Session.new("DEMO")
s.new_cell("SHAPES")

r = s.rect(1, 0, 0, 1000, 500)
p = s.polygon(1, [(1500, 0), (2500, 0), (2500, 250), (2000, 500), (1500, 500)])
c = s.circle(2, (3500, 250), 250, segments=64)
w = s.path(3, [(0, 1000), (2000, 1000), (2000, 1500)], width=100, cap="round")
t = s.text(4, (0, 2000), "REV A", height=200)

for shape in (r, p, c, w, t):
    print(type(shape).__name__, shape.layer, shape.bbox())
print(len(c.points), w.width, w.cap, t.text)
```

Output

```
Rectangle 1 (0, 0, 1000, 500)
Polygon 1 (1500, 0, 2500, 500)
Polygon 2 (3250, 0, 3750, 500)
Path 3 (-50.0, 950.0, 2050.0, 1550.0)
Text 4 (0, 2000, 600, 2200)
64 100 PathCap.ROUND REV A
```

A loop is the usual reason to script. This draws a 4 x 3 grid of 0.5 mm pads on a 1.27 mm pitch and numbers them as pins.

```
from hlayout.scripting.api import Session

s = Session.new("DEMO")
s.new_cell("PAD_GRID")

pitch, size, n = 1270, 500, 0
for row in range(3):
    for col in range(4):
        n += 1
        x, y = col * pitch, row * pitch
        s.rect(1, x - size // 2, y - size // 2, x + size // 2, y + size // 2,
               pin=str(n))
```

```
print(len(s.cell.shapes), s.bbox())
print([(p["index"], p["pin"]) for p in s.list_pins()][:4])
```

Output

```
12 (-250, -250, 4060, 2790)
[(0, '1'), (1, '2'), (2, '3'), (3, '4')]
```

Geometry that is easier to compute than to draw works the same way. This is a 12-tooth ring, built as one polygon.

```
import math
from hlayout.scripting.api import Session

s = Session.new("DEMO")
s.new_cell("GEAR")

teeth, r_in, r_out = 12, 800, 1000
points = []
for i in range(teeth * 2):
    radius = r_out if i % 2 == 0 else r_in
    angle = math.pi * i / teeth
    points.append((round(radius * math.cos(angle)),
                   round(radius * math.sin(angle))))

gear = s.polygon(1, points)
print(len(gear.points), gear.bbox())
```

Output

```
24 (-1000, -1000, 1000, 1000)
```

Text is drawn in a stroke font. `rotation` is in degrees, anticlockwise.

```
from hlayout.scripting.api import Session

s = Session.new("DEMO")
s.new_cell("LABELS")
for i, angle in enumerate((0, 90, 180, 270)):
    s.text(4, (0, 0), f"  {angle} DEG", height=150, rotation=angle)
print([(t.text.strip(), t.rotation) for t in s.cell.shapes])
```

Output

```
[('0 DEG', 0), ('90 DEG', 90), ('180 DEG', 180), ('270 DEG', 270)]
```

19.6 Querying and editing geometry

```
s.shapes_on_layer(layer)                        -> [shape, ...]   (current cell)
s.remove(shape)
s.delete_last()                                 -> "shape" | "sref" | "aref" | None
s.clear_layer(layer, cell_names=None)           -> {cell: shapes_deleted}
s.remap_layers({old: new, ...}, cell_names=None) -> {old_layer: shapes_moved}
s.convert_rects_to_outline(layer, width, cell_names=None) -> {cell: converted}
```

```
from hlayout.scripting.api import Session

s = Session.new("DEMO")
s.new_cell("TOP")
big   = s.rect(1, 0, 0, 5000, 5000)
small = s.rect(1, 6000, 0, 6100, 100)
s.rect(2, 0, 6000, 1000, 7000)

print(len(s.shapes_on_layer(1)))
```

```
# delete everything on layer 1 smaller than 0.5 mm across
for shape in list(s.shapes_on_layer(1)):
    x1, y1, x2, y2 = shape.bbox()
    if min(x2 - x1, y2 - y1) < 500:
        s.remove(shape)
print(len(s.shapes_on_layer(1)))

s.circle(3, (0, 0), 100)
print(s.delete_last())              # removes the circle just added
```

Output

```
2
1
shape
```

`remap_layers()` scans every cell in the library unless `cell_names` restricts it; `clear_layer()` and `convert_rects_to_outline()` act on the current cell's own shapes unless `cell_names` says otherwise. None of the three descends into placed cells.

```
from hlayout.scripting.api import Session

s = Session.new("DEMO")
s.new_cell("A"); s.rect(3, 0, 0, 100, 100); s.rect(3, 200, 0, 300, 100)
s.new_cell("B"); s.rect(3, 0, 0, 100, 100); s.rect(5, 0, 0, 50, 50)

print(s.remap_layers({3: 7}))                    # all cells
print(s.remap_layers({5: 6}, cell_names=["B"]))  # one cell
print(s.clear_layer(7, cell_names=["A"]))
print(s.clear_layer(7))                          # current cell (B)
```

Output

```
{3: 3}
{5: 1}
{'A': 2}
{'B': 1}
```

`convert_rects_to_outline()` turns a filled rectangle or polygon into a closed stroked path through the same corners, which is how a substrate or package body is normally drawn.

```
from hlayout.scripting.api import Session

s = Session.new("DEMO")
s.new_cell("SUBSTRATE")
s.rect(10, 0, 0, 25400, 12700)                # 1.0 x 0.5 inch
print(s.convert_rects_to_outline(10, width=100))
outline = s.cell.shapes[0]
print(type(outline).__name__, outline.width, len(outline.points))
```

Output

```
{'SUBSTRATE': 1}
Path 100 5
```

19.7 Cell instances and arrays

```
s.sref(cell_name, origin, rotation=0.0, magnification=1.0,
       x_reflection=False, label="", pin="")                    -> SRef
s.aref(cell_name, origin, columns=1, rows=1, col_spacing=0.0, row_spacing=0.0,
       rotation=0.0, magnification=1.0, x_reflection=False)     -> ARef
s.smash_aref(aref)                                              -> [SRef, ...]
```

`sref()` places one instance of a cell that already exists in the design; `aref()` places a rectangular array of it. `label` is the instance's reference designator (the Label field of Edit Selected...), which netlist extraction reads. A cell cannot be placed inside itself, directly or through another cell.

```
from hlayout.scripting.api import Session
from hlayout.core.errors import LayoutError

s = Session.new("DEMO")
s.new_cell("C0805")
s.rect(1, -1000, -625, -400, 625, pin="1")
s.rect(1,   400, -625, 1000, 625, pin="2")

s.new_cell("BOARD")
s.sref("C0805", (0, 0), label="C1")
s.sref("C0805", (4000, 0), rotation=90, label="C2")
s.sref("C0805", (8000, 0), x_reflection=True, label="C3")

for ref in s.cell.srefs:
    print(ref.label, ref.cell_name, ref.origin, ref.rotation, ref.x_reflection)
print(s.bbox())

s.use_cell("C0805")
try:
    s.sref("BOARD", (0, 0))           # BOARD already places C0805
except LayoutError as exc:
    print("refused:", exc)
```

Output

```
C1 C0805 (0, 0) 0.0 False
C2 C0805 (4000, 0) 90 False
C3 C0805 (8000, 0) 0.0 True
(-1000.0, -1000.0, 9000.0, 1000.0)
refused: cannot place 'BOARD' inside 'C0805': this would create a cyclic cell reference (a
cell can't contain an instance of itself, directly or indirectly)
```

```
from hlayout.scripting.api import Session

s = Session.new("DEMO")
s.new_cell("VIA"); s.circle(1, (0, 0), 100)

s.new_cell("TOP")
array = s.aref("VIA", (0, 0), columns=5, rows=2, col_spacing=500, row_spacing=800)
print(array.count, array.instance_origins()[:3], s.bbox())

refs = s.smash_aref(array)            # back to individual instances
print(len(refs), len(s.cell.arefs), len(s.cell.srefs))
s.remove(refs[0])                     # which can then be edited one by one
print(len(s.cell.srefs))
```

Output

```
10 [(0, 0), (500, 0), (1000, 0)] (-100.0, -100.0, 2100.0, 900.0)
10 0 10
9
```

19.8 Undo, redo and batching

```
s.undo() -> bool        s.redo() -> bool
s.can_undo() -> bool    s.can_redo() -> bool
s.batch()               context manager: several edits, one undo step
s.checkpoint()          push an undo point before editing s.library directly
```

```
s.clear_undo_history()
s.on_change(callback)   call callback() after every edit
```

```
from hlayout.scripting.api import Session

s = Session.new("DEMO")
s.new_cell("TOP")
s.rect(1, 0, 0, 100, 100)
s.rect(1, 200, 0, 300, 100)
print(len(s.cell.shapes), s.can_undo(), s.can_redo())

s.undo()
print(len(s.cell.shapes), s.can_redo())
s.redo()
print(len(s.cell.shapes))

with s.batch():                         # ten rectangles, one undo step
    for i in range(10):
        s.rect(2, i * 200, 500, i * 200 + 100, 600)
print(len(s.cell.shapes))
s.undo()
print(len(s.cell.shapes))
```

Output

```
2 True False
1 True
2
12
2
```

on_change() is how the GUI knows to repaint. A script can use it to count or log edits.

```
from hlayout.scripting.api import Session

s = Session.new("DEMO")
edits = []
s.on_change(lambda: edits.append(s.current_cell_name))

s.new_cell("TOP")
s.rect(1, 0, 0, 100, 100)
s.circle(1, (500, 500), 50)
print(len(edits), edits)
```

Output

```
3 ['TOP', 'TOP', 'TOP']
```

The session's own methods record their own undo steps. A script that edits `s.library` or a `Cell` directly bypasses that, so call `checkpoint()` first to make the edit undoable. `clear_undo_history()` empties the history, for instance after a long generator script whose steps are not worth keeping.

```
from hlayout.scripting.api import Session

s = Session.new("DEMO")
s.new_cell("TOP")
shape = s.rect(1, 0, 0, 100, 100)

s.checkpoint()                          # then edit the data model directly
s.cell.shapes[0].pin = "1"
print(s.cell.shapes[0].pin, s.can_undo())
s.undo()
print(repr(s.cell.shapes[0].pin))
```

```
s.clear_undo_history()
print(s.can_undo(), s.can_redo())
```

Output

```
1 True
''
False False
```

19.9 Parts libraries

```
s.attach_library(path)                -> Library      (at most 2 attached)
s.detach_library(index)
s.list_attached_libraries()           -> [{'index','name','path','cells'}, ...]
s.place_cell(cell_name, origin, source=None, rotation=0.0, magnification=1.0,
             x_reflection=False, label="", pin="")              -> SRef
s.place_cell_array(cell_name, origin, columns=1, rows=1, col_spacing=0.0,
                   row_spacing=0.0, source=None, ...)           -> ARef
s.create_via_cell(layer1, layer2, shape="square", size=100, index=None,
                  overwrite=False, name=None,
                  dielectric_layer=None, dielectric_window=None)
                                                       -> (cell_name, path)
s.create_pad_cell(name, conductor_layer, paste_layer, length, width, margin,
                  rows=1, cols=1, pitch_x=None, pitch_y=None,
                  index=None, overwrite=False)       -> (cell_name, path, pad_count)
s.create_chip_cell(name, length, width, graphics_layer, epoxy_length,
                   epoxy_width, epoxy_layer, outline_width=None, overwrite=False)
s.export_cell_to_library(cell_name, index=0, overwrite=False,
                         reuse_shared=True)          -> (name_in_library, path)
s.merge_gds(path_a, path_b, out_path, library_name=None) -> {'cells', 'renamed'}
```

An attached library sits beside the design and is not merged into it. `place_cell()` finds a cell by exact name, copies it (and anything it references) into the design if it lives in an attached library, and places an instance. `source` is `"current"`, an attached library's index, or `None` to search the design first and then each attached library.

```
from hlayout.scripting.api import Session

# 1. publish a small parts library
lib = Session.new("SCRATCH")
for name, half_len in (("R0603", 800), ("R0805", 1000), ("R1206", 1600)):
    lib.new_cell(name)
    lib.rect(1, -half_len, -400, -half_len + 500, 400, pin="1")
    lib.rect(1, half_len - 500, -400, half_len, 400, pin="2")
lib.save("resistors.gds", library_name="RESISTORS")

# 2. use it from a design
s = Session.new("BOARD")
s.new_cell("TOP")
s.attach_library("resistors.gds")

for entry in s.list_attached_libraries():
    print(entry["index"], entry["name"], entry["cells"])
print(s.find_cells("080"))
print([c["name"] for c in s.list_library(0)])

s.place_cell("R0805", (0, 0), label="R1")
s.place_cell("R1206", (5000, 0), rotation=90, label="R2")
s.place_cell_array("R0603", (0, 3000), columns=4, col_spacing=2500)

print(sorted(s.library.cells))         # the placed parts were copied in
```

```
print([(r.label, r.cell_name) for r in s.cell.srefs], len(s.cell.arefs))
s.detach_library(0)
print(s.list_attached_libraries())
```

Output

```
0 RESISTORS 3
[{'source': 0, 'name': 'R0805'}]
['R0603', 'R0805', 'R1206']
['R0603', 'R0805', 'R1206', 'TOP']
[('R1', 'R0805'), ('R2', 'R1206')] 1
[]
```

`create_via_cell()` and `create_pad_cell()` are the Via Wizard and Pad Wizard. They write the new cell into an attached library and save that library's file in place, so the part is available to every design that attaches it. A two-pad footprint is numbered pin 1 and pin 2 automatically.

```
import os
from hlayout.scripting.api import Session

s = Session.new("BOARD")
s.new_cell("TOP")
s.attach_library("resistors.gds")

name, path = s.create_via_cell(1, 3, shape="round", size=300,
                               dielectric_layer=2, dielectric_window=200)
print(name, os.path.basename(path))

name, path, count = s.create_pad_cell("CONN_2PIN", conductor_layer=1, paste_layer=5,
                                      length=1500, width=1000, margin=100,
                                      cols=2, pitch_x=2540)
print(name, count)

s.place_cell(name, (0, 0), label="J1")
s.place_cell("VIA_Layer1_to_Layer3", (5000, 0), pin="7")
print(s.list_pins("CONN_2PIN"))
print([c["name"] for c in s.list_library(0)])
```

Output

```
VIA_Layer1_to_Layer3 resistors.gds
CONN_2PIN 2
[{'index': 0, 'layer': 1, 'pin': '1', 'bbox': (-2020, -500, -520, 500)}, {'index': 1, 'layer':
5, 'pin': '', 'bbox': (-1920, -400, -620, 400)}, {'index': 2, 'layer': 1, 'pin': '2', 'bbox':
(520, -500, 2020, 500)}, {'index': 3, 'layer': 5, 'pin': '', 'bbox': (620, -400, 1920, 400)}]
['R0603', 'R0805', 'R1206', 'VIA_Layer1_to_Layer3', 'CONN_2PIN']
```

`export_cell_to_library()` goes the other way: it copies a cell you built in the design, with everything it places, into an attached library and saves it. `create_chip_cell()` starts a bare-die footprint (a body outline and an epoxy die-attach pad) ready for its bond pads.

```
from hlayout.scripting.api import Session

s = Session.new("BOARD")
s.attach_library("resistors.gds")

s.create_chip_cell("OPAMP_DIE", length=1500, width=1200, graphics_layer=50,
                   epoxy_length=1900, epoxy_width=1600, epoxy_layer=51)
pad_centres = [(-600, -450), (0, -450), (600, -450), (0, 450)]
for n, (x, y) in enumerate(pad_centres, start=1):
    s.rect(52, x - 50, y - 50, x + 50, y + 50, pin=str(n))     # bond pads

print(s.current_cell_name, s.bbox())
```

```
name, path = s.export_cell_to_library("OPAMP_DIE")
print(name, [c["name"] for c in s.list_library(0)])
```

Output

```
OPAMP_DIE (-950, -800, 950, 800)
OPAMP_DIE ['R0603', 'R0805', 'R1206', 'VIA_Layer1_to_Layer3', 'CONN_2PIN', 'OPAMP_DIE']
```

`merge_gds()` works on files only and does not touch the open design. A cell name present in both inputs is renamed in the output.

```
from hlayout.scripting.api import Session

a = Session.new("A"); a.new_cell("PAD"); a.rect(1, 0, 0, 100, 100); a.save("a.gds")
b = Session.new("B"); b.new_cell("PAD"); b.rect(1, 0, 0, 200, 200)
b.new_cell("VIA"); b.circle(1, (0, 0), 50); b.save("b.gds")

result = Session.new().merge_gds("a.gds", "b.gds", "merged.gds",
                                 library_name="MERGED")
print(result)
print(sorted(Session.open("merged.gds").library.cells))
```

Output

```
{'cells': 3, 'renamed': {'PAD': 'PAD_2'}}
['PAD', 'PAD_2', 'VIA']
```

19.10 Pin numbering

```
s.list_pins(cell_name=None)             -> [{'index','layer','pin','bbox'}, ...]
s.set_pin(index, pin, cell_name=None)
```

Only rectangles and polygons carry a pin. `index` is the shape's position in the cell's shape list, as `list_pins()` reports it. This numbers a dual-in-line footprint anticlockwise from the bottom-left pad.

```
from hlayout.scripting.api import Session

s = Session.new("DEMO")
s.new_cell("DIP8")
for i in range(4):                              # drawn in no useful order
    s.rect(1, i * 2540 - 400, 7220, i * 2540 + 400, 8020)     # top row
    s.rect(1, i * 2540 - 400, -400, i * 2540 + 400, 400)      # bottom row

pads = s.list_pins()
def centre(p):
    x1, y1, x2, y2 = p["bbox"]
    return ((x1 + x2) / 2, (y1 + y2) / 2)

bottom = sorted((p for p in pads if centre(p)[1] < 3000),
                key=lambda p: centre(p)[0])
top = sorted((p for p in pads if centre(p)[1] > 3000),
             key=lambda p: -centre(p)[0])

for number, pad in enumerate(bottom + top, start=1):
    s.set_pin(pad["index"], str(number))

for p in s.list_pins():
```

```
    print(p["pin"], centre(p))
s.set_pin(0, "")                                # an empty string clears a pin
```

Output

```
8 (0.0, 7620.0)
1 (0.0, 0.0)
7 (2540.0, 7620.0)
2 (2540.0, 0.0)
6 (5080.0, 7620.0)
3 (5080.0, 0.0)
5 (7620.0, 7620.0)
4 (7620.0, 0.0)
```

19.11 Vias and automatic routing

```
s.drop_via(x, y, layer_a, layer_b, size=100, shape="square",
           dielectric_layer=None, dielectric_window=None)
s.auto_route(a, b, layer=None, grid=100, width=20, clearance=None,
             margin=None, boundary_layer=None, cell_name=None)   -> Path
```

`drop_via()` draws a pad on both conductor layers straight into the current cell, with an optional window on the dielectric between them.

```
from hlayout.scripting.api import Session

s = Session.new("DEMO")
s.new_cell("TOP")
s.path(1, [(0, 0), (3000, 0)], width=200)             # lower conductor
s.path(3, [(3000, 0), (3000, 3000)], width=200)       # upper conductor
s.drop_via(3000, 0, 1, 3, size=400, shape="round",
           dielectric_layer=2, dielectric_window=300)

for layer in (1, 2, 3):
    print(layer, [type(x).__name__ for x in s.shapes_on_layer(layer)])
```

Output

```
1 ['Path', 'Polygon']
2 ['Polygon']
3 ['Path', 'Polygon']
```

`auto_route()` finds an orthogonal path between two points on one layer that keeps clear of everything else on that layer. Each end is a point or a pad shape; given a shape, the route ends at its centre and may land on it. It raises `LayoutError` when no clear route exists at the given grid and width.

```
from hlayout.scripting.api import Session

s = Session.new("DEMO")
s.new_cell("TOP")
pad_a = s.rect(1, -200, -200, 200, 200, pin="1")
pad_b = s.rect(1, 5800, -200, 6200, 200, pin="2")
s.rect(1, 2700, -600, 3300, 600)                      # something in the way

trace = s.auto_route(pad_a, pad_b, layer=1, grid=100, width=150, clearance=150)
print(type(trace).__name__, trace.width, len(trace.points))
print(trace.points)
```

Output

```
Path 150 4
[(0, 0), (0, -900), (6000, -900), (6000, 0)]
```

19.12 Wire bonds

```
s.wire_bond(points, layer=None, width=None, pad=None, min_length=None,
            max_length=None, cell_name=None)
s.find_wire_bond(point, layer=None, cell_name=None, tol=None)
                                              -> (path, [ball, ball]) | None
s.move_wire_bond(near, to, layer=None, cell_name=None, tol=None)
                                              -> (fixed_end, new_end)
s.delete_wire_bond(point, layer=None, cell_name=None, tol=None)  -> endpoints
```

A bond is drawn on the bond layer as a ball at each end joined by a round-capped path. The layer, wire width, ball diameter and length limits default to Setup → Technology Properties..., Wire Bonds tab, so save those once (see 19.4) or pass them on each call. A bond outside the length limits, or one that would cross an existing bond, is refused.

```
from hlayout.scripting.api import Session
from hlayout.core.errors import LayoutError
from hlayout.io import techfile

s = Session.new("DEMO")
s.new_cell("TOP")
techfile.save_bond_rules(s.library, {"layer": 20, "width": 25, "pad": 75,
                                     "min_length": 300, "max_length": 4000})

die_pads   = [(0, 0), (0, 300), (0, 600)]
board_pads = [(1500, -300), (1500, 300), (1500, 900)]
for die, board in zip(die_pads, board_pads):
    s.wire_bond([die, board])
print(len(s.shapes_on_layer(20)))                # 3 bonds x (path + 2 balls)

for bad in ([(0, 0), (100, 0)],                  # too short
            [(0, 0), (1500, 900)]):              # crosses the others
    try:
        s.wire_bond(bad)
    except LayoutError as exc:
        print("refused:", exc)

path, balls = s.find_wire_bond((750, -150))       # any point along the wire
print(path.points, len(balls))
print(s.move_wire_bond(near=(1500, 900), to=(1500, 1200)))
print(s.delete_wire_bond((0, 0)))
print(len(s.shapes_on_layer(20)))
```

Output

```
9
refused: wire bond span 100 DBU is shorter than the minimum 300 (Setup > Wire Bond Rules...)
refused: that wire bond would cross an existing bond wire -- bond wires share one plane and
can't cross
[(0, 0), (1500, -300)] 2
((0, 600), (1500, 1200))
((0, 0), (1500, -300))
6
```

19.13 Resistors

```
s.generate_resistor(x, y, body_width, body_length, rotation=0.0, ink_rsh=None,
                    ref_des=None, overlap_a=None, pad_b=None, space_c=None,
                    glaze_oversize=None, resistor_layer=None, conductor_layer=None,
                    glaze_layer=None, keepout_layer=None, keepout_oversize=None,
                    cell_name=None, n_folds=1)                    -> name
s.generate_resistor_table(cell_name, layer, origin=(0, 0), ...)   -> name
```

`generate_resistor()` builds a thick-film resistor (ink body, glaze overprint and two end pads, numbered pin 1 and pin 2) as its own cell and places one instance at `(x, y)`. Dimensions and layers left out come from Setup → Resistor Geometry Rules... The resistance is sheet resistance × `body_length` / `body_width`.

```
from hlayout.scripting.api import Session

s = Session.new("DEMO")
s.new_cell("TOP")

# 1 kohm/sq ink on layer 30: 10 k = 10 squares, 2.2 k = 2.2 squares
jobs = [("R1", 10_000, 500, (0, 0), 0), ("R2", 2_200, 800, (0, 4000), 90)]
for ref, ohms, width, (x, y), angle in jobs:
    length = round(ohms / 1000 * width)
    name = s.generate_resistor(x, y, body_width=width, body_length=length,
                               rotation=angle, ref_des=ref, resistor_layer=30)
    print(name, length, s.bbox(name))

print([(r.label, r.cell_name, r.rotation) for r in s.cell.srefs])
s.use_cell("R1")
print(sorted({shape.layer for shape in s.cell.shapes}), s.list_pins())
```

Output

```
R1 5000 (-3200, -450, 3200, 450)
R2 1760 (-1580, -600, 1580, 600)
[('R1', 'R1', 0), ('R2', 'R2', 90)]
[30, 31, 32] [{'index': 0, 'layer': 30, 'pin': '', 'bbox': (-2700, -250, 2700, 250)},
{'index': 1, 'layer': 32, 'pin': '', 'bbox': (-2750, -300, 2750, 300)}, {'index': 2, 'layer':
31, 'pin': '1', 'bbox': (-3200, -450, -2500, 450)}, {'index': 3, 'layer': 31, 'pin': '2',
'bbox': (2500, -450, 3200, 450)}]
```

`generate_resistor_table()` draws the resistor BOM table from the rows saved by the Thick-Film Resistor Designer. A script can save those rows itself with `techfile.save_resistor_list()`. The table and sheet generators (this one, `generate_pin_function_table()` and `generate_drawing_sheet()`) and `create_chip_cell()` leave the cell they built as the current cell, so call `use_cell()` before placing it.

```
from hlayout.scripting.api import Session
from hlayout.io import techfile

s = Session.new("DEMO")
s.new_cell("TOP")
techfile.save_resistor_list(s.library, [
    {"ref_des": "R1", "value": 10000, "tolerance": 1, "power_mw": 50,
     "body_width": 500, "selected_ink": 1000},
    {"ref_des": "R2", "value": 2200, "tolerance": 5, "power_mw": 100,
     "body_width": 800, "selected_ink": 1000},
])

table = s.generate_resistor_table("RESTABLE_1", layer=62)
print(table, s.current_cell_name, s.bbox(table))

s.use_cell("TOP")                       # the generator left the table cell current
s.sref(table, (20000, 0))
print(sorted({x.text for x in s.library.cells[table].shapes if hasattr(x, "text")}))
```

Output

```
RESTABLE_1 RESTABLE_1 (-10.0, -580.0, 8710.0, 440.0)
['1', '1,000', '1,760', '10,000', '2,200', '5', '7,600', 'AIM VALUE', 'INK (OHMS/SQ)', 'R1',
'R2', 'REF DES', 'RESISTOR TABLE', 'TOL%', 'VALUE (OHMS)']
```

The Resistor Trim Simulator's engine is available without the GUI as `hlayout.core.trim_sim.TrimSimulator`. This plunges a laser cut into a 1 mm × 0.5 mm resistor of 1 kΩ/sq ink and then turns it into an L-cut.

```
from hlayout.core.trim_sim import TrimSimulator

sim = TrimSimulator(length_um=1000, width_um=500, sheet_ohms_per_sq=1000)
plunge = sim.add_plunge(entry_edge="bottom", kerf_um=40, depth_um=300, steps=6)
leg = sim.add_l_cut(turn_length_um=200, steps=4)

for step in plunge + leg:
    print(f"{step.resistance_ohm:8.1f} ohm")
print(sim.is_open(), sim.cut_path_points())
```

Output

```
  2013.4 ohm
  2083.9 ohm
  2212.3 ohm
  2362.4 ohm
  2595.7 ohm
  2896.3 ohm
  3062.9 ohm
  3239.1 ohm
  3406.1 ohm
  3574.9 ohm
False [(500.0, 0.0), (500.0, 300.0), (700.0, 300.0)]
```

19.14 Mask operations and dielectric fill

```
s.mask_op(op, layer_a, result_layer, layer_b=None, amount=0.0, cell_names=None,
          datatype=0, flatten=True, simplify=None, join_style="mitre")
                                                          -> {cell: polygons}
s.run_mask_op_batch(steps=None, cell_names=None)       -> [(step, written), ...]
s.generate_dielectric(layer=None, keepout_layers=None, clearance=None,
                      margin=None, cell_name=None)     -> polygon count
s.clear_dielectric(layer=None, cell_name=None, all_datatypes=False) -> removed
s.mask_op_top_view(result_layer, layers=None, exclude_layers=None,
                   hatch_conductor_pitch=None, line_width=100.0, ...)
                                                          -> {cell: paths}
```

op is `"OVERSIZE"` or `"UNDERSIZE"` (one layer and an `amount` in DBU), or `"AND"`, `"OR"`, `"NOT"` or `"XOR"` (two layers). The result is written to `result_layer` as new polygons. Placed cells are included unless `flatten=False`.

```
from hlayout.scripting.api import Session

s = Session.new("DEMO")
s.new_cell("TOP")
s.rect(1, 0, 0, 1000, 1000)
s.rect(1, 800, 800, 1800, 1800)          # overlaps the first
s.rect(2, 500, 500, 1300, 1300)

def area(layer):
    total = 0
    for shape in s.shapes_on_layer(layer):
        pts = shape.points
        total += abs(sum(x1 * y2 - x2 * y1 for (x1, y1), (x2, y2)
                         in zip(pts, pts[1:] + pts[:1]))) / 2
    return total

for op, result in (("OR", 10), ("AND", 11), ("NOT", 12), ("XOR", 13)):
```

```
    print(op, s.mask_op(op, 1, result, layer_b=2), area(result))

print(s.mask_op("OVERSIZE", 2, 14, amount=100), s.shapes_on_layer(14)[0].bbox())
print(s.mask_op("UNDERSIZE", 2, 15, amount=100), s.shapes_on_layer(15)[0].bbox())
```

Output

```
OR {'TOP': 1} 2140000.0
AND {'TOP': 1} 460000.0
NOT {'TOP': 2} 1500000.0
XOR {'TOP': 4} 1680000.0
{'TOP': 1} (400, 400, 1400, 1400)
{'TOP': 1} (600, 600, 1200, 1200)
```

A common use is deriving one mask from another. Here a solder-paste layer is the pad layer shrunk by 0.1 mm, generated for a placed part.

```
from hlayout.scripting.api import Session

s = Session.new("DEMO")
s.new_cell("C0805")
s.rect(1, -1000, -625, -400, 625)
s.rect(1,   400, -625, 1000, 625)
s.new_cell("TOP")
s.sref("C0805", (0, 0)); s.sref("C0805", (4000, 0))

print(s.mask_op("UNDERSIZE", 1, 5, amount=100))          # pads -> paste
print([x.bbox() for x in s.shapes_on_layer(5)][:2])
# flatten=False looks at TOP's own shapes only, and TOP has none on layer 1
print(s.mask_op("UNDERSIZE", 1, 6, amount=100, flatten=False))
```

Output

```
{'TOP': 4}
[(4500, -525, 4900, 525), (3100, -525, 3500, 525)]
{}
```

`run_mask_op_batch()` runs an ordered recipe as one undo step. A later step may read a layer that an earlier step wrote. With no argument it runs the recipe saved by Tools → Mask Operation Batch...

```
from hlayout.scripting.api import Session

s = Session.new("DEMO")
s.new_cell("TOP")
s.rect(1, 0, 0, 2000, 2000)
s.rect(2, 500, 500, 1500, 1500)

recipe = [
    {"kind": "OVERSIZE", "layer_a": 2, "result_layer": 20, "amount": 200},
    {"kind": "NOT", "layer_a": 1, "layer_b": 20, "result_layer": 21},
]
for step, written in s.run_mask_op_batch(recipe):
    print(step.kind, written)
print(len(s.shapes_on_layer(21)))
s.undo()                                  # the whole recipe
print(len(s.shapes_on_layer(20)), len(s.shapes_on_layer(21)))
```

Output

```
OVERSIZE {'TOP': 1}
NOT {'TOP': 1}
1
0 0
```

`generate_dielectric()` fills the dielectric layer. Draw the fill boundary and any openings on that layer (datatype 0); the largest shape that encloses the others is the boundary and the rest become holes, each grown by `clearance`. The fill is written on the same layer at datatype 1, so the call can be repeated after the layout changes.

```
from hlayout.scripting.api import Session

s = Session.new("DEMO")
s.new_cell("TOP")
s.rect(2, 0, 0, 10000, 6000)                          # fill boundary
s.drop_via(2000, 3000, 1, 3, size=400,
           dielectric_layer=2, dielectric_window=300)    # a via window
s.rect(2, 6000, 2000, 8000, 4000)                     # a hand-drawn opening

print(s.generate_dielectric(layer=2, clearance=150))
fill = [x for x in s.shapes_on_layer(2) if x.datatype == s.DIELECTRIC_FILL_DATATYPE]
print(len(fill), len(s.shapes_on_layer(2)))
print(s.clear_dielectric(layer=2), len(s.shapes_on_layer(2)))
```

Output

```
4
4 7
4 3
```

`mask_op_top_view()` draws the hidden-line assembly view of section 10.5 as paths on one layer. It appends to that layer, so clear it before each run.

```
from hlayout.scripting.api import Session

s = Session.new("DEMO")
s.new_cell("TOP")
s.rect(10, 0, 0, 10000, 6000)                         # substrate
s.path(31, [(1000, 3000), (9000, 3000)], width=400)  # conductor
s.generate_resistor(5000, 3000, body_width=600, body_length=1200,
                    ref_des="R1", resistor_layer=30)

s.clear_layer(63)
print(s.mask_op_top_view(63, line_width=50))
print(s.mask_op_top_view(63, layers=[10, 31], line_width=50))   # explicit subset
```

Output

```
{'TOP': 36}
{'TOP': 21}
```

19.15 Design rule check

```
s.run_drc(rules=None, cell_names=None, error_layer=None)   -> [DrcViolation, ...]
```

A rule is a dict (or a `hlayout.core.drc.DrcRule`) with `kind`, `layer_a`, `value` and, for the two-layer kinds, `layer_b`. The kinds are `MIN_WIDTH`, `MIN_AREA` and `MIN_SPACE` (one layer) and `MIN_SPACE_BETWEEN` and `MIN_ENCLOSURE` (two layers). With `rules=None` the rule set saved in the technology file is used. Each violation has `.rule`, `.cell_name`, `.message`, `.points` and `.bbox()`.

```
from hlayout.scripting.api import Session

s = Session.new("DEMO")
s.new_cell("TOP")
s.rect(1, 0, 0, 2000, 80)                 # an 80 um trace
s.rect(1, 0, 150, 2000, 400)              # 70 um from the first
s.rect(1, 3000, 0, 4000, 1000)            # a pad ...
```

```
s.rect(2, 3050, 50, 3950, 950)            # ... whose window is only 50 um inside it

rules = [
    {"kind": "MIN_WIDTH", "layer_a": 1, "value": 100, "name": "C.W.1"},
    {"kind": "MIN_SPACE", "layer_a": 1, "value": 100, "name": "C.S.1"},
    {"kind": "MIN_ENCLOSURE", "layer_a": 1, "layer_b": 2, "value": 100,
     "name": "C.E.1"},
]
violations = s.run_drc(rules, error_layer=60)
for v in violations:
    print(v.rule.name, v.cell_name, v.bbox())
    print("   ", v.message)
print(len(s.shapes_on_layer(60)))         # markers drawn on layer 60
```

Output

```
C.W.1 TOP (0, 0, 2000, 80)
    C.W.1: MIN_WIDTH(layer 1) >= 100
C.S.1 TOP (1950, 30, 2050, 200)
    C.S.1: MIN_SPACE(layer 1) >= 100
C.E.1 TOP (2950, -50, 4050, 1050)
    C.E.1: MIN_ENCLOSURE(layer 1, layer 2) >= 100
3
```

To make a rule set the default for a design, save it into the technology parameters. `run_drc()` with no arguments then uses it, as does `DRC RUN` in the command bar.

```
from hlayout.scripting.api import Session
from hlayout.io import techfile

s = Session.new("DEMO")
s.new_cell("TOP")
s.rect(1, 0, 0, 2000, 80)
techfile.save_drc_rules(s.library, [
    {"kind": "MIN_WIDTH", "layer_a": 1, "value": 100, "name": "C.W.1"},
])
print(len(s.run_drc()))

s.clear_layer(1)
s.rect(1, 0, 0, 2000, 120)                # fixed
print(len(s.run_drc()))
```

Output

```
1
0
```

19.16 Netlist extraction

```
s.extract_netlist(conductor_layers=None, cell_name=None, include_bonds=True) -> [Net, ..]
s.export_netlist(nets, path, fmt=None, cell_name=None) -> 'csv'|'report'|'kicad'|'tango'
s.export_netlist_csv(nets, path)
techfile.load_netlist_settings(library) / save_netlist_settings(library, settings)
```

Copper that touches on the listed layers is one net. A net's members are named from each pad's pin number and the reference designator (label) of the instance that places it. Each Net has .id, .pins (a list with .ref_des, .pin and .xy -- the pin's position in DBU), .unresolved_count, .bonds, .open_bond_ends (bond ends that landed on nothing) and .location (a point on the net). With conductor_layers=None the design's saved selection is used -- the Conductor ticks in Tools → Extract Netlist..., stored by techfile.save_netlist_settings().

```
from hlayout.scripting.api import Session
```

```
s = Session.new("DEMO")
s.new_cell("R0805")
s.rect(1, -1000, -400, -500, 400, pin="1")
s.rect(1,   500, -400, 1000, 400, pin="2")

s.new_cell("TOP")
s.sref("R0805", (0, 0), label="R1")
s.sref("R0805", (5000, 0), label="R2")
s.sref("R0805", (10000, 0), label="R3")
s.path(1, [(750, 0), (4250, 0)], width=200)        # R1.2 to R2.1
s.path(1, [(5750, 0), (9250, 0)], width=200)       # R2.2 to R3.1

nets = s.extract_netlist([1])
for net in nets:
    print(net.id, [f"{p.ref_des}.{p.pin}" for p in net.pins])

s.export_netlist_csv(nets, "netlist.csv")
print(open("netlist.csv").read())
```

Output

```
NET1 ['R1.1']
NET2 ['R1.2', 'R2.1']
NET3 ['R2.2', 'R3.1']
NET4 ['R3.2']
net,ref_des,pin
NET1,R1,1
NET2,R1,2
NET2,R2,1
NET3,R2,2
NET3,R3,1
NET4,R3,2
```

Wire bonds join nets. When Setup → Technology Properties..., Wire Bonds tab names a bond layer, bonds are included automatically: each bond joins whatever its two endpoints land on, not the copper it flies over. Pass include_bonds=False to see the copper alone.

```
from hlayout.scripting.api import Session
from hlayout.io import techfile

s = Session.new("DEMO")
techfile.save_bond_rules(s.library, {"layer": 20, "width": 25, "pad": 75,
                                     "min_length": 300, "max_length": 4000})
s.new_cell("DIE");  s.rect(1, -50, -50, 50, 50, pin="3")
s.new_cell("LAND"); s.rect(1, -200, -200, 200, 200, pin="1")
s.new_cell("TOP")
s.sref("DIE", (0, 0), label="U1")
s.sref("LAND", (1500, 0), label="J1")
s.wire_bond([(0, 0), (1500, 0)])

show = lambda nets: [[f"{p.ref_des}.{p.pin}" for p in n.pins] for n in nets]
print(show(s.extract_netlist([1])))
print(show(s.extract_netlist([1], include_bonds=False)))
```

Output

```
[['U1.3', 'J1.1']]
[['U1.3'], ['J1.1']]
```

Chip-and-wire: list the die-pad layer (and a header-pin layer) in the bond rules' die_pad_layers. Each shape on those layers is kept apart from the copper underneath -- here the die-attach pad under U1 -- and joins a net only where a bond lands on it. A bonded pad with no pin number is reported as RefDes.@x,y, and a bond end that misses its pad appears in open_bond_ends.

```
from hlayout.scripting.api import Session
from hlayout.io import techfile

s = Session.new("DEMO")
techfile.save_bond_rules(s.library, {"layer": 59, "width": 25, "pad": 75,
                                     "min_length": 300, "max_length": 4000,
                                     "die_pad_layers": [58]})
s.new_cell("DIE")
s.rect(58, -400, -50, -300, 50, pin="1")
s.rect(58,  300, -50,  400, 50)              # pad left unnumbered
s.new_cell("LAND")
s.rect(3, -200, -200, 200, 200, pin="1")
s.new_cell("TOP")
s.rect(3, -800, -800, 800, 800)              # die-attach pad under the die
s.sref("DIE", (0, 0), label="U1")
s.sref("LAND", (-2800, 0), label="J1")
s.sref("LAND", (2800, 0), label="J2")
s.wire_bond([(-350, 0), (-2800, 0)])         # U1.1 to J1
s.wire_bond([(350, 0), (2800, 400)])         # misses J2's pad

techfile.save_netlist_settings(s.library, {"conductor_layers": [3]})
for n in s.extract_netlist():                # no layers: the saved selection
    print(n.id, [(p.label(), p.xy) for p in n.pins], n.bonds,
          n.open_bond_ends, n.location)
```

Output

```
NET1 [('J1.1', (-2800, 0)), ('U1.1', (-350, 0))] 1 [] (-2800, 0)
NET2 [] 0 [] (0, 0)
NET3 [('J2.1', (2800, 0))] 0 [] (2800, 0)
NET4 [('U1.@350,0', (350, 0))] 1 [(2800, 400)] (350, 0)
```

19.17 Drawing sheets, tables and printing

```
s.generate_drawing_sheet(cell_name, layer, size="A3", orientation="portrait",
                         origin=(0, 0), margin_mm=10.0, title_block_height_mm=45.0,
                         text_height_mm=3.0, line_width_mm=0.35, title="", dwg_no="",
                         issue="", drawn_by="", drawn_date="", checked_by="",
                         checked_date="", approved_by="", approved_date="",
                         overwrite=True)                                -> cell name
s.generate_pin_function_table(cell_name, layer, pin_count, pins_per_group=10,
                              functions=None, origin=(0, 0), ...)     -> cell name
s.print_cell(cell_name=None, out_path=None, scale=None,
             pin_label_layers=None)                                     -> html path
```

Each generator builds its own cell and rebuilds it when called again, so place the cell once with `sref()` and re-run the generator whenever the content changes (Chapter 18). The sheet's distances are in millimetres.

```
import os
from hlayout.scripting.api import Session
from hlayout.core.units import mm_to_dbu

s = Session.new("DEMO")
s.new_cell("ASSEMBLY")
s.rect(10, 0, 0, 25400, 12700)

sheet = s.generate_drawing_sheet("DWGSHEET", 62, size="A4", orientation="landscape",
                                 title="NOTCH FILTER ASSEMBLY", dwg_no="HL-0042",
                                 issue="B", drawn_by="A. BROWN",
                                 drawn_date="2026-10-08")
pinout = s.generate_pin_function_table("PINOUT_1", 62, pin_count=8,
                                       pins_per_group=4,
                                       functions={1: "IN+", 2: "IN-", 4: "V-",
```

```
                                                    7: "V+", 8: "OUT"})
print(sheet, s.bbox(sheet))
print(pinout, s.bbox(pinout))

s.new_cell("DRAWING_1")
s.sref(sheet, (0, 0))
s.sref("ASSEMBLY", (mm_to_dbu(40), mm_to_dbu(120)))
s.sref(pinout, (mm_to_dbu(200), mm_to_dbu(120)))

page = s.print_cell("DRAWING_1", out_path="drawing.html", scale=1.0)
print(os.path.basename(page), os.path.getsize(page) > 0)
```

Output

```
DWGSHEET (9825.0, 9825.0, 287175.0, 200175.0)
PINOUT_1 (-10.0, -1080.0, 5610.0, 440.0)
drawing.html True
```

`print_cell()` only writes the HTML page; it does not open a print dialog. Open the file in a browser and print from there, or use `PRINT` in the command bar, which does both.

19.18 Manufacturing export

```
s.export_gerber(out_prefix, cell_name=None, layers=None) -> (written, skipped_text)
s.run_gerber_export_batch(jobs=None, cell_name=None)
                                          -> [(job, written, skipped_text), ...]
s.export_dxf(path, cell_name=None, layers=None)  -> (written_layers, entity_count)
s.run_dxf_export_batch(jobs=None, cell_name=None)
                                    -> [(job, written_layers, entity_count), ...]
s.export_freecad(path, cell_name=None, layer_thickness=None,
                 component_models=None, loop_height_um=None)
s.match_freecad_components(cell_name=None, step_dir=None)  -> MatchReport
s.generate_component_library(out_dir, path=None, ceramic_caps=None,
                             pkg20=None, sot_packages=None) -> macro path
```

Gerber export writes one file per visible layer that has data, named `<out_prefix>_L<NN>_<layer name>.gbr`. Placed cells are flattened and text is skipped. `layers` restricts the export to a list of layer numbers.

```
import glob, os
from hlayout.scripting.api import Session

s = Session.new("DEMO")
s.set_layer_style(1, name="Conductor")
s.set_layer_style(3, name="Resistor")
s.new_cell("PAD"); s.rect(1, -250, -250, 250, 250)
s.new_cell("TOP")
s.sref("PAD", (0, 0)); s.sref("PAD", (3000, 0))
s.path(1, [(0, 0), (3000, 0)], width=150)
s.rect(3, 1000, -300, 2000, 300)
s.text(4, (0, 1000), "TOP")

os.makedirs("fab", exist_ok=True)
written, skipped_text = s.export_gerber("fab/board")
print(sorted(os.path.basename(f) for f in glob.glob("fab/*.gbr")), skipped_text)

s.export_gerber("fab/cond_only", layers=[1])
```

```
print(sorted(os.path.basename(f) for f in glob.glob("fab/cond_only*")))
s.save_design("board.gds")               # keeps the layer names
```

Output

```
['board_L01_Conductor.gbr', 'board_L03_Resistor.gbr'] 1
['cond_only_L01_Conductor.gbr']
```

DXF export writes a single file with one DXF layer per HLayout layer. Text is included.

```
import os
from hlayout.scripting.api import Session

s = Session.open_design("board.gds")
s.use_cell("TOP")
layers, entities = s.export_dxf("board.dxf")
print(layers, entities, os.path.getsize("board.dxf") > 0)
```

Output

```
[1, 3, 4] 3 True
```

A batch runs several export jobs in one call. With no argument it runs the jobs saved by Tools → Gerber Export Sets... or Tools → DXF Export Sets...

```
import os
from hlayout.scripting.api import Session

s = Session.open_design("board.gds")
s.use_cell("TOP")
os.makedirs("out", exist_ok=True)

gerber_jobs = [
    {"name": "Conductor print", "out_prefix": "out/cond", "layers": [1]},
    {"name": "Resistor print",  "out_prefix": "out/res",  "layers": [3]},
]
for job, written, skipped in s.run_gerber_export_batch(gerber_jobs):
    print(job.name, len(written))

dxf_jobs = [{"name": "Mechanical", "path": "out/mech.dxf", "layers": [1, 3]}]
for job, layers, entities in s.run_dxf_export_batch(dxf_jobs):
    print(job.name, layers, entities)
print(sorted(os.listdir("out")))
```

Output

```
Conductor print 1
Resistor print 1
Mechanical [1, 3] 2
['cond_L01_Conductor.gbr', 'mech.dxf', 'res_L03_Resistor.gbr']
```

`export_freecad()` writes a macro that rebuilds the design as a 3D solid stack when run inside FreeCAD (section 14.3). `layer_thickness` overrides the saved film thicknesses, in microns, for this export only. `generate_component_library()` writes the macro that builds the STEP models for placed components.

```
import os
from hlayout.scripting.api import Session

s = Session.open_design("board.gds")
s.use_cell("TOP")

s.export_freecad("board.FCMacro", layer_thickness={1: 12, 3: 20})
print(os.path.getsize("board.FCMacro") > 0)
```

```
macro = s.generate_component_library("step_models")
print(os.path.basename(macro))
```

Output

```
True
build_component_library.FCMacro
```

`match_freecad_components()` reports which placed cells match a STEP model in a component library folder, by name (an EIA size such as 0805 in the cell name) or by pad pattern. The folder must already hold the `.step` files that the `generate_component_library()` macro builds. A placed cell counts as a component when it draws on a layer that has no film thickness. Pass its `component_models()` to `export_freecad()` to use the matches for one export without saving them.

```
import os
from hlayout.scripting.api import Session
from hlayout.io import techfile

s = Session.new("DEMO")
techfile.save_layer_thickness(s.library, {1: 12})  # layer 1 is the only printed film
s.new_cell("0805")
s.rect(1, -1000, -625, -400, 625, pin="1")
s.rect(1,   400, -625, 1000, 625, pin="2")
s.rect(50, -1000, -625, 1000, 625)                 # body, on a non-film layer
s.new_cell("TOP")
s.sref("0805", (0, 0), label="C1")

os.makedirs("step_models", exist_ok=True)
open("step_models/CAP_0805.step", "w").close()     # stands in for a built model

report = s.match_freecad_components(step_dir="step_models")
for line in report.summary_lines():
    print(line)
s.export_freecad("matched.FCMacro", component_models=report.component_models())
print(os.path.getsize("matched.FCMacro") > 0)
```

Output

```
Matched 1 component cell(s):
  0805 -> CAP_0805  (matched by name)
True
```

19.19 Thermal analysis

```
s.thermal_parts(cell_name=None)   -> [{'ref_des','kind','cell','power_mw','source'}, ..]
s.set_thermal_power(ref_des, power_mw)   power_mw=None clears it
s.run_thermal(cell_name=None, grid=None, base_temp=None, rules=None) -> ThermalResult
s.export_thermal_report(result, path)    -> path (HTML)
s.export_thermal_csv(result, path)       -> path
s.draw_thermal_contours(result, layer, step=5.0, which=None, width=None,
                        cell_name=None)  -> number of paths drawn
```

`run_thermal()` solves steady-state heat conduction for the current cell and returns how hot each resistor, die and powered component runs (Chapter 20). It needs a substrate outline, which it finds on a layer whose name contains "substrate". A resistor with no power set uses its Resistor Designer power, matched by reference designator and, failing that, by the row its cell was generated from. The result's `parts` are `PartResult` objects with `ref_des`, `kind`, `power_mw`, `t_max`, `t_avg`, `rise`, `limit` and `ok`. Thermal analysis needs numpy, scipy and shapely.

```
from hlayout.scripting.api import Session
```

```
s = Session.new("DEMO")
s.set_layer_style(10, name="Substrate")
s.new_cell("TOP")
s.rect(10, 0, 0, 10000, 6000)                    # 10 x 6 mm substrate outline
s.generate_resistor(3000, 3000, body_width=1000, body_length=2000,
                    ref_des="R1", resistor_layer=30)
s.generate_resistor(7000, 3000, body_width=1000, body_length=2000,
                    ref_des="R2", resistor_layer=30)
s.use_cell("TOP")

s.set_thermal_power("R1", 100)                   # mW
s.set_thermal_power("R2", 25)
for part in s.thermal_parts():
    print(part)

result = s.run_thermal(grid=250)                 # 250 um grid
for p in result.parts:
    print(f"{p.ref_des} {p.power_mw:.0f} mW  max {p.t_max:.2f} C  "
          f"rise {p.rise:.2f} C  ok={p.ok}")
print(f"hottest {result.max_temp:.2f} C, heat balanced: {result.energy_error < 1e-6}")
s.save_design("thermal_demo.gds")
```

Output

```
{'ref_des': 'R1', 'kind': 'resistor', 'cell': 'R1', 'power_mw': 100.0, 'source': 'thermal'}
{'ref_des': 'R2', 'kind': 'resistor', 'cell': 'R2', 'power_mw': 25.0, 'source': 'thermal'}
R1 100 mW  max 26.15 C  rise 1.15 C  ok=True
R2 25 mW  max 25.29 C  rise 0.29 C  ok=True
hottest 26.15 C, heat balanced: True
```

The boundary conditions, limits and film materials are a rule set saved with the technology, read and written with `techfile.load_thermal_rules()` and `save_thermal_rules()`. Passing `rules=` to `run_thermal()` tries a change without saving it, which makes a substrate comparison a loop.

```
from hlayout.scripting.api import Session
from hlayout.io import techfile

s = Session.open_design("thermal_demo.gds")
s.use_cell("TOP")
s.set_thermal_power("R1", 150)

rules = techfile.load_thermal_rules(s.library)
print(sorted(rules))
print(rules["limits"])

for material in ("alumina_96", "aln"):
    rules["boundary"]["substrate_material"] = material
    r1 = s.run_thermal(grid=250, rules=rules).parts[0]
    print(f"{material:<11} R1 rise {r1.rise:.2f} C")

rules["boundary"]["substrate_material"] = "alumina_96"
rules["boundary"]["base_temp_c"] = 85.0          # hot baseplate
rules["limits"]["resistor_max_c"] = 86.0         # a deliberately tight limit
techfile.save_thermal_rules(s.library, rules)

result = s.run_thermal(grid=250)
print([(p.ref_des, round(p.t_max, 1), p.ok) for p in result.parts])
print([p.ref_des for p in result.failures])
```

Output

```
['boundary', 'limits', 'materials', 'parts', 'powers']
{'resistor_max_c': 150.0, 'die_max_c': 125.0, 'component_max_c': 125.0, 'resistor_max_w_mm2':
```

```
0.08}
alumina_96  R1 rise 1.72 C
aln         R1 rise 0.48 C
[('R1', 86.7, False), ('R2', 85.3, True)]
['R1']
```

A bare die or a component joins the model once it has a power. It is built up as an attach layer and a body above the films; the defaults are 380 µm of silicon on 40 µm of silver-filled epoxy for a die and a ceramic block on solder for a component, and the `parts` section of the rules overrides them per cell or reference designator.

```
from hlayout.scripting.api import Session
from hlayout.io import techfile

s = Session.open_design("thermal_demo.gds")
s.new_cell("OPAMP")
s.rect(50, -600, -500, 600, 500)                # 1.2 x 1.0 mm die outline
s.use_cell("TOP")
s.sref("OPAMP", (5000, 1200), label="U1")
s.set_thermal_power("U1", 400)

rules = techfile.load_thermal_rules(s.library)
rules["parts"]["U1"] = {"kind": "die", "body_material": "silicon", "body_um": 380,
                        "attach_material": "epoxy_ag", "attach_um": 40}
rules["boundary"]["base_temp_c"] = 25.0
rules["limits"]["resistor_max_c"] = 150.0
techfile.save_thermal_rules(s.library, rules)

result = s.run_thermal(grid=250)
u1 = next(p for p in result.parts if p.ref_des == "U1")
under = result.temperature_at((5000, 1200), which="substrate 5")
print(f"U1 junction {u1.t_max:.1f} C, substrate under it {under:.1f} C")
print(result.slice_names())

# the same die on a non-conductive epoxy
rules["parts"]["U1"]["attach_material"] = "epoxy"
worse = next(p for p in s.run_thermal(grid=250, rules=rules).parts if p.ref_des == "U1")
print(f"on plain epoxy: {worse.t_max:.1f} C")
s.save_design("thermal_demo.gds")
```

Output

```
U1 junction 38.0 C, substrate under it 30.9 C
['substrate 1', 'substrate 2', 'substrate 3', 'substrate 4', 'substrate 5', '30: L30', '31:
L31', '32: L32', 'part attach', 'part body (lower)', 'part body (top)']
on plain epoxy: 63.6 C
```

The result can be written as an HTML report or a CSV, and its isotherms drawn onto a layer as paths so they print with the assembly drawing.

```
import os
from hlayout.scripting.api import Session

s = Session.open_design("thermal_demo.gds")
s.use_cell("TOP")
result = s.run_thermal(grid=250)

s.export_thermal_report(result, "thermal.html")
s.export_thermal_csv(result, "thermal.csv")
print(os.path.getsize("thermal.html") > 0)
print(open("thermal.csv").read().splitlines()[:2])
```

```
drawn = s.draw_thermal_contours(result, 56, step=5.0)
print(drawn > 0, {shape.layer for shape in s.cell.shapes[-drawn:]})
for line in result.summary_lines()[:3]:
    print(line)
```

Output

```
True
['ref_des,kind,cell,power_mW,area_mm2,density_W_per_mm2,t_max_C,t_avg_C,rise_C,limit_C,status'
, 'U1,die,OPAMP,400.000,1.2000,0.33333,37.97,37.96,12.97,125.0,OK']
True {56}
thermal: 5080 unknowns, grid 40 x 24 at 250 um, 11 slices
  power in 525.0 mW, out 525.0 mW (base 524.8, top 0.2); energy balance error 0.000%
  hottest point 38.0 C (13.0 C above reference)
```

Because the problem is linear, one solve per part gives every part's temperature for any set of powers. `build_thermal_model()` returns the model `run_thermal()` solves; its `coupling()` method returns that table.

```
from hlayout.scripting.api import Session
from hlayout.core.thermal_build import build_thermal_model

s = Session.open_design("thermal_demo.gds")
model = build_thermal_model(s.library, "TOP", grid_um=250)
coupling = model.coupling()

print(coupling.names)
for name, row in zip(coupling.names, coupling.matrix()):
    print(name, [round(v, 2) for v in row])          # deg C rise per watt

# what if U1 ran at 1 W and the resistors were off?
temps = coupling.temperatures({"U1": 1.0})
print({name: round(t_max, 1) for name, (t_max, t_avg) in temps.items()})
```

Output

```
['R1', 'R2', 'U1']
R1 [11.45, 0.01, 0.19]
R2 [0.01, 11.45, 0.19]
U1 [0.04, 0.04, 32.42]
{'R1': 25.2, 'R2': 25.2, 'U1': 57.4}
```

19.20 A complete example

This script builds a small two-resistor divider on a ceramic substrate from an empty library: layer stack, substrate, three lands, two thick-film resistors, routing, a design rule check, a netlist, and Gerber output.

```
import os
from hlayout.scripting.api import Session
from hlayout.io import techfile

s = Session.new("DIVIDER")

# --- layer stack ------------------------------------------------------
stack = ((10, "Substrate", "#808080"), (30, "Resistor", "#202020"),
         (31, "Conductor", "#d4a017"), (32, "Glaze", "#3cb371"),
         (60, "DRC", "#ff0000"))
for number, name, color in stack:
    s.set_layer_style(number, name=name, color=color)

# --- a reusable land --------------------------------------------------
s.new_cell("LAND")
s.rect("Conductor", -500, -500, 500, 500, pin="1")
```

```
# --- the board ---------------------------------------------------------
s.new_cell("TOP")
with s.batch():
    s.rect("Substrate", 0, 0, 16000, 6000)
    s.convert_rects_to_outline("Substrate", width=100)
    for ref, x in (("VIN", 2000), ("VOUT", 9000), ("GND", 14000)):
        s.sref("LAND", (x, 3000), label=ref)

# 1 kohm/sq ink, 0.3 mm wide: 10 k is 10 squares, 4.7 k is 4.7 squares
for ref, ohms, x in (("R1", 10_000, 5500), ("R2", 4_700, 11500)):
    s.generate_resistor(x, 3000, body_width=300,
                        body_length=round(ohms / 1000 * 300),
                        ref_des=ref, resistor_layer="Resistor",
                        conductor_layer="Conductor", glaze_layer="Glaze")

# join each resistor pad to the nearest land
s.use_cell("TOP")
ends = []
for ref in s.cell.srefs:
    if ref.label.startswith("R"):
        x1, y1, x2, y2 = s.bbox(ref.cell_name)
        ends += [(ref.origin[0] + x1 + 100, 3000), (ref.origin[0] + x2 - 100, 3000)]
lands = [r.origin for r in s.cell.srefs if r.cell_name == "LAND"]
for end in ends:
    land = min(lands, key=lambda p: abs(p[0] - end[0]))
    s.path("Conductor", [end, land], width=300)

# --- checks ------------------------------------------------------------
violations = s.run_drc([
    {"kind": "MIN_WIDTH", "layer_a": 31, "value": 200, "name": "C.W.1"},
    {"kind": "MIN_SPACE", "layer_a": 31, "value": 200, "name": "C.S.1"},
], error_layer="DRC")
print("DRC violations:", len(violations))

for net in s.extract_netlist(["Conductor"]):
    print(net.id, sorted(f"{p.ref_des}.{p.pin}" for p in net.pins))

# --- output ------------------------------------------------------------
s.save_design("divider.gds")
os.makedirs("divider_fab", exist_ok=True)
written, _ = s.export_gerber("divider_fab/divider")
print(sorted(os.listdir("divider_fab")))
print(s.bbox(), [c["name"] for c in s.list_library()])
```

Output

```
DRC violations: 0
NET1 ['R1.1', 'VIN.1']
NET2 ['R1.2', 'R2.1', 'VOUT.1']
NET3 ['GND.1', 'R2.2']
['divider_L10_Substrate.gbr', 'divider_L30_Resistor.gbr', 'divider_L31_Conductor.gbr',
'divider_L32_Glaze.gbr']
(-50.0, -50.0, 16050.0, 6050.0) ['LAND', 'TOP', 'R1', 'R2']
```

19.21 Method index

The `Session` methods covered in this chapter, with the section that shows each one in use.

Method	What it does	Section
`Session.new`	Start an empty library	19.2

Method	What it does	Section
`Session.open`	Open a GDSII file	19.2
`Session.open_design`	Open a GDSII file and its sidecar	19.2
`save`	Write a bare GDSII file, optionally renaming the library	19.2
`save_design`	Write the GDSII file and its sidecar	19.2
`new_cell`	Create a cell and make it current	19.3
`use_cell`	Make an existing cell current	19.3
`cell, current_cell_name`	The current cell object and its name	19.3
`list_library`	List the cells of the design or an attached library	19.3, 19.9
`find_cells`	Search cell names in the design and attached libraries	19.3, 19.9
`bbox`	Bounding box of a cell, placed instances included	19.3
`set_layer_style`	Set a layer's name, color, stipple, fill, visibility	19.4
`load_technology`	Apply a technology file to the design	19.4
`save_technology`	Write the layer stack and parameters to a technology file	19.4
`show_layers_of_cell`	Make every layer a cell draws on visible	19.4
`rect`	Draw a rectangle	19.5
`polygon`	Draw a polygon (up to 200 points)	19.5
`circle`	Draw a circle as an 8, 16, 32 or 64-sided polygon	19.5
`path`	Draw a path with a width and end cap	19.5
`text`	Draw a text label	19.5
`shapes_on_layer`	Shapes of the current cell on one layer	19.6
`remove`	Delete a shape or instance	19.6, 19.7
`delete_last`	Delete the most recently added shape or instance	19.6
`clear_layer`	Delete every shape on a layer	19.6
`remap_layers`	Move shapes from one layer to another in bulk	19.6
`convert_rects_to_outline`	Replace filled shapes with stroked outlines	19.6
`sref`	Place one instance of a cell	19.7
`aref`	Place an array of instances	19.7
`smash_aref`	Explode an array into single instances	19.7
`undo, redo`	Step the undo history	19.8
`can_undo, can_redo`	Whether a step is available	19.8
`batch`	Group several edits into one undo step	19.8
`checkpoint`	Push an undo point before editing the library directly	19.8
`clear_undo_history`	Discard the undo history	19.8
`on_change`	Register a callback run after every edit	19.8
`attach_library`	Attach a GDSII file as a reference library	19.9
`detach_library`	Detach a reference library	19.9
`list_attached_libraries`	List the attached libraries	19.9
`place_cell`	Find a cell by name and place an instance	19.9
`place_cell_array`	Find a cell by name and place an array	19.9
`create_via_cell`	Via Wizard: add a via cell to an attached library	19.9

Method	What it does	Section
create_pad_cell	Pad Wizard: add a pad cell to an attached library	19.9
create_chip_cell	Start a bare-die footprint cell	19.9
export_cell_to_library	Copy a cell into an attached library and save it	19.9
merge_gds	Merge two GDSII files into a third	19.9
list_pins	List the pads of a cell with their pin numbers	19.10
set_pin	Set or clear a pad's pin number	19.10
drop_via	Draw a via in the current cell	19.11
auto_route	Route a clear orthogonal trace between two points	19.11
wire_bond	Draw a wire bond	19.12
find_wire_bond	Find the wire bond nearest a point	19.12
move_wire_bond	Move one end of a wire bond	19.12
delete_wire_bond	Delete a wire bond	19.12
generate_resistor	Build and place a thick-film resistor	19.13
generate_resistor_table	Build the resistor BOM table cell	19.13
mask_op	Oversize, undersize or boolean-combine layers	19.14
run_mask_op_batch	Run an ordered mask-operation recipe	19.14
generate_dielectric	Fill the dielectric layer around its openings	19.14
clear_dielectric	Remove the generated dielectric fill	19.14
mask_op_top_view	Draw the hidden-line assembly view	19.14
run_drc	Run design rule checks	19.15
extract_netlist	Extract the netlist from copper connectivity	19.16
export_netlist_csv	Write a netlist to a CSV file	19.16
generate_drawing_sheet	Build a drawing sheet and title block cell	19.17
generate_pin_function_table	Build a pinout table cell	19.17
print_cell	Render a cell as a printable HTML page	19.17
export_gerber	Export Gerber RS-274X, one file per layer	19.18
run_gerber_export_batch	Run several Gerber export jobs	19.18
export_dxf	Export a DXF file	19.18
run_dxf_export_batch	Run several DXF export jobs	19.18
export_freecad	Write the FreeCAD 3D macro	19.18
match_freecad_components	Match placed parts to STEP models	19.18
generate_component_library	Write the macro that builds the STEP component library	19.18
thermal_parts	List the parts that can be given a power	19.19
set_thermal_power	Set or clear the power a part dissipates	19.19
run_thermal	Steady-state thermal analysis of a cell	19.19
export_thermal_report	Write the thermal result as an HTML report	19.19
export_thermal_csv	Write the thermal parts table to a CSV file	19.19
draw_thermal_contours	Draw the result's isotherms onto a layer	19.19

20. Thermal Analysis

Thermal Analysis estimates how hot each printed resistor, bare die and powered component runs, from the layout, the film stack and a power figure per part. It solves steady-state heat conduction through the substrate and the printed films and reports each part's temperature against a limit. Open it from Tools → Thermal Analysis...; the same analysis is available from the command bar (§20.5) and from Python (§19.19). It needs numpy, scipy and shapely.

20.1 What the model includes

The hybrid is treated as a stack of flat layers on one grid, bottom to top:

- An optional mounting layer under the substrate (epoxy, solder or grease).
- The substrate, over the area of its outline.
- Each printed film in layer-number order, which is HLayout's print order: conductor, dielectric, resistor ink and glaze. A film is present only where its layer has geometry, so a resistor printed straight onto the substrate is in contact with the substrate.
- For each powered die or component, an attach layer and a body.

Heat is generated in each resistor's ink between its terminations, on the top face of each die, and through the body of each component. It leaves through the base of the stack, which is either held at a fixed temperature or cooled through a heat-transfer coefficient, and optionally through the top surface.

What the model leaves out:

- Conductivity does not change with temperature. Alumina and silicon conduct less well when hot, so a large rise is slightly underestimated.
- Conductor traces do not heat themselves; only the parts you give a power dissipate.
- Components are plain blocks on their footprint. A die or component with no power and no part definition is left out of the model entirely.
- Airflow, enclosures, wire bonds and leads are not modelled, and the analysis is steady state only.

Treat the result as a design-stage estimate. It has been checked against closed-form conduction cases but not yet against a finite-element tool or a measured hybrid.

20.2 What it reads from the design

Needed	Where it comes from
Substrate outline	The layer whose name contains "substrate", "alumina" or "ceramic", or the layer set on the Boundary tab. A closed outline path counts as the area it encloses. With no outline, the films' bounding box plus 0.5 mm is used and a note says so.
Which layers are films	Resistor Geometry Rules (ink, conductor, glaze), Route Layers (conductors) and Dielectric Fill Rules, plus any layer given a material on the Stack tab.
Film thickness	Setup → Technology Properties → Thicknesses, the same values FreeCAD export uses.
Resistors	Every placed cell that has ink on a resistor layer. Its reference designator is the instance's Label.
Resistor power	A power set on the Powers tab; otherwise the Resistor Designer row with that reference designator; otherwise the row the resistor's cell was generated from, which covers a resistor relabelled on the layout (shown as "cell" in the From column). If several rows share that name, the highest power is used.

Needed	Where it comes from
Dice and components	Placed cells that draw on non-film layers. A cell that places chip_pin cells is treated as a die.

20.3 The Thermal Analysis window

The left side holds the setup in four tabs. Every edit is saved straight into the design's technology parameters, so it is kept by File → Save GDS... and by Save Technology File...

- **Powers.** One row per resistor, die and component, with its power in mW and where that power came from. Select a row and choose Set Power..., or double-click it. Clear returns a resistor to its Resistor Designer power.
- **Stack.** The substrate and each film layer with its role, material, conductivity and thickness. Set Material... takes a material name or a conductivity in W/m·K; Role Default removes the override.
- **Boundary.** Substrate material and thickness, the base condition, ambient temperature, the top-surface coefficient, an optional mounting layer, and the grid pitch. Press Apply to save.
- **Limits.** The temperatures and resistor power density above which a part is reported as over limit.

Run Analysis solves the current cell. The right side then shows:

- **Temperature map.** Drawn in the classic thermal-simulator layout: solid colour bands from purple (coolest) through blue, teal, green, yellow, orange and red to white (hottest), inside a frame with Length [mm] and Width [mm] axes, and a LEGEND on the right giving the lowest value of each band, coolest at the top. The Slice list switches between the top surface, which is the topmost material at every point, and any single layer of the stack. Moving the pointer over the map reads out the position, the temperature and the rise above the base.
- **Map options.** Above the map: Style — Banded, or Smooth for a continuous blend; Bands — 3 to 20, default 10; Show — Rise above the base (or ambient, for a cooled base) or actual Temperature; Spacing — Closer at cool end (the default, which squares the fraction across the range so most of the board gets several bands and the hot spots still get their own colours) or Even. The choices are saved with the design and used by the report too.
- **Parts.** One row per part, with failures first and tinted red: power, power density, peak and average temperature, rise above the base, limit and status. Selecting a row outlines that part on the map.
- **Status line.** The verdict, the hottest temperature, total power, the heat-balance error, the grid size and the solve time. The heat leaving through the boundaries should equal the total power; an error above about 0.1% means the result should not be trusted.

Report... writes an HTML report — the parts table plus the top-surface and substrate maps, each drawn with the same bands, legend and axes as the window and on one shared scale so the two can be compared — CSV... writes the parts table, and Draw Contours... draws the isotherms of the displayed slice onto a layer of the layout as paths, in one undoable step.

20.4 Default settings and materials

Setting	Default
Substrate	96% alumina, 635 µm
Base	Fixed at 25 °C
Ambient	25 °C
Top surface	10 W/m²·K to ambient; 0 seals the top

Setting	Default
Mounting layer	None
Grid pitch	Automatic: 100 µm, coarsened in 25 µm steps to keep a layer at or under 40,000 cells
Resistor film limit	150 °C
Resistor power density limit	0.08 W/mm² (about 50 W/in²)
Die junction limit	125 °C
Component limit	125 °C
Die build-up	380 µm silicon on 40 µm silver-filled epoxy
Component build-up	600 µm ceramic body on 50 µm solder

The named materials and the conductivity each one stands for are listed below. They are approximate general values intended as starting points. Thick-film paste conductivity in particular varies with the paste and the firing profile, so replace them with your own data wherever it matters.

Material name	Description	Conductivity (W/m·K)
`alumina_96`	96% alumina	24
`alumina_99`	99.6% alumina	30
`aln`	Aluminium nitride	170
`beo`	Beryllia	250
`conductor`	Fired thick-film conductor	80
`dielectric`	Thick-film dielectric	3
`resistor`	Resistor ink	2
`glaze`	Glaze or overglaze	1
`silicon`	Silicon die	130
`gaas`	Gallium arsenide die	46
`ceramic_chip`	Ceramic chip component	5
`epoxy_ag`	Silver-filled epoxy	2.5
`epoxy`	Non-conductive epoxy	0.5
`solder`	Solder	50
`grease`	Thermal grease	1
`kovar`	Kovar package base	17
`copper`	Copper	390
`aluminium`	Aluminium	200

20.5 Command bar

```
THERMAL PARTS [cell=<name>]
THERMAL POWER <ref_des> <mW>
THERMAL POWER <ref_des> CLEAR
THERMAL SET <key> <value>
THERMAL MATERIAL <layer> <material|k>
THERMAL MATERIAL <layer> CLEAR
THERMAL MATERIALS
THERMAL RUN [cell=<name>] [grid=<dbu>] [base=<degC>]
```

```
THERMAL REPORT out=<path.html> [csv=<path.csv>]
THERMAL CONTOURS <layer> [step=<degC>] [slice=<name>]
```

THERMAL SET changes one Boundary or Limits setting by its key: base_temp_c, ambient_c, base_mode (fixed or htc), base_htc, top_htc, substrate_material, substrate_um, substrate_layer, mount_material, mount_um, grid_um, resistor_max_c, die_max_c, component_max_c or resistor_max_w_mm2. THERMAL RUN prints each part's temperature, hottest first; REPORT and CONTOURS then act on that run.

```
THERMAL POWER U1 400
THERMAL SET base_temp_c 85
THERMAL MATERIAL 31 conductor
THERMAL RUN grid=250
THERMAL REPORT out=thermal.html csv=thermal.csv
CLEAR 56
THERMAL CONTOURS 56 step=5
```

20.6 Getting a useful answer

- Set the base temperature to the hottest case or baseplate temperature the hybrid will see. Every temperature in the result moves up by the same amount.
- Powers default to each resistor's rated design power, which is a worst case. Enter actual dissipation where you know it.
- A die's junction temperature is dominated by its attach layer. Set its material and thickness for the adhesive you use (§19.19 shows how).
- If the peak temperature changes by more than a few percent when the grid pitch is halved, use the finer grid.
- Features narrower than the grid pitch can drop out of the model. Parts that do are listed in a note after the run.

21. Autorouting with Freerouting

HLayout hands routing to Freerouting, a free, open-source autorouter, through the Specctra file formats. HLayout writes the board as a design file (.dsn), Freerouting routes it and writes a session file (.ses), and HLayout reads the tracks and vias back as one undo step, then checks every net. This works for two-conductor multilayer thick-film designs, where a via is a window in the printed dielectric, and for one-conductor chip-and-wire hybrids, where bare dice are wire-bonded to the substrate (§21.5).

21.1 Installing Freerouting

- Download Freerouting from freerouting.app and install it.
- Use the **Java build** (the freerouting.exe launcher in the installed app folder, or freerouting-executable.jar). It routes the full number of passes. A .jar needs Java 25 or later; give its java.exe in the Java field if it isn't found automatically.
- The **native CLI build** (freerouting-cli.exe) also works, but it crashes on multi-pass runs, so HLayout limits it to a single pass. Expect fewer nets completed.
- Pick the launcher in the Autoroute window (Launcher / .jar). The path is remembered for every design.

21.2 Before you route

- **Route layers.** Setup → Route Layers sets the 1st and 2nd conductor, the via cell and the dielectric layer.
- **Numbered pads.** Every pad the router should reach needs a pin number (Edit → Number Pins...). Pads inside a placed part are named RefDes.Pin (U1.3). Pads drawn directly in the routed cell, such as module I/O pads, are IO.<pin>. Die pads and header pins are named by their part (ML1.3, J1.19).
- **Net names.** The router has to know which pads belong together. Import them from a CSV with a net column and either a pad column (U1.3) or ref and pin columns: Import Net Names... in the window, NETNAMES IMPORT <file.csv>, or Session.import_net_names(). Edit → Highlight Net then reports the name of any net you click.
- **Board outline.** The routing area is the extent of the substrate layer (a layer named substrate, outline or boundary), including a substrate drawn as its own placed cell. Without one, it is the pads and copper grown by 1 mm.

21.3 The Autoroute window

Tools → Autoroute (Freerouting)... opens the window. The rules are saved with the design; the router paths are saved for all designs.

Field	What it does
Launcher / .jar, Java	The Freerouting program, and the Java runtime for a .jar. The status line says whether it is ready.
Signal / Power width	Track width for ordinary nets and for the power nets.
Clearance	Minimum gap between copper of different nets.
Signal / Power via, window	Via pad size and its dielectric window, for signal and power nets.
Max passes	The router's pass limit (the native CLI build always does one).
Bond finger	Chip-and-wire: size of the square finger laid where a wire bond lands on the substrate (default 0.4 mm).

Field	What it does
Die keep-out	Chip-and-wire: how far tracks keep clear of a bare die and its attach epoxy (default 0.2 mm).
Power nets	Nets routed at the power width with power vias (default VCC, GND).
Leave unrouted	Nets the router must not touch.
90-degree routing only	Horizontal and vertical tracks only.
Keep existing tracks and vias	Existing routing is kept and protected; otherwise it may be ripped up and rerouted.
Remove ALL existing tracks and vias	Deletes every track and via on the route layers first and routes from scratch.
No tracks between a part's own pads	Closes the area under a chip capacitor and across a printed resistor's ink and glaze.
Layers	Auto (one layer when nothing is drawn on the 2nd conductor), 1st conductor only, or both.
Route / Stop	Run Freerouting in the background and import the result; Stop ends the run.
Export DSN... / Import SES...	The two halves by hand, for running Freerouting yourself (its GUI, for instance).

21.4 What HLayout tells the router

- **Your design rules win.** The clearance is raised to the largest MIN_SPACE or MIN_SPACE_BETWEEN on the route layers in Tools → DRC, and the track widths to the largest MIN_WIDTH, so the routing passes your own DRC.
- **Two conductors.** The 2nd conductor is the top layer. A via is a pad on both conductors plus its window in the dielectric. Outside the printed dielectric the conductors aren't insulated, so only the 1st conductor may run there and no via may be placed.
- **I/O pads.** A module I/O pad may only be entered within its first 0.3 mm from the inner edge; the rest of the pad is an obstacle.
- **Keep-outs.** Dice, part bodies and the edge band are sent as netless pads rather than Specctra keep-out areas, because the native CLI build crashes on keep-out areas.

21.5 Chip-and-wire hybrids

Freerouting only knows flat tracks and vias; it can't place or route a wire bond, and it doesn't decide which bond finger a die pad goes to. You bond the dice first, and HLayout turns the bonds into something the router understands:

- **Bond fingers.** Every wire bond on the bond layer (Setup → Wire Bond Rules) with one end on a die pad or header pin (the die-pad layers) and the other end on the substrate makes that substrate end a bond finger: a square pin, Bond finger wide, on the net of the pad at the bond's other end. Bonds stitched on the same spot share a finger. A bond from a die straight to a header pin needs no substrate copper.
- **Dice are keep-outs.** The die outline and attach epoxy, grown by Die keep-out, are closed to tracks. An enclosing header package that spans most of the board is not treated as a die.
- **Part bodies.** With No tracks between a part's own pads ticked, the area between a capacitor's terminations and across a printed resistor is closed.

- **One conductor.** Layers = Auto routes a design with nothing on the 2nd conductor on the 1st alone, with no vias and no dielectric.
- **On import** a finger rectangle is laid on the 1st conductor at every landing that isn't already covered by copper. The completion check counts each landing as a pin of its net, so a die pad only counts as connected when the copper under its bond reaches the rest of its net.

The workflow:

1. Place the dice, the passives and the header, and draw the substrate.
2. Wire-bond each die pad and header pin to the spot on the substrate where it should land (Create → Wire Bond).
3. Name the nets, including the die pads and header pins (ML1.3, J1.19).
4. Route: Tools → Autoroute (Freerouting)..., Remove ALL ticked for a fresh start, then Route.
5. Check: Tools → DRC, and Tools → Extract Netlist with wire bonds included.

21.6 Reading the result

- The log reports the tracks and vias imported, the bond fingers laid, and "Nets complete: N of M" with the names of any net still unrouted.
- **Neck-downs.** Freerouting sometimes narrows a track where it squeezes into a pad. HLayout widens it back to the minimum width on import and says how many it changed; run Tools → DRC to check that the wider track still has its clearance.
- Use Edit → Highlight Net (or HILITE <net>) on an unrouted net to see its pads and the pieces already routed.
- The whole import is one undo step.

21.7 Worked example: the notch filter

hlayout/examples/notch_filter.gds is a one-conductor chip-and-wire hybrid: two bare LM741 dice (ML1, ML2) wire-bonded to the substrate, a 20-pin header (J1) bonded to the substrate, six printed resistors and six chip capacitors, all on layer 3. As a test, all of its hand-drawn layer-3 copper was removed and the 17 nets were named from the original connectivity, bond wires included. Freerouting then routed it in about 1.5 seconds:

- 22 bond fingers laid, 39 tracks, 17 of 17 nets complete.
- Extract Netlist (with wire bonds) gives exactly the original connectivity.
- Tools → DRC is clean against the design's 0.3 mm width and spacing rules; no track crosses a die or a part body, and everything stays on the substrate.

The routed design is hlayout/examples/notch_filter_freerouted.gds.

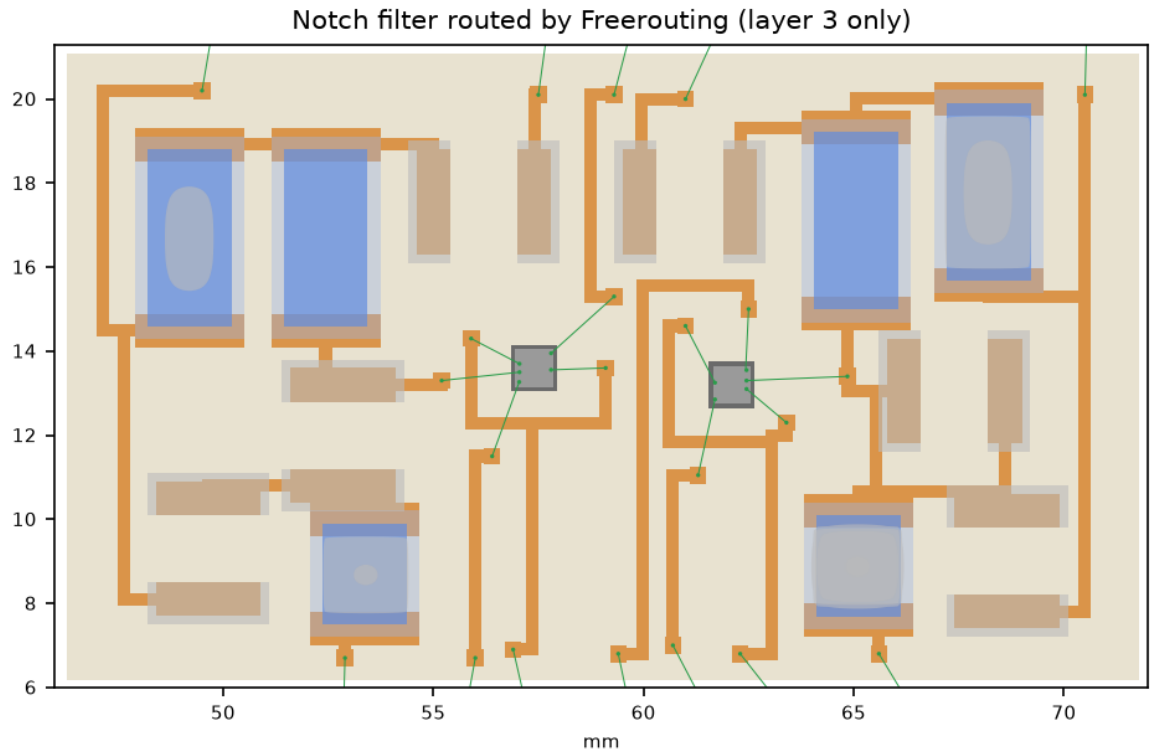


Figure 21.1 – The notch filter routed from scratch by Freerouting: layer-3 tracks and bond fingers (orange), dice (grey), resistor ink and glaze (blue), wire bonds (green).

21.8 Command bar and Python

```
NETNAMES IMPORT nets.csv
FREEROUTE RUN passes=100 fresh=1 layers=auto finger=400
FREEROUTE EXPORT board.dsn
FREEROUTE IMPORT board.ses keep=1

s.import_net_names("nets.csv")
s.set_freerouting_rules(layer_mode="auto", bond_finger=400, die_keepout=200)
r = s.freeroute(remove_existing=True)
print(r["complete"], "of", r["nets"], r["unrouted"], r["fingers"])
s.bond_landings()        # [[x, y, ["ML1.3"]], ...] where each bond lands
```

21.9 Chip-and-wire router results and the bond diagram

HLayout's own chip-and-wire router (the hybrid fork of Freerouting in `router\freerouting`) runs as a separate program, `HybridRouterCli`. It reads a `design.hybrid.json` and writes a `design.routed.json`, and with `--bond-diagram wires.csv` it also writes the bond diagram. HLayout brings the result back as follows:

- **HYBRID IMPORT** `<file.routed.json>` (or Session.import_routed_hybrid()) draws the routed design in its own cell, as one undo step:
  - the substrate;
  - each die and each printed resistor or chip part as a placed cell, labelled with its reference designator;
  - the bond fingers, jumper lands and external pads as numbered pads;
  - the routed traces;

- each dielectric crossover as a pad on both conductors plus its window, followed by the dielectric fill;
- every wire bond and jumper, as a line on the wire-bond layer at the wire diameter.

The import also sets the pad nets as the design's net names and keeps the bond diagram with the design. The router's conductor layers map onto the two Route Layers, its dielectric onto the dielectric layer and its resistor layer onto the resistor layer; `layers=` overrides any of these.

- **The bond diagram** lists every wire in bonding order: die bonds die by die in pad order, then die-to-die wires, then jumpers. Each row gives both ends and their positions, the wire material and diameter, the bond type, the loop height, the length, and the angle the wire leaves each die pad at, measured from the die-edge normal. **HYBRID BONDS** lists it, and **HYBRID BONDS** `<file.csv>` writes it in the router's CSV layout.
- **BONDTABLE** `<cell> <layer>` (Session.generate_bond_table()) draws it as a table. Tools → Assembly Drawing adds the bond table automatically when the design has one; untick Bond diagram table to leave it out.
- Print-layer Gerbers come from the normal Gerber export of the imported cell.

```
HYBRID IMPORT router\examples\single-die-op-amp.routed.json
HYBRID BONDS wires.csv
BONDTABLE BONDS 61

st = s.import_routed_hybrid("design.routed.json")
print(st["traces"], st["wires"], st["jumpers"], st["unrouted"])
s.export_bond_diagram("wires.csv")
s.generate_bond_table("BONDS", 61)
```

21.10 Editing a routed hybrid: bond info, finger moves, resistor resize

After HYBRID IMPORT, three tools work on the imported cell:

- **Hovering a wire bond** (in Select, Move Bond Finger or Resize Resistor) highlights it in yellow. The status bar shows its wire id and net, both ends, length, wire metal and diameter, bond type, loop height, and the angle it leaves the die at. BONDINFO `<x,y>` gives the same from the command bar.
- **Edit → Move Bond Finger:** click a bond finger or jumper land, then click its new centre. Every wire ending on it follows. While you move, each wire is drawn green if it still meets the bond rules (minimum and maximum length, maximum angle off the die-edge normal) and red if not, and the status bar says which rule fails. It also says when the finger has left its printed trace, so that net needs re-routing. The move is one undo step and updates the bond diagram. MOVEFINGER `<pad> <x,y>` does it from the command bar.
- **Edit → Resize Resistor:** click a printed resistor, then move along its length. The new body outline and its value follow the cursor; the value is R = Rs × L / W, where the sheet resistance Rs comes from the value and size in the router file. Click to apply. The pins move with the body ends, and the status bar says by how much, since the traces on them then need re-routing. RESIZE `<refdes> length=<um> [width=<um>]` does it exactly, and RESIZE `<refdes>` on its own reports the length, width, number of squares, value and Rs.

```
BONDINFO 4500,2800
MOVEFINGER F1 4500,2300
RESIZE R1 length=900 preview=1
RESIZE R1 length=900

info = s.bond_wire_info((4500, 2800))
r = s.move_bond_finger((4500, 2500), (4500, 2300))
```

```
print(r["warnings"], r["on_trace"])
s.resize_resistor("R1", length=900)["value_ohms"]   # 1200.0
```